

MCA

COMPLETE EXAM NOTES

Modern Computer Architecture • Compact Revision Edition

30 PAGE REVISION PDF

Included Material

- Boolean Algebra and Logic Gates
- Logic Minimisation and Universal Gates
- Combinational Circuits
- MUX, DEMUX and Decoders
- Half Adder, Full Adder and Ripple Adder
- Number Systems and Two's Complement
- Adder-Subtractor and Overflow
- Latches, Flip-Flops and Timing
- Registers and Shift Registers
- Counters and State Machines
- Register Files and FSMs
- SRAM, DRAM and Computer Architectures
- Verilog Setup and Hardware Implementation
- CISC vs RISC and ISA
- Uploaded lecture + laboratory material

=====

=====

DOCUMENT 1

=====

=====

Lecture: 0110

& State machines

Building Memory That Does Something

Chain flip-flops together and a circuit can store words, count, and follow a plan.

L06 • MODERN COMPUTER ARCHITECTURE

One Flip-Flop = One Bit

L5 gave us a reliable 1-bit memory. Today we build things worth remembering.

The D flip-flop

Captures one bit on the clock edge, holds it for the cycle (L5).

Edge-triggered & safe

Master-slave design

blocks data races — predictable updates.

Timing rules

Setup, hold, metastability

— the constraints that keep it working (L5).

>

But just ONE bit...

A CPU needs 32-bit words, counters, and control. So we chain them.

The jump from 'store a bit' to 'store a word, count events, run a program' is what today is about.

L06 • MODERN COMPUTER ARCHITECTURE

Today's Roadmap

Three structures, each more capable than the last.

1

Registers

n flip-flops in parallel = an n-bit word store. Add shifting for data movement.

→

2

Counters

Flip-flops that count clock pulses

— the basis of timers and addresses.

→

3

State Machines

The general model of any sequential circuit: states, transitions, outputs.

The big idea: an FSM is the universal blueprint — registers and counters are just special cases of it. It's how a CPU 'thinks'.

L06 • MODERN COMPUTER ARCHITECTURE

0

The Flip-Flop

Toolkit

Four flip-flops, four truth tables — and the one reverse-lookup table that makes design possible.

PRIMARY DEFINITION

A flip-flop is a one-bit memory that updates only on a clock edge. Change what feeds its input and you change what it does.

SR

Set / Reset — the original latch

D

Delay — copies its input, one cycle later

JK

The universal one — SR with toggle

T

Toggle — flips on command. Counters live here.

L06 • MODERN COMPUTER ARCHITECTURE

Latch vs Flip-Flop

Both remember a bit. The difference is WHEN they are allowed to change.

CLK / EN

D

Q latch

Q flip-flop

Same clock, same data — two very different outputs. The latch copies D the whole time EN is high; the flip-flop looks once, at the edge.

LEVEL-TRIGGERED — LATCH

Transparent: while the enable is high, Q follows D continuously. Any wobble on D passes straight through.

EDGE-TRIGGERED — FLIP-FLOP

Q samples D at one instant only, then holds for the whole cycle. This is why we can reason about timing at all.

Everything from here on uses edge-triggered flip-flops. When this lecture says 'flip-flop', assume the rising clock edge.

L06 • MODERN COMPUTER ARCHITECTURE

The SR Latch — Where Memory Starts

Two NOR gates feeding each other. The simplest circuit that can remember.

CHARACTERISTIC TABLE

S

R

Q

MEANING

0

0

Q

Hold — remembers the last value

0

1

0

Reset the bit to 0

1

0

1

Set the bit to 1

1

1

—

Forbidden — both outputs go 0

THE FLAW

S = R = 1 breaks the promise that Q and Q' are opposites — and when both inputs drop together the result is unpredictable.

Fixing that forbidden row is exactly what the JK flip-flop is for — and the 1x turns out to give us the toggle we need for counters.

L06 • MODERN COMPUTER ARCHITECTURE

The D Flip-Flop — One Input, No Surprises

Tie S and R together through an inverter and the forbidden state becomes unreachable.

SYMBOL & PIN NAMES

CHARACTERISTIC TABLE

Q

D

Q(n+1)

WHAT HAPPENED

0

0

0

stayed at 0

0

1

1

loaded a 1

1

0

0

loaded a 0

1

1

1

stayed at 1

CHARACTERISTIC EQUATION

Q(n+1) = D — the next state is simply whatever D was at the edge.

WHY IT IS THE DEFAULT

One data input means one thing can go wrong instead of four. Every register, pipeline stage and state register in a modern CPU is built from D flip-flops.

THE COST

A D flip-flop cannot toggle by itself — it only copies. To make it flip, you must feed it the opposite of its own output. Hold that thought.

L06 • MODERN COMPUTER ARCHITECTURE

The JK Latch — Reclaiming the Fourth Row

Same four input combinations as SR, but the illegal one is redefined as 'toggle'.

CHARACTERISTIC TABLE

J

K

Q

MEANING

0

0

0

Q

Hold — no change

0

1

0

Reset to 0

1

0

1

Set to 1

1

1

Q'

TOGGLE — flip to the opposite

CHARACTERISTIC EQUATION

Q(n+1) = JQ'(n) + QK(n)

WHY THIS MATTERS TODAY

The JK flip-flop can do everything SR and D can do, plus one new trick: toggle. Toggling is counting. Tie J and K together and you get the flip-flop the rest of this lecture depends on.

L06 • MODERN COMPUTER ARCHITECTURE

The T Flip-Flop — The One Counters Are Made Of

A single input with a single job: T = 1 means 'flip', T = 0 means 'stay'.

SYMBOL

CHARACTERISTIC TABLE

T

Q

Q

ACTION

0

0

0

hold

0

1

1

hold

1

0

1

flip

1

1

0

0

flip

CHARACTERISTIC EQUATION

Q(n+1) = TQn' + T'Qn = T XOR Qn

TWO WAYS TO BUILD ONE

1 • From a JK flip-flop

Tie J and K to the same wire.

2 • From a D flip-flop

Remember this device. Every counter in the next two sections is a row of T flip-flops with different logic deciding when each one flips.

L06 • MODERN COMPUTER ARCHITECTURE

Hold T at 1 and You Get a Divide-by-2

A permanently toggling flip-flop halves the clock frequency. Chain them and you have a counter.

CLK

Q0 (÷2)

Q1 (÷4)

Q0 completes one cycle for every two clock cycles. Q1 is clocked by Q0, so it completes one cycle for every four. Each extra flip-flop halves the frequency again — and the bits you are looking at spell out 0, 1, 2, 3 in binary.

AS A DIVIDER

n stages divide the clock by 2^n. This is how a 32 768 Hz watch crystal becomes a 1 Hz tick.

AS A COUNTER

Read the stages as bits and the same circuit is counting up in binary — same wires, different reading.

THE LINK

Counting and dividing are the same operation. That is why one circuit serves timers, PWM and program counters.

L06 • MODERN COMPUTER ARCHITECTURE

The Four Flip-Flops at a Glance

One table to keep. Everything else in this lecture is built from these four rows.

TYPE

INPUTS

WHAT IT DOES

CHARACTERISTIC EQ.

TYPICALLY USED FOR

SR

S, R

Set, reset or hold — but S = R = 1 is illegal

Q' = S + R'Q

Latches, simple control

D

D

Copies its input to the output on the edge

Q' = D

Registers, pipeline stages, state registers

JK

J, K

Set, reset, hold — and toggle when both are 1

Q' = J·Q' + K'·Q

General-purpose sequential design

T

T

Flips when T = 1, holds when T = 0

Q' = T XOR Q

Counters and frequency dividers

THEY ARE ALL THE SAME DEVICE

Every one of these is an SR latch pair with different logic bolted onto the front. Choosing a flip-flop type is really choosing how much of the next-state logic you want built in already.

WHAT REAL HARDWARE USES

Modern chips and every FPGA use D flip-flops almost exclusively — they are the smallest and fastest. JK and T are design conveniences: you reason in T, the synthesis tool builds it out of D and an XOR.

L06 • MODERN COMPUTER ARCHITECTURE

Preset, Clear and Reset

Every real flip-flop has a back door that ignores the clock. Counters and state machines both need it.

THE EXTRA PINS

ASYNCHRONOUS

SYNCHRONOUS

When it acts

Immediately

On the next clock edge

Needs the clock?

No

Yes

Timing risk

Can violate setup/hold

None — it is ordinary

Logic

Typical use

Power-on reset

Mod-N counter wrap,

FSM restart

Rule of thumb: one asynchronous reset for power-on, synchronous

resets for everything the design does while it is running.

WITHOUT A RESET

Flip-flops power up in a random state. A counter might start at 173, and a state machine might start in a state that has no way out. The circuit works perfectly in simulation and hangs on the bench.

WITH A RESET

One known starting point, guaranteed. Every FSM you draw from now on gets a reset arrow pointing at its initial state — no exceptions.

L06 • MODERN COMPUTER ARCHITECTURE

1

Registers

The foundational building blocks of CPU

storage.

PRIMARY DEFINITION

A register is just a row of **Flip-Flops** sharing a clock — the fastest, closest memory a CPU has.

Flip-Flop Row

Multi-bit storage

Shared Clock

Atomic updates

Fastest Speed

Closest to CPU

L06 • MODERN COMPUTER ARCHITECTURE

From One Bit to Many

Put **Flip-Flops** side by side, share a clock, and you store a whole word at once.

Atomic Multi-Bit Storage

All four bits are captured on the SAME clock edge — the word updates atomically, as one single unit.

THE SYSTEM SECRET

What is a CPU register?

This is literally what a CPU register is: **32 or 64 D-Flip-Flops** sharing a single, synchronous clock. That is the entire secret.

L06 • MODERN COMPUTER ARCHITECTURE

The Register

Store, hold, and read back an n-bit value — the CPU's working scratchpad.

BLOCK DIAGRAM

8-BIT REGISTER

D7 ... D0

in (8 bits)

out

CLK

WHERE REGISTERS LIVE

The register **file** — **32 registers** in MIPS/RISC-V — is a CPU's fastest storage.

It serves as the **immediate staging area** for active operations, feeding data directly to execution units.

Also: the program counter (PC), instruction register, and pipeline stage registers are all just registers.

Faster than cache, faster than RAM — because it's **Flip-Flops** right next to the ALU.

L06 • MODERN COMPUTER ARCHITECTURE

The Load Enable

A register must **HOLD** its value across clock edges until you choose to update it.

THE PROBLEM

A plain Flip-Op grabs D on EVERY clock edge. But a register should keep its value for many cycles, updating only when told.

THE FIX — a load-enable MUX per bit:

LOAD = 1 → feed new data in.

LOAD = 0 → feed the Flip-Op's OWN output back in (hold).

A 2:1 MUX on each bit's D input chooses between 'new value' and 'keep current'.

ONE BIT'S PATH

The MUX + feedback loop = a register that remembers until LOAD says otherwise.

L06 • MODERN COMPUTER ARCHITECTURE

The Shift Register

Chain **Flip-Flops** output-to-input: each clock edge, every bit shifts one place.

BLOCK DIAGRAM

HOW IT SHIFTS

Each **Flip-Op's** output feeds the **NEXT** one's input. A bit entered at 'in' walks right, one FF per clock.

It's a conveyor belt for bits — the basis of serial data transfer and multiply/divide-by-2 operations.

L06 • MODERN COMPUTER ARCHITECTURE

Four Shift Register Types

Named by how data goes IN and comes OUT: serial or parallel.

SISO

Serial In, Serial Out

One bit in, one bit out per clock — a pure delay line.

SIPO

Serial In, Parallel Out

Bits enter one at a time, read out all at once — **serial→parallel** converter.

PISO

Parallel In, Serial Out

Load a whole word, shift it out bit by bit — **parallel→serial** converter.

PIPO

Parallel In, Parallel Out

Load and read a full word — this is just a normal register with load.

SIPO and PISO are the workhorses: they convert between serial links (few wires) and parallel buses (fast).

L06 • MODERN COMPUTER ARCHITECTURE

What Shift Registers Do

Small circuit, surprisingly many jobs.

Serial ↔ parallel

UART, SPI, USB all send data one bit at a time over few wires; shift registers reassemble it into bytes.

x2

Multiply / divide by 2

Shifting binary left multiplies by 2; shifting right divides by 2. Free arithmetic, no adder.

Delay lines & buffers

A SISO register delays a signal by exactly N clock cycles — used in DSP and pipelines.

Pseudo-random (LFSR)

Add feedback via XOR and a shift register generates long pseudo-random sequences — used in crypto & test.

Thanks

for watching!

DOCUMENTS

Lab: 0100

Full Adder & 4-Bit Ripple Adder Adders

Compose half adders into full adders. Cascade four full adders. Watch the carry ripple.

LAB 03 • MODERN COMPUTER ARCHITECTURE

What You'll Build Today

Three modules. Each one composes the previous. By the end, you'll add two 4-bit numbers in silicon.

01

Half Adder

Two inputs A, B. Produces Sum and Cout. The simplest 1-bit add — but it can't accept a carry-in.

02

Full Adder

Three inputs A, B, Cin. Built from two half adders + an OR gate. The true 1-bit adder cell.

03

4-Bit Ripple Adder

Four full adders chained. Cout of stage i becomes Cin of stage i+1. Adds two 4-bit numbers.

LAB 03 • MODERN COMPUTER ARCHITECTURE

Why Adders Matter

Addition is the atom of arithmetic. Every CPU instruction that does math leans on one of these cells.

Subtraction

A – B is just A + (–B) + 1 — done with the same adder, with Cin = 1 and one input inverted.

Multiplication

A × B is a sequence of shifts and adds. Multipliers are arrays of adders.

Program counters

PC ← PC + 4 happens every clock cycle. Always an adder. Address arithmetic too.

Master a 1-bit adder, and you've cracked open everything that follows: subtractors, multipliers, ALUs, the entire datapath.

BINARY ADDITION · 0 + 0 = 00 0 + 1 = 01 1 + 0 = 01 1 + 1 = 10

LAB 03 • MODERN COMPUTER ARCHITECTURE

Half Adder · The Equations

Two 1-bit inputs. Two 1-bit outputs. Two Boolean equations. That's the whole device.

SUM

Sum = A ⊕ B

Sum is 1 when A and B differ. That's XOR.

CARRY-OUT

Cout = A · B

Carry-out is 1 only when both are 1. That's AND.

WHY "HALF"?

A half adder has no place to accept a carry from a previous stage. That's only half of what real binary addition needs — so you can't chain them. The full adder, two slides later, fixes this.

LAB 03 • MODERN COMPUTER ARCHITECTURE

Half Adder · Truth Table & Schematic

Four input rows. Two outputs each. Read it left-to-right.

TRUTH TABLE

A	B	Sum	Cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Last row · Sum = 0, Cout = 1. That's the carry: 1+1 = 10 in binary.

SCHEMATIC

LAB 03 • MODERN COMPUTER ARCHITECTURE

Half Adder · Verilog Implementation

Two continuous assignments. No always blocks, no clocks. Pure combinational.

```
half_adder.v
// 1-bit half adder

input wire a,
input wire b,
output wire sum,
output wire cout
);
```

```
assign sum = a ^ b; // XOR
assign cout = a & b; // AND
endmodule
```

NOTES

^ bitwise XOR. & bitwise AND.

Two assign statements run in parallel — there's no ordering.

Both outputs settle the moment a or b changes.

In real silicon, this is one XOR cell and one AND cell, side by side.

LAB 03 • MODERN COMPUTER ARCHITECTURE

Half Adder · Self-Checking Testbench

Four input combinations. Compare to expected. Print PASS or FAIL for each row of the truth table.

```
ha_tb.v

reg a, b; wire sum, cout;
half_adder dut(a,b);
dut.sum(sum); dut.cout(cout);
task check(input ea,eb,es,ec);
begin a=ea; b=eb; #1;
$display("%s a=%b b=%b s=%b c=%b",
{sum==es && cout==ec}?"PASS":"FAIL",
a, b, sum, cout);
end
endtask

initial begin
$dumpfile("ha.vcd"); $dumpvars(0,ha_tb);
check(0,0,0,0); check(0,1,1,0);
check(1,0,1,0); check(1,1,0,1);
$finish;
end
endmodule
```

EXPECTED OUTPUT

PASS a=0 b=0 s=0 c=0

PASS a=0 b=1 s=1 c=0

PASS a=1 b=0 s=1 c=0

PASS a=1 b=1 s=0 c=1

Four lines, all PASS. If any FAIL appears — check operator precedence in your assigns.

LAB 03 • MODERN COMPUTER ARCHITECTURE

Why a Half Adder Isn't Enough

Try to add two multi-bit numbers and you'll hit a wall on the very second column.

ADD 11 + 01

1 1 ← A

+ 0 1 ← B

1 0 0

Bit 0: 1 + 1 → sum=0, carry=1 ✓

Bit 1: 1 + 0 + carry → needs 3 inputs!

THE PROBLEM

Bit position 1 has three signals to sum:

A[1]

B[1]

Carry-in from bit 0

A half adder has only two inputs. It cannot accept that carry. We need a 3-input adder cell — the full adder.

LAB 03 • MODERN COMPUTER ARCHITECTURE

Full Adder · The Equations

Three 1-bit inputs (A, B, Cin) summed into a 2-bit result (Cout, Sum).

SUM

Sum = A ⊕ B ⊕ Cin

Three-way XOR. Sum is 1 when an odd number of inputs are 1.

CARRY-OUT

Cout = (A · B) + (Cin · (A ⊕ B))

Carry when both A,B are 1 – or when the partial sum (A⊕B) is 1 and Cin is also 1.

KEY INSIGHT

Look at the Sum formula: it's a chain of XORs. And the Cout formula has (A⊕B) baked into it. That shared (A⊕B) term is the hint — you can build a full adder from two half adders.

LAB 03 • MODERN COMPUTER ARCHITECTURE

Full Adder · Truth Table

Eight rows — all 2³ combinations of (A, B, Cin). Sum is 1 on odd-count rows; Cout is 1 when two or more inputs are 1.

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

1
1
1
1
1

READING THE TABLE

Sum
1 when an odd number of inputs (1 or 3) are 1. Pure parity function.

Count
1 when two or more inputs are 1. Pure majority function.

Sum + Count
Together they encode the 2-bit count of how many inputs are 1.

LAB 03 · MODERN COMPUTER ARCHITECTURE

Full Adder · Built From Two Half Adders

Stage 1 adds A and B. Stage 2 adds that result to Cin. An OR gate combines the two carry signals.

A
B
Cin
HA 1
a, b → sum1, cout1
sum1
cout1
HA 2
sum1, cin → sum, cout2
Sum
cout2
OR
Count

Sum from HA2 is the final sum. Carry from either HA is the final carry — that's why OR works.

LAB 03 · MODERN COMPUTER ARCHITECTURE

Why the OR Gate Works for the Carry-Out

The two half-adder carries can never be 1 at the same time. So OR is enough — no third AND is needed.

01
count1 = A · B
First half adder sees A and B. Its carry is 1 only when both A and B are 1.

02
cout2 = sum1 · Cin
Second half adder sees sum1 = (A ⊕ B) and Cin. Its carry is 1 when sum1 and Cin are both 1.

03
They can't both be 1
If A=B=1, then sum1 = A⊕B = 0, so cout2 = 0. cout1 and cout2 are mutually exclusive — OR or XOR both work.

ALGEBRA · Cout = cout1 + cout2 = AB + (A⊕B)·Cin
→ the standard full-adder carry equation.

LAB 03 · MODERN COMPUTER ARCHITECTURE

Full Adder · Verilog (Structural)

Instantiate two half_adder modules, OR their carries together. No new assigns — pure composition.

```
full_adder.v
// 1-bit full adder · 2 HAS + 1 OR
```

```
input wire a, b, cin,
output wire sum,
output wire cout
);
wire s1, c1, c2;
half_adder ha1 (a(a), b(b),
    .sum(s1), .cout(c1));
half_adder ha2 (a(s1), b(cin),
    .sum(sum), .cout(c2));
assign cout = c1 | c2; // OR
endmodule
```

WHAT'S WIRED

ha1 sees a, b directly. Its sum is the partial sum s1.
ha2 sees s1 and cin. Its sum becomes the final Sum.
c1, c2 are the two HA carry-outs.
cout = c1 | c2 combines them — exactly one (or neither) is ever 1.

LAB 03 · MODERN COMPUTER ARCHITECTURE

Full Adder · Testbench (All 8 Cases)

Pack three input bits at Once — same {a, b, cin} = i[2:0] trick you used for the MUX testbench in Lab 2.

```
fa_tb.v
```

```
reg a, b, cin; wire sum, cout;
half_adder ha(a(a), b(b), cin(cin),
    .sum(sum), .cout(cout));
integer i;
initial begin
    $dumpfile("fa.vcd"); $dumpvars(0, fa_tb);
    for (i = 0; i < 8; i = i + 1) begin
        {a, b, cin} = i[2:0]; #5;
        $display("a=%b b=%b cin=%b → sum=%b cout=%b",
            a, b, cin, sum, cout);
    end
    $finish;
end
endmodule
```

RUN IT

```
$iverilog -o fa.vvp \
half_adder.v \
full_adder.v \
fa_tb.v
$vvp fa.vvp
$gtkwave fa.vcd
```

Pass all three .v files. Verilog won't find half_adder otherwise.

LAB 03 · MODERN COMPUTER ARCHITECTURE

4-Bit Ripple-Carry Adder · The Idea

Chain four full adders. Bit 0 on the right, bit 3 on the left. Each carry-out feeds the next carry-in.

FA3

```
bit 3
a[3]
```

```

s[3]
s[3]
FA2
bit 2
a[2]
b[2]
s[2]
FA1
bit 1
a[1]
b[1]
s[1]
FA0
bit 0
a[0]
b[0]
s[0]
c[1]
c[2]
c[3]
cin
cout
"Ripple" = the carry has to travel through every
stage before the last sum is correct. Each stage
waits for the previous.
LAB 03 · MODERN COMPUTER ARCHITECTURE
4-Bit Ripple Adder · Verilog
Four full_adder instances. One internal 3-bit carry wire. Connect
cout[i] to cin[i+1].
adder4.v

input wire [3:0] a, b,
input wire cin,
output wire [3:0] sum,
output wire cout
);
wire c1, c2, c3; // internal carries
full_adder fa0(.a(a[0]),.b(b[0]),.cin(cin),
.sum(sum[0]),.cout(c1));
full_adder fa1(.a(a[1]),.b(b[1]),.cin(c1),
.sum(sum[1]),.cout(c2));
full_adder fa2(.a(a[2]),.b(b[2]),.cin(c2),
.sum(sum[2]),.cout(c3));
full_adder fa3(.a(a[3]),.b(b[3]),.cin(c3),
.sum(sum[3]),.cout(cout));
endmodule
KEY POINTS
[3:0] declares a 4-bit vector. a[0] is the LSB.
c1, c2, c3 are internal — only carry-in and
carry-out are exposed.
cin → fa0, and fa3.cout → cout. The chain is
complete.
Try generate loops in the stretch problems
for an N-bit version.
LAB 03 · MODERN COMPUTER ARCHITECTURE
4-Bit Ripple Adder · Testbench Vectors
Four targeted test cases. Each one stresses a different aspect of
carry behaviour.
adder4_tb.v

reg [3:0] a, b; reg cin;
wire [3:0] sum; wire cout;
adder4 dut(a(a),b(b),cin(cin),
.sum(sum),.cout(cout));
initial begin
$dumpfile("adder4.vcd");
$dumpvars(0, adder4_tb);
cin = 0;
a=4'b0001; b=4'b0001; #10; // 1+1=2
a=4'b0111; b=4'b0001; #10; // 7+1=8
a=4'b1111; b=4'b0001; #10; // overflow
a=4'b1000; b=4'b1000; #10; // 8+8=16
$finish;
end
endmodule
WHY THESE 4?
0001 + 0001 smallest carry — just bit 0 to bit
1.
0111 + 0001 carry ripples through bits 0, 1, 2.
1111 + 0001 carry ripples through ALL four
bits — overflow out.
1000 + 1000 one-bit add, immediate
overflow to cout.
LAB 03 · MODERN COMPUTER ARCHITECTURE
Walking Through Test Cases · Part 1
Trace each carry by hand. If your simulation disagrees, you have a
bug in the adder — not in the testbench.
0001 + 0001 = 00010
0 0 0 1 ← a
+ 0 0 0 1 ← b
■■■■■■■■■■
0 0 1 0 sum = 0010
cout = 0
Bit 0: 1+1 → s=0, c=1. Bit 1: 0+0+1 → s=1, c=0.
No further carry.
0111 + 0001 = 01000
0 1 1 1 ← a
+ 0 0 0 1 ← b
■■■■■■■■■■
1 0 0 0 sum = 1000
cout = 0
Carry travels: Bit 0 → 1 → 2 → 3, and bit 3
finally outputs s=1, c=0.
LAB 03 · MODERN COMPUTER ARCHITECTURE
Walking Through Test Cases · Part 2
Now the edge cases — when 4 bits are too few to hold the result, the
final carry-out tells you so.
1111 + 0001 = 10000 ← overflow
1 1 1 1 ← a
+ 0 0 0 1 ← b
■■■■■■■■■■
0 0 0 0 sum = 0000
cout = 1 ← overflow!
Carry ripples through ALL four stages. sum looks like
zero, but
cout=1 means "there's a fifth bit".
1000 + 1000 = 10000
1 0 0 0 ← a
+ 1 0 0 0 ← b
■■■■■■■■■■
0 0 0 0 sum = 0000
cout = 1
Only bit 3 has a 1+1 → carry out immediately.
No rippling, but still

```

an overflow.
LAB 03 · MODERN COMPUTER ARCHITECTURE
Watching the Carry Ripple in GTKWave
 Open adder4.vcd and add a, b, sum, cout, c1, c2, c3 to the waveform. Zoom in on the 1111 + 0001 transition.
 time →
 (propagation through stages)
 c1 (FA0 → FA1)
 ↑
 c2 (FA1 → FA2)
 ↑
 c3 (FA2 → FA3)
 ↑
 cout (FA3 out)
 ↑
 inputs change
 Each carry rises slightly later than the previous one — that staircase is the "ripple" propagating in real time.
LAB 03 · MODERN COMPUTER ARCHITECTURE
Why Ripple-Carry is Slow
 In real silicon, every gate has a delay. The carry has to walk through all of them — serially.
 ~2
 gate delays per FA stage
 Each full adder adds 2 levels of logic
 between its Cin and its Cout.
 8
 delays for a 4-bit add
 4 stages × 2 delays = 8. Doable. But scale
 up...
 128
 delays for a 64-bit add
 64 stages × 2 delays. Modern CPUs can't
 tolerate this latency.
THE FIX
 Compute every carry in parallel using the equation $Cout = G + (P \cdot Cin)$, where $G = A \cdot B$ and $P = A \oplus B$. That's the carry-look-ahead adder — Lab 4.
LAB 03 · MODERN COMPUTER ARCHITECTURE
Common Errors · And the Fix
 Five traps specific to this lab. Spot them quickly — save yourself 30 minutes of waveform-staring.
 cout always 0, even when 1+1+1
 Probably wrote cout = c1 & c2 instead of c1 | c2. AND only fires when BOTH carries are 1, which never happens.
 sum bits in reverse order
 a[0] is the LSB. If you wired sum[3] to fa0, you've reversed everything. Double-check your indices.
 Last sum is always X (red)
 fa3 has no Cin connection — you forgot to wire c3 in. Unassigned Verilog inputs default to X (unknown).
 Waveform shows correct sum, but at time 0
 You added all inputs at t=0 with no #delay. Add small delays between vectors so transitions are visible.
LAB 03 · MODERN COMPUTER ARCHITECTURE
Take These Home
 Three stretch problems. Solutions discussed at the start of Lab 4.
 01
Build it from one half adder
 Refactor full_adder.v to use only ONE half_adder instance plus a few primitive gates. Verify it still passes all 8 cases.
 02
Parametric N-bit adder
 Rewrite adder4.v as a parameterised adder#(N) module with a Verilog generate loop. Test with N = 8 and N = 16.
 03
Overflow detection
 Add an overflow output to your 4-bit adder using signed arithmetic semantics — $V = c3 \oplus c4$ (carry into MSB XOR carry out).
 Pick one. All three if you're feeling sharp.
LAB 03 · MODERN COMPUTER ARCHITECTURE
Why This Matters
 The 1-bit full adder is the building block. Every arithmetic structure in a CPU starts here.
TODAY
 You built an ALU's most basic operation from primitive gates. Two HAS, four FAs, one VCD trace.
TOMORROW
 Lab 4 trades depth for width — carry-look-ahead computes every carry in parallel. Faster, more transistors.
REAL CHIPS
 Modern CPUs use carry-select, Kogge-Stone, or Brent-Kung adders. All of them still decompose into FA cells like the one you just wrote.
BEYOND
 Multipliers are arrays of FAs. Floating-point units are wider
 FAs. Even GPUs and TPUs lean on these cells billions of times per second.
 You just built, in 90 minutes, the cell that ships in every CPU on Earth.
END OF LAB 03
 Two half adders.
 One full adder.
 Four cells, one ALU's heart.
Questions?
Thanks
 for watching!
 =====
 =====
 =====

Lab: 0011

From Boolean minimisation to the data-selector circuits that route every bus in a modern CPU.

Where We Left Off

Lab 1 built the gates. Lecture 2 shrank them. Today we build the circuits that pick between them.

Lab 1

Seven gates in Verilog — AND, OR, NOT, XOR, NAND, NOR, XNOR — plus a self-checking testbench.

Lab 2

SOP / POS, Karnaugh maps, and the proof that NAND (or NOR) alone can build any circuit.

Lab 3 — Today

Multiplexers and demultiplexers: the gate patterns that select and route data — the ternary operator, the case statement, and structural instantiation.

What You'll Build Today

By the end of this lab, you'll have four data-selector circuits running in simulation.

2:1 MUX

Two inputs, one select line

4:1 MUX

Four inputs, two select lines

1:2 DEMUX

One input routed to one of two outputs

1:4 DEMUX

One input routed to one of four outputs

Naming convention: an n:1 MUX has n inputs. A demultiplexer is the mirror image, so this course names it by its 1:n output count — 1:2 and 1:4 — to avoid an "n:1 demux" that would read backwards.

Behavioural Modeling

Procedural blocks — and the single most-tested rule in this entire lecture.

The Initial Block

Runs exactly once, starting at simulation time zero — testbench territory.

INITIAL — RUNS ONCE

```
initial begin
a = 0; b = 0;
#10 a = 0; b = 1;
#10 $finish;
end
```

A module can contain more than one initial block — each starts independently at t=0 and runs to completion (or the next timing control) concurrently with the others.

The always Block

Re-triggers every time its sensitivity list changes — the procedural equivalent of assign.

```
always @(*) – combinational
Re-evaluates whenever any signal read inside the block changes. The (*) auto-infers the sensitivity list — the modern,
error-proof way to write it.
always @(*) y = a & b;
always @(posedge clk) – sequential
Re-evaluates only on the rising edge of clk. This is how
registers and flip-flops are described — a full preview,
properly covered in Module 2.
always @(posedge clk) q <= d;
The left-hand side inside any always block must be declared reg —
this is Part II's rule, now in action.
`timescale, `define, `include
Three directives you'll see at the top of nearly every file this course.
`timescale
Sets the simulation time unit and
precision. Every Lab 1 file starts with
this exact line.
`timescale 1ns/1ps
`define
A text macro, substituted before
compilation — like #define in C. Used
sparingly; parameter is usually the
better tool.
`define WIDTH 8
`include
Splices another file's text in verbatim
at compile time — how large designs
share common definitions across files.
`include "defs.v"
Testbench Theory
The constructs behind Lab 1's testbench — explained, not yet typed.
Why a Testbench Has No Ports
The one module in your entire design that connects to nothing but
simulation itself.
Nothing outside simulation ever instantiates it
Every other module in this course gets instantiated by something —
gates_top instantiates and_gate; a real chip's top-level module gets
synthesized and fabricated. A testbench is instantiated by nobody. It
sits at the very top of the hierarchy purely to drive stimulus into the
design under test (DUT) and observe what comes back — so it has
no reason to expose a port list at all.
$time, $finish, $stop
Controlling and reading the simulation clock itself.
$time
Returns the current simulation time —
what powers the t=%0t field in every
$monitor line you've seen.
$finish
Ends the simulation entirely and exits
the simulator. This is how Lab 1's
testbench cleanly stops after its
stimulus sequence.
$stop
Pauses simulation without exiting —
hands control back to an interactive
simulator session. Rare in scripted,
batch-run testbenches.
$dumpfile & $dumpvars
```

The two lines that turn a silent simulation into a viewable waveform.

ENABLING WAVEFORM CAPTURE

```
$dumpfile("gates_tb.vcd"); // names the output file
$dumpvars(0, gates_tb); // records every signal from
level 0 down
$dumpfile alone creates an empty VCD — both lines are
required. This is exactly the pair Lab 1's Common
Mistakes slide warns
about forgetting.

Generating Stimulus



initial plus #delay is the entire vocabulary needed to drive a DUT through every input combination.



A FOUR-COMBINATION STIMULUS SEQUENCE



```
initial begin
a=0; b=0;
#10; a=0; b=1;
#10; a=1; b=0;
#10; a=1; b=1;
#10; $finish;
end
#10 between each line advances simulation time by 10
units before the next assignment — without it, every
line would execute
at t=0.
Verilog in 60 Seconds
Just enough to start. We'll come back to syntax in later labs.

module

Defines a hardware unit. Has input and output ports.

wire

A net — connects things. Used for combinational signals.

reg

A variable — needed for assignments inside `always' blocks.

assign

Continuous assignment. Use this for combinational logic.

always @(*)

Procedural block. Re-runs when ANY listed signal changes.

Why a Data Selector Circuit?

One circuit primitive shows up in almost every block of a CPU datapath.

ALU operand select

Choosing between a register value and an immediate constant

before the ALU.

PC update

Choosing the next instruction address: PC+4, a branch target, or a jump target.

Memory routing

Sending one data bus to one of several destinations — registers, cache, or I/O.

Every one of these is either a multiplexer (many signals → one) or a demultiplexer (one signal → many).

Part 1 · Multiplexers

Many signals in. One signal out — chosen by a select line.

What Is a Multiplexer?

A digital switch: it selects exactly one of several inputs and routes it to a single output.

MUX = Multiplexer = Data Selector

♦

A MUX with n select lines can choose among 2n inputs.

♦

1 select line → 2 inputs (2:1 MUX).

♦

2 select lines → 4 inputs (4:1 MUX).

♦

Only one input ever reaches the output at a time — the rest are ignored.

2:1 MUX · Truth Table

One select line S chooses between I0 and I1.


```
S
0
1
Y
0
I0
1
I1
2:1
MUX
A
B
S
Y
S = 0 → Y = A
S = 1 → Y = B
"if S is 0, pass I0; if S is 1, pass I1." Exactly one product term survives
for any value of S — that's what makes it a selector
and not just a
logic function.
2:1 MUX · Expression & From Equation to
Gates
Two AND gates, one OR gate, one inverter — the equation built in
silicon.
Boolean expression (sum-of-products form):
Y = S'·I0 + S·I1
This is exactly the SOP form Lecture 2 taught you to derive from a
truth table — the MUX equation is just the K-map result written out.
Exercise 1 · Build a 2:1 MUX (Gate-Level)
Write the equation from Slide 7 directly as Verilog primitives.
mux2_1_gate.v
```



```
input wire a, b, sel,
output wire y
);
wire sel_n, a0, a1;
not u0 (sel_n, sel); // S'
and u1 (a0, a, sel_n); // S'·I0
and u2 (a1, b, sel); // S·I1
or u3 (y, a0, a1); // sum
endmodule
KEY POINTS
sel_n
A named wire for the inverted select — no gate
output is left anonymous.
not / and / or
The same primitive gates from Lab 1, just wired to a
new pattern.
u0, u1...
Instance names are mandatory for primitive gates
with more than one instance of the same type in a
module.
```


```


```

New Construct · The Conditional Operator

A one-line hardware selector — the exact shape of a multiplexer. syntax

```
assign y = condition ? value_if_1 : value_if_0;
? reads as "if" — evaluates condition first.
: reads as "else" — the fallback value.
This is a continuous assignment, so it belongs after assign, never
inside an always block.
sel ? in1 : in0 is a 2:1 MUX in eight characters. Chain them and you
can build any MUX width — but as the chain grows, a case
statement (next up) reads far more clearly.

Exercise 1 · Alternate: the Ternary Version



Same interface, one line of behavioral Verilog instead of four gates. mux2_1_ternary.v



```
input wire a, b, sel,
output wire y
);
assign y = sel ? b : a;
endmodule
WHY BOTH VERSIONS?
Gate-level
Shows exactly which transistors switch — this is what
synthesis produces either way.
Ternary
Shows what you'll actually type once you trust the
tool to do the gate mapping.
Same module name, same port list — pick one file at a time. Both
simulate identically. From here on, this course defaults to the shorter
behavioral style unless a lab specifically asks for gate-level, exactly
like Lab 1's assign vs primitive distinction.

Exercise 1 · Testbench for the 2:1 MUX

Drive all four (I0, I1, sel) combinations and record the waveform. mux2_1_tb.v


```
reg a, b, sel;
wire y;
mux2x1 uut (.a(a), .b(b), .sel(sel), .y(y)
);
initial begin
sel = 0; a = 0; b = 0;
#10 sel = 0; a = 0; b = 1;
#10 sel = 0; a = 1; b = 0;
#10 sel = 0; a = 1; b = 1;
#10 sel = 1; a = 0; b = 0;
#10 sel = 1; a = 0; b = 1;
#10 sel = 1; a = 1; b = 0;
#10 sel = 1; a = 1; b = 1;
#10 $finish;
end
initial begin
$dumpfile("mux2x1_wave.vcd");
$dumpvars(0, mux2x1_tb);
$monitor("Time = %0t | sel = %b, a = %b, b = %b | y = %b",
$time, sel, a, b, y);
end
endmodule
4:1 MUX · Truth Table
Two select lines now address four inputs — exactly 22.
S1
S0
Y
0
0
I0
0
1
I1
1
0
I2
1
1
I3
Boolean expression:
Y = S1'S0'I0 + S1'S0'I1 + S1S0'I2 + S1S0'I3
Four AND terms, one per input, each gated by the unique minterm of
(S1, S0) that selects it — then OR'd together. Writing all four terms
out by hand is exactly why Lecture 2's SOP-from-truth-table method
matters once a circuit outgrows two inputs.

4:1 MUX · Two Ways to Build It



Structural composition vs. behavioral description — same circuit, two
mindsets.



Structural



Instantiate the mux2_1 module three times and wire them into a
tree. Mirrors Lab 1's top.v — building bigger circuits out of
smaller,
already-verified ones.



Behavioral



Describe the truth table directly with a case statement inside an
always block. Faster to write, and how you'll build
most real designs.



New Construct · The Case Statement



A readable, self-documenting way to describe a truth table inside an
always block. syntax



```
always @(*) begin
case (sel)
2'b00: y = i0;
2'b01: y = i1;
2'b10: y = i2;
default: y = i3; // covers 2'b11 AND catches X/Z
endcase
end
y must be declared reg, not wire — it's assigned inside a procedural
block. default is not optional: an incomplete case list infers a
latch instead of a MUX — the single most common bug in this
exercise.

Exercise 2 · 4:1 MUX — Behavioral Build

The same interface, described with a case statement. mux4_1_behavioral.v


```
input wire i0, i1, i2, i3,
input wire s0, s1,
output reg y
);
always @(*) begin
case ({s1, s0})
2'b00: y = i0;
```


```


```


```


```

```
2'b01: y = i1;
2'b10: y = i2;
default: y = i3;
endcase
end
endmodule
Exercise 2 · Testbench — Sweep Every Select Value
New construct: a for loop generates all four select combinations automatically.
mux4_1_tb.v
```

```
reg i0, i1, i2, i3, s0, s1;
wire y;
integer k;
mux4_1 dut (.i0(i0), .i1(i1), .i2(i2), .i3(i3),
.s0(s0), .s1(s1), .y(y));
initial begin
$dumpfile("mux4_1_tb.vcd"); $dumpvars(0, mux4_1_tb);
i0=0; i1=1; i2=0; i3=1;
for (k = 0; k < 4; k = k + 1) begin
{s1, s0} = k; #10; // walks 00, 01, 10, 11
end
$finish;
end
endmodule
```

New Construct · for Loops in a Testbench
Stop hand-typing every input combination — let the simulator count for you.
integer k;
Declares a variable to hold the loop counter — testbench-only, never in synthesizable logic.
for (k=0; k<4; k=k+1)
Same three-part syntax as C: init; condition; step.
{s1, s0} = k;
Concatenation on the left-hand side — splits k's two low bits straight into the two select wires.
#10 inside the loop
Advances simulation time on every iteration, so each combination gets its own visible slice of the waveform.
Part 2 · Demultiplexers
One signal in. Many possible outputs — routed by a select line.
What Is a Demultiplexer?
The mirror image of a MUX: one input, routed to exactly one of several outputs.

◆ **DEMUX = a data distributor, the reverse of a data selector.**
◆ n select lines route the input to one of 2ⁿ outputs.
◆ **Every output not selected is driven to 0 — nothing is left floating.**
Naming note: "2:1 DEMUX" would literally mean 2 inputs → 1 output — backwards for what we're building. This course calls

these circuits 1:2 and 1:4 DEMUX (1 input : n outputs) instead.
1:2 DEMUX · Truth Table & Expression
One select line S routes input D to either Y0 or Y1.

S
Y0
Y1
0
D
0
1
0
D
Boolean expressions:
Y0 = S'D Y1 = S·D
Notice the symmetry with the 2:1 MUX equation on Slide 7 — same AND/inverter pattern, just fanning one input out to two gated outputs instead of gating two inputs into one.

1:4 DEMUX
D
O0
O1
O2
O3
S1
S0

Exercise 3 · Build a 1:2 DEMUX
Two AND gates and an inverter — the mirror image of Exercise 1.
demux1_2.v

```
input wire i, sel,
output wire y0, y1
);
assign y0 = ~sel & i;
assign y1 = sel & i;
endmodule
```

No case statement needed here — with only one select bit and two mutually exclusive outputs, two continuous assignments are already as clear as it gets.
Exercise 3 · Testbench for the 1:2 DEMUX
Confirm the unselected output always reads 0 — the defining behavior of a DEMUX.
demux1_2_tb.v

```
reg i, sel;
wire y0, y1;
demux1_2 dut (.i(i), .sel(sel), .y0(y0), .y1(y1));
initial begin
$dumpfile("demux1_2_tb.vcd"); $dumpvars(0,
demux1_2_tb);
i=1; sel=0; #10; // expect y0=1, y1=0
i=1; sel=1; #10; // expect y0=0, y1=1
i=0; sel=1; #10; // expect y0=0, y1=0
$finish;
end
endmodule
```

1:4 DEMUX · Truth Table
Two select lines route D to exactly one of four outputs; the rest read 0.
S1
S0

Y0
Y1
Y2
Y3
0
0
D
0
0
0
0
1
0
D
0
0
1
0
0
0
0
D
0
1
1
0
0
0
D

With a case statement, this table becomes almost literal Verilog — each row is one case branch that drives the whole output bus at once.
1:4 DEMUX · Where This Shows Up
Any time one resource must be shared by steering it to different destinations in turn.
Memory address decoding
The high bits of an address 'demux' a read/write enable signal to
exactly one memory block or chip.
Telecom switching
A single voice/data line is routed to one of several trunk lines based on a routing code.
Serial-to-many I/O
One microcontroller output pin, time-shared across several
peripheral chip-select lines.
Exercise 4 · Build a 1:4 DEMUX
A 4-bit output bus, driven entirely by a case statement.
demux1_4.v

```
input wire d,
input wire [1:0] sel,
output reg [3:0] y
);
always @(*) begin
y = 4'b0000; // clear all outputs first
case (sel)
2'b00: y[0] = d;
2'b01: y[1] = d;
2'b10: y[2] = d;
default: y[3] = d;
endcase
end
endmodule
```

Exercise 4 · Testbench for the 1:4 DEMUX
Reuse the for-loop pattern from Exercise 2 to sweep all four select values.
demux1_4_tb.v

```
reg d;
reg [1:0] sel;
wire [3:0] y;
integer k;
demux1_4 dut (.d(d), .sel(sel), .y(y));
initial begin
$dumpfile("demux1_4_tb.vcd"); $dumpvars(0,
demux1_4_tb);
d = 1;
for (k = 0; k < 4; k = k + 1) begin sel = k; #10; end
$finish;
end
endmodule
```

Common Errors · And the Fix
Four mistakes that will eat 30 minutes if you don't recognize them.
Missing default in a case
Every select value not covered infers a latch, not a MUX. Always add default (or cover every combination explicitly).
y declared as wire but assigned in always
Anything assigned inside always must be reg. Continuous assign targets must be wire.
Overlapping case items

Verilog case matches top-to-bottom and stops at the first hit — duplicate or missing bit patterns silently misbehave instead of erroring.
Select bus built from the wrong bit order
{s1, s0} ≠ {s0, s1}. Double-check concatenation order against your truth table before simulating.

Take These Home
Three stretch problems. Solutions discussed at the start of Lab 4.
Build an 8:1 MUX
Combine two 4:1 MUXes and one 2:1 MUX (the third select bit) into an 8-input selector. Reuse Exercise 2's structural pattern.
Parameterize the MUX

Rewrite mux4_1 with a parameter WIDTH so i0–i3 and y become WIDTH-bit buses instead of single wires.
03

Add an enable input
Add an active-low enable_n to demux1_4 so all outputs read 0 whenever enable_n = 1, regardless of sel.
Thanks

```
for
watching!
```

=====
=====
=====

=====
=====
=====

=====

Lecture: 0011

Blocks
Two Philosophies for a CPU's Language
The instruction set is the contract between hardware and software —
and there are two great ways to write it.
L03 · MODERN COMPUTER ARCHITECTURE
Where We Left Off
From single gates to reusable blocks — L1 & L2 gave us the raw materials.
&
Gates & truth tables
AND, OR, NOT, XOR — and
how any behaviour becomes a truth table (L1).
Σ
SOP / POS forms
Any truth table → a Boolean expression we can build in hardware (L2).
■ **K-map**
minimisation
Fewest gates for a given
function — area, power, delay all matter (L2).
■ **NAND is universal**
One gate builds everything. Today we build USEFUL things from
gates.
Today: stop thinking in individual gates. Start thinking in blocks you reuse everywhere.
L03 · MODERN COMPUTER ARCHITECTURE
Today's Roadmap
Four building blocks — then we snap them together into a CPU's core.
1
Multiplexer/Selectors
Multiplexers pick one of many
inputs; demultiplexers route one input to many outputs.
→
2
Decoders
Turn an n-bit code into one active
line out of 2ⁿ — the heart of addressing.
→
3
Adders
Half and full adders — the
arithmetic every ALU is built on.
Then — the payoff! — we show exactly where MUXes, decoders, and adders live inside an ALU and a register
■
L03 · MODERN COMPUTER ARCHITECTURE
Why Reusable Blocks?
Nobody designs a CPU one gate at a time — they use a parts catalogue.
~5
core block types
MUX, decoder, adder, register,
and a few more — repeated millions of times — build an entire processor.
■ **ABSTRACTION**
Think 'select this input', not '17
gates wired thus'.
■ **REUSE**
One verified MUX design,
dropped in everywhere it's needed.
✓
VERIFICATION
Test the block once; trust every copy of it.
■ **SCALE**
Small blocks cascade into big
ones — 2:1 MUX → 4:1 → 8:1.
1. Multiplexers
A data selector: many inputs go in, a select code picks exactly one to come out.
L03 · MODERN COMPUTER ARCHITECTURE
1. Multiplexers
The fundamental routing block of digital computer systems.
CORE DEFINITION
A Data Selector
A multiplexer acts as a digital traffic controller for
processing signals.
Many inputs go in, but a dynamic select code picks
exactly one to come out.
IN
Many Inputs
Multiple source channels arrive carrying unique parallel data
streams.
SEL
The Select Code
Control signals act as the selector, dynamically determining the
routing path.
OUT
Exactly One Output

Only the chosen input line is connected and successfully passes outward.

L03 • MODERN COMPUTER ARCHITECTURE

The 2:1 Multiplexer

One select line chooses between two inputs — the simplest selector.

2:1 MUX

A

B

S

Y

$S = 0 \rightarrow Y = A$ $S = 1 \rightarrow Y = B$

BOOLEAN EQUATION

$Y = S' \cdot A + S \cdot B$

THINK: RAILWAY SWITCH

Two tracks (A, B) converge to one line (Y). The select signal S is the switch lever deciding which train passes through.

Change S,

and a completely different input flows out — instantly, no data is stored.

L03 • MODERN COMPUTER ARCHITECTURE

Building a 2:1 MUX from Gates

$Y = S' \cdot A + S \cdot B$ — two ANDs, one NOT, one OR.

READING IT

- The top AND passes A only when $S'=1$ (i.e. $S=0$).
- The bottom AND passes B only when $S=1$.
- At any moment exactly one AND is 'open'; the OR merges them into Y.

L03 • MODERN COMPUTER ARCHITECTURE

The 4:1 Multiplexer

Two select lines address four inputs — 2 select bits choose 1 of $2^2 = 4$.

4:1 MUX

I0

I1

I2

I3

S1

S0

Y

SELECT → OUTPUT

S1

S0

Y

0

0

I0

0

1

I1

1

0

I2

1

1

I3

$Y = S1'S0'I0 + S1'S0'I1 + S1S0'I2 + S1S0'I3$

Each select combination = one minterm that gates exactly one input through.

L03 • MODERN COMPUTER ARCHITECTURE

Cascading: Build Big MUXes from Small Ones

A 4:1 MUX is just three 2:1 MUXes in a tree.

WHY IT MATTERS

One small, verified block scales to any size.

Need an 8:1? Two 4:1s feeding a 2:1.

Need 16:1? Keep tree-ing.

This 'compose small proven parts into big ones' is exactly how real chips — and good software — are built.

L03 • MODERN COMPUTER ARCHITECTURE

A MUX Can Be Any Logic Function

Wire the inputs to constants, and a 4:1 MUX implements ANY 2-variable function.

THE TRICK

Feed the input variables into the SELECT lines. Then wire each data input to the 0 or 1 that the truth table demands for that row.

Example — implement XOR(A,B):

$S1=A, S0=B$ (selects)

$I0 = 0$ ($A=0, B=0 \rightarrow XOR=0$)

$I1 = 1$ ($A=0, B=1 \rightarrow XOR=1$)

$I2 = 1$ ($A=1, B=0 \rightarrow XOR=1$)

$I3 = 0$ ($A=1, B=1 \rightarrow XOR=0$)

WHY THIS IS A BIG DEAL

A 2ⁿ:1 MUX implements ANY n-variable function — no minimisation needed.

This is the principle behind the Lookup Tables (LUTs) inside every FPGA.

An FPGA is, at its core, a sea of tiny MUX-based LUTs

you reprogram by changing what the data inputs are wired to — which is why an FPGA can 'become' any circuit.

2. Demultiplexers & Decoders

The reverse of a MUX — route one input to many outputs, or light up one line from a code.

L03 • MODERN COMPUTER ARCHITECTURE

The Demultiplexer (DEMUX)

One input, many outputs — the select code decides which output it reaches.

1:4 DEMUX

D

O0

O1

O2

O3

S1

S0

MUX RUN BACKWARDS

A MUX has many inputs → one output. A DEMUX tips it: one input → many outputs. The select bits choose the single destination; all other outputs stay at 0.

REAL USE

A single data line fanned out to one of many destinations — e.g. a CPU writing a result to one chosen register, or a network switch routing a packet to one output port.

L03 • MODERN COMPUTER ARCHITECTURE

The Demultiplexer (DEMUX)

One input, many outputs — the select code decides which output it reaches.

ROUTING TABLE

A

B

Y0

Y1

Y2

Y3

0

0

0

0

0

1

0

D

0

0

0

1

0

0

D

0

0

0

0

1

1

0

0

0

D

0

1

1

0

0

0

D

=====

=====

=====

=====

=====

=====

=====

=====

Lecture: 0001

Boolean Algebra

The journey from a single transistor to the GPUs that train modern AI — starting with 1s and 0s.

Where It All Begins

L01 • MODERN COMPUTER ARCHITECTURE

Welcome to Computer Architecture

The course that reveals what's happening beneath every line of code you've ever written.

THE PROMISE

By the end, you'll understand — from the ground up — how a computer actually runs a program.

Not 'a CPU executes instructions' hand-waving. The real thing: transistors → gates → adders → registers → datapath → instructions → your Python.

You'll be able to trace $A[i] = B[i] + C[i]$ through every single layer.

WHY IT MATTERS TO YOU

Performance: why some code is 100× faster.

AI/ML engineering: GPUs and TPUs are architecture — you can't optimise what you don't understand.

Interviews & systems work: this is the layer that separates people who use computers from people who understand them.

L01 • MODERN COMPUTER ARCHITECTURE

What Is Computer Architecture?

The design of how hardware is organised to run software — the bridge between the two.

ARCHITECTURE

The 'what' — the interface

What instructions a CPU offers.

How registers & memory are seen.

The programmer's contract.

e.g. 'this CPU has an ADD instruction'.

ORGANISATION

The 'how' — the implementation

How the ADD is actually built.

Pipelines, caches, clock speed.

Hidden from the programmer.

e.g. 'ADD takes 1 cycle via this adder'.

This course covers both — starting from the gates up, so 'how' and 'what' make sense together.

L01 • MODERN COMPUTER ARCHITECTURE

Why Should You Care?

Four concrete reasons this is one of the most useful courses you'll take.

PERFORMANCE

Understanding caches & pipelines is why one loop runs 100× faster than another.

AI / ML

GPUs, TPUs, and tensor cores ARE architecture.

Module 6 connects directly to modern AI.

■ **SYSTEMS DEPTH**

OS, compilers, databases — every systems field rests on the hardware model you learn here.

★ **INTERVIEWS**

Architecture questions separate candidates.

This is signal that you understand computing deeply.

1. Course Logistics

How this course runs: 12 weeks, six modules, labs and contests — and how you'll be graded.

L01 • MODERN COMPUTER ARCHITECTURE

How the Course Runs

12 weeks • 24 lectures • 2 lectures + 2 labs each week.

■ 24 lectures

Two 60-minute lectures per week across 12 weeks.

■ **Hands-on labs**

Two labs weekly: build real circuits in Verilog, simulate, write assembly in MIPS.

◆ 6 modules

From digital logic up to multicore & AI accelerators — each 2 weeks, 4 lectures.

★ **Contests & project**

Regular contests keep skills sharp; a project ties concepts into something you build.

L01 • MODERN COMPUTER ARCHITECTURE

The Six-Module Journey

Each module builds on the last — from a single gate to an AI accelerator.

M1

Wk 1–2

Digital Foundations

Gates, Boolean algebra, combinational blocks, number systems.

M2

Wk 3–4

Sequential Circuits & Memory

Flip-flops, registers, counters, FSMs, register files.

M3

Wk 5–6

Computer Organization & ISA

Instruction sets, assembly, how programs become machine code.

M4

Wk 7–8

CPU Datapath & Pipelining

Build a working CPU; make it fast with pipelining.

M5

Wk 9–10

Memory Hierarchy & Cache

Why memory is slow, and how caches hide it.

M6

Wk 11–12

Parallelism & Modern Multicore, GPUs, TPUs — the hardware behind modern AI.

L01 • MODERN COMPUTER ARCHITECTURE

How You're Graded

Weighted to reward consistent effort, not just exam-cramming.

40%

20%

20%

10%

10%

100% total — spread across the semester

40%

End-sem exam

Comprehensive final.

20%

Contests

Regular skill checks — keep pace.

20%

Mid-sem exam

Halfway checkpoint.

10%

Projects

Build something real.

10%

Assignments

Weekly practice.

L01 • MODERN COMPUTER ARCHITECTURE

Tools We'll Use

Real industry-adjacent tools — you'll build and simulate, not just read.

■ **Verilog + Simulation**

Describe hardware in a real HDL and watch signals in a waveform viewer. Modules 1–2.

■ **MIPS**

A MIPS assembler & simulator — write and run assembly by hand. Module 3.

■ **Hardware boards**

Real hardware to see logic and timing in the physical world.

L01 • MODERN COMPUTER ARCHITECTURE

How to Succeed Here

This course rewards building intuition, not memorising facts.

1

Draw everything

Circuits, truth tables, timing — get it on paper. Architecture is a visual subject.

Do the labs seriously
You truly learn this by building and simulating, not just watching slides.

3
Connect the layers
Always ask: how does this connect up to code, and down to transistors?

4
Ask in lectures
Cold-calls and questions are normal here — confusion voiced early saves hours later.

5
Keep pace with contests
They're spaced so you can't cram. Steady effort beats a panic before finals.

6
Think in trade-offs
There's rarely one 'right' design — area vs speed vs power. That mindset is the goal.

2. The Abstraction Stack
From a switch made of silicon to a neural network — every layer built on the one below it.

L01 · MODERN COMPUTER ARCHITECTURE
The Abstraction Stack
Each layer hides the one below it — you can work at one level trusting the rest.

AI / ML models
neural nets, training
Applications & languages
Python, your programs
Operating system
processes, memory mgmt
Instruction set (ISA)
the CPU's vocabulary
Datapath & control
registers, ALU, FSMs
Logic gates
AND, OR, NOT
Transistors
silicon switches
↑ higher = more abstract
This course climbs the middle: gates → datapath → ISA — the heart of the stack.

L01 · MODERN COMPUTER ARCHITECTURE
One Line of Code, Every Layer
Watch $A[i] = B[i] + C[i]$ descend from Python to silicon.

Python
 $A[i] = B[i] + C[i]$
You write one obvious line.
↓
Machine code
`lw · lw · add · sw`
Compiler turns it into CPU instructions.
↓
Datapath
regfile → ALU → regfile
Registers feed an adder; result written back.
↓
Gates & transistors
XOR, AND, carry logic
The adder is gates; gates are transistors.
Same computation, four languages of description. By this course's end, you'll speak all four.

L01 · MODERN COMPUTER ARCHITECTURE
Where We Start: The Bottom
Today we begin at the very foundation — the logic of 1s and 0s.
EVERYTHING RESTS ON THIS
A transistor is just a switch: on or off, 1 or 0.
Combine switches and you get logic gates. Combine gates and you get arithmetic, memory, and eventually a CPU.
The mathematics that governs how 1s and 0s combine is Boolean algebra — and that's exactly where we start today.

TODAY'S DESTINATION
1
Boolean values
what 1 and 0 really mean
2
The core gates
AND, OR, NOT, XOR
3
Truth tables
the complete behaviour of a gate
4
Boolean laws
the rules for simplifying logic
3. Boolean Basics
The algebra of true and false — invented by George Boole in 1847, and the language every computer speaks.

L01 · MODERN COMPUTER ARCHITECTURE
Boolean Values: Just 1 and 0
Two values, infinite possibility. Everything digital is built from this binary choice.

1
TRUE · HIGH · ON
voltage present (~3.3V or 5V)
0
FALSE · LOW · OFF
no voltage (~0V, ground)
The big idea: represent everything — numbers, text, images, instructions — as patterns of these two values.
Reliability comes from only needing to tell 'voltage' from 'no voltage'.

L01 · MODERN COMPUTER ARCHITECTURE
The NOT Gate (Inverter)
One input, one output — it simply flips the value. 1 becomes 0, 0 becomes 1.
 $Y = A'$ (also written NOT A, or $\neg A$)

TRUTH TABLE

A
Y
0
1
1
1
0

The simplest gate — but essential. Inversion is what lets logic express 'not this'.

L01 · MODERN COMPUTER ARCHITECTURE

The AND Gate

Output is 1 only when BOTH inputs are 1. Think: a series of two switches.

$Y = A \cdot B$ (also written $A \text{ AND } B$)

TRUTH TABLE

A
B
Y
0
0
0
0
0
1
0
1
0
0
0
1
1
1

Only the last row — both 1 — gives output 1. 'All conditions must hold.'

L01 · MODERN COMPUTER ARCHITECTURE

The OR Gate

Output is 1 when AT LEAST ONE input is 1. Think: two switches in parallel.

$Y = A + B$ (also written $A \text{ OR } B$)

TRUTH TABLE

A
B
Y
0
0
0
0
0
1
1
1
1
0
1
1
1
1
1

Only the first row — both 0 — gives 0. 'Any one condition is enough.'

L01 · MODERN COMPUTER ARCHITECTURE

The NAND Gate

Output is 0 when both the inputs are 1.

TRUTH TABLE

A
B
Y
0
0
0
0
1
1
1
1
1
0
1
1
1
1
0

Output is 1 when both the inputs are 0.

L01 · MODERN COMPUTER ARCHITECTURE

The NOR Gate

Output is 1 when the inputs DIFFER. The 'exclusive or' — one or the other, not both.

TRUTH TABLE

A
B
Y
0
0
0
0
0
1
1
1
1
0
1
1
1
1
0

Same inputs → 0, different → 1. XOR is the heart of the adder you'll build in L3.

L01 · MODERN COMPUTER ARCHITECTURE

The XNOR Gate

Output is 1 when the inputs are same.

TRUTH TABLE

A
B
Y
0
0
1
0
1
0
1
0
0
1
1
1

Same inputs then output is $\rightarrow$ 1, different $\rightarrow$ 0.

L01 · MODERN COMPUTER ARCHITECTURE

The Four Gates at a Glance

Same two inputs A, B — four different rules for the output.

AND

A · B
A
B
Y
0
0
0
0
1
0
1
0
0
1
1
1

OR

A + B
A
B
Y
0
0
0
0
1
1
0
1
1
1
1
1

XOR

A ⊕ B
A
B
Y
0
0
0
0
1
1
1
1
0
1
1
0

NAND

(A · B)'
A
B
Y
0
0
1
0
1
1
1
1
0
1
1
1

Notice: NAND is just AND with the output flipped. That tiny change makes it something special — next slide.

L01 · MODERN COMPUTER ARCHITECTURE

NAND & NOR: The Universal Gates

A teaser for L2 — from just one gate type, you can build everything.

ONE GATE TO RULE THEM ALL

NAND = NOT-AND. NOR = NOT-OR.

Remarkable fact: using ONLY NAND gates, you can build AND, OR, NOT — and therefore ANY logic circuit at all.

NAND is called 'functionally complete' or universal. NOR is too.

e.g. NOT A = A NAND A

Feed A into both inputs of a NAND $\rightarrow$ you get NOT A.

WHY IT'S HUGE

Chip factories can optimise for making ONE gate type extremely well.

NAND is cheap, fast, and dense in silicon — so real chips use enormous numbers of them.

We'll prove NAND's universality properly in L2, then use it to minimise real circuits.

L01 · MODERN COMPUTER ARCHITECTURE

Quick Check

Show of hands — no notes.

Q. For inputs A = 1, B = 0 — which gate outputs 1: AND, OR, or both?

8
4
2
1
32 + 8 + 2 = 42
Just read off the 1-columns and sum their weights.
DECIMAL → BINARY
Repeatedly divide by 2; read remainders bottom-up.
42 ÷ 2 = 21 r 0
21 ÷ 2 = 10 r 1
10 ÷ 2 = 5 r 0
5 ÷ 2 = 2 r 1
2 ÷ 2 = 1 r 0
1 ÷ 2 = 0 r 1
read up ↑ = **101010**
L04 • MODERN COMPUTER ARCHITECTURE
Hexadecimal: Binary for Humans
One hex digit = exactly 4 bits. That's the whole reason it exists.
Group binary into nibbles of 4 — each becomes one hex digit:
1
0
1
1
0
1
↓
↓
B
5
10110101 = 0xB5 = 181
HEX DIGITS BEYOND 9
Hex
Dec
Binary
A
10
1010
B
11
1011
C
12
1100
D
13
1101
E
14
1110
F
15
1111
Two hex digits = one byte. That's why colors (#FF8800), memory addresses, and MAC addresses all use hex.
L04 • MODERN COMPUTER ARCHITECTURE
Worked Example: 0x2A
Convert one hex value to binary and decimal, step by step.
STEP 1 — Hex to binary (nibble each digit)
2
→ **0010**
A
→ **1010**
0x2A = 0010 1010
STEP 2 — Binary to decimal (sum the 1-weights)
0
0
1
0
1
0
1
0
0
128
64
32
16
8
4
2
1
32 + 8 + 2 = 4210
ANSWER
0x2A
=
0010 1010
=
4210
L04 • MODERN COMPUTER ARCHITECTURE
Powers of 2 — Memorise These
The multiplication table of computer architecture.
2⁰
1
2¹
2
2²
4
2³
8
2⁴
16
2⁵
32
2⁶
64
2⁷
128
2⁸
256
THE BIG MILESTONES
Power
Value
Name
2⁰
1,024
1 KB

2²⁰
1,048,576
1 MB
2³⁰
~1 billion
1 GB
2⁴⁰
~1 trillion
1 TB
WHY IT PAYS OFF
Instantly know an n-bit range.
8 bits → 2⁸ = 256 values (0-255).
16 bits → 65,536. 32 bits → ~4.3 billion.
This is why a 32-bit counter overflows at ~4.3 billion —
the
exact bug that broke YouTube's view counter.
2. Signed Integers
Binary is naturally unsigned. Representing negatives
took three tries to get
right.
L04 • MODERN COMPUTER ARCHITECTURE
Attempt 1: Sign-Magnitude
The obvious idea: use the top bit as a sign flag. Intuitive — but flawed.
Leftmost bit = sign (0 = +, 1 = -). Remaining 7 bits = magnitude.
0
0
0
1
0
0
1
0
0
= **+18**
0
0
0
1
0
0
1
0
= **-18**
Only the sign bit flips between +18 and -18.
PROBLEM
00000000 = +0 and 10000000 = -0. Two zeros! And addition needs special
sign-checking logic — extra hardware.
VERDICT
✓ Easy for humans to read.
✗ Two representations of zero.
✗ Addition doesn't 'just work' — the
hardware must inspect signs and decide whether to add or subtract.
Used today only for floating-point sign bits —
never for integers.
L04 • MODERN COMPUTER ARCHITECTURE
Attempt 2: One's Complement
To negate, flip every bit. Closer — but zero is still doubled.
Negate by inverting ALL bits (NOT each one).
0
0
0
1
0
0
1
0
= **+18**
flip every bit ↓
1
1
1
0
1
1
1
0
= **-18**
STILL A PROBLEM
00000000 = +0 and 11111111 = -0. Two zeros
again. Also needs an
'end-around carry' fix for addition.
PROGRESS
✓ Negation is trivial: just NOT.
✓ Addition nearly works...
✗ ...but needs the carry-out added back in ('end-around carry').
✗ STILL two zeros.
So close. One tiny tweak fixes everything →
L04 • MODERN COMPUTER ARCHITECTURE
Attempt 3: Two's Complement ✓
Flip all bits, then add 1. This one tweak fixes everything.
Negate: invert every bit, then add 1.
0
0
0
1
0
0
1
0
+18
invert ↓
1
1
1
0
1
1
1
0
(1's comp)
add 1 ↓
1
1
1
0
1
1
1

0
= **-18**
WHY IT WINS
✓ Exactly ONE zero (00000000).
✓ Ordinary binary addition just works — for positives AND negatives, no special cases.
✓ Subtraction = add the negative. ONE
adder circuit does everything.
This is why every modern CPU uses two's
complement for signed integers. Universal.
The trick: negation and addition use the SAME hardware. That's the entire payoff.
L04 • MODERN COMPUTER ARCHITECTURE
Reading a Two's Complement Number
The top bit has NEGATIVE weight. That's the whole secret.
The most-significant bit's place value is negative:
1
1
1
0
1
1
1
0
-128
64
32
16
8
4
2
1
-128 + 64 + 32 + 8 + 4 + 2 = -18
The -128 weight (red) is what makes the number negative; the rest add back positive value.
NEGATE = INVERT + 1 (works both ways)
To go from +x to -x OR from -x to +x, the operation is identical: flip all bits, add 1. Symmetric and simple.
QUICK SIGN CHECK
The MSB alone tells you the sign: 0 = non-negative, 1 = negative. No need to decode the whole number to know
if it's below zero.
L04 • MODERN COMPUTER ARCHITECTURE
Range & the Number Wheel
For n bits: -2⁽ⁿ⁻¹⁾ to +2⁽ⁿ⁻¹⁾-1. And the numbers wrap in a circle.
SIGNED RANGE BY WIDTH
Bits
Min
Max
4
-8
+7
8
-128
+127
16
-32,768
+32,767
32
-2.1 billion
+2.1 billion
Note the asymmetry: one more negative value than positive
(because 0 takes a 'positive' slot). That lone extra negative — like
-128 in 8 bits — has no positive twin.
L04 • MODERN COMPUTER ARCHITECTURE
Sign Extension
Widening a number? Copy the sign bit — don't just pad with zeros.
To widen 4-bit → 8-bit, repeat the sign bit into the new positions:
+5:
0
1
0
1
→
1
1
1
1
1
1
The copied sign bits (highlighted) preserve the value. Both still equal ±5.
Padding -5 with ZEROS would give 00001011 = +11. Wrong! Always extend
the sign.
WHERE YOU HIT THIS
Every time a CPU loads a byte into a 32-bit
register.
MIPS & RISC-V have distinct load
instructions: 'load byte' (sign-extends) vs 'load byte unsigned' (zero-pads).
Picking the wrong one is a classic bug —
you'll meet it again in Module 3.
L04 • MODERN COMPUTER ARCHITECTURE
Quick Check
Show of hands — decode this byte.
Q. In 8-bit two's complement, what decimal value is 10000000 ?
A
0

−128
C
+128
D
−0
Trap: it looks like −0 (sign bit set, magnitude 0) — but in two's complement there's no −0. It's the most negative value.
B
✓ MSB weight = −128, rest = 0
3. Binary Arithmetic
Addition works exactly like grade-school carrying.
Subtraction is just addition in disguise.
L04 · MODERN COMPUTER ARCHITECTURE
Binary Addition
Four rules, column by column, carrying the 1 — just like decimal.
THE FOUR RULES
0 + 0 = 0
0 + 1 = 1
1 + 1 = 10 (0, carry 1)
1 + 1 + 1 = 11 (1, carry 1)
WORKED: 13 + 11 (4-bit)
carries
1
1
1
1
A = 13
1
1
1
0
B = 11
1
0
1
1
= 24
1
1
0
0
0
0
carry-out (5th bit) means the 4-bit result overflowed — more on that soon.
L04 · MODERN COMPUTER ARCHITECTURE
Subtraction = Adding the Negative
No subtractor circuit needed. Negate the second operand and add.
A − B becomes A + (−B) = A + (invert B, add 1)
1
Take B
The number you're subtracting.
2
Two's-complement it
Invert all bits, add 1 → that's
−B.
3
Add to A
Feed A and −B into the SAME
adder you already built.
4
Discard final carry
The carry-out beyond the width is ignored for signed results.
L04 · MODERN COMPUTER ARCHITECTURE
Worked: 5 − 3 in 4-Bit
Watch subtraction happen entirely through addition.
STEP 1 — Encode +3, then negate to get −3
+3 =
0
0
1
1
invert+1 → −3 =
1
1
0
0
STEP 2 — Add 5 + (−3)
+5 =
0
1
1
0
1
−3 =
1
1
0
0
= 1
0
0
1
0
discard the 5th-bit carry → 0010 = 2 ✓
THE MAGIC
5 − 3 = 2. Correct!
We never subtracted. We negated 3 and added.
The carry-out past bit 4 is simply thrown away — and the signed answer is right.
One adder. Add and subtract. This is the entire reason two's complement won.
4. Overflow
When the true answer won't fit in the bits you have — and the result silently lies.
L04 · MODERN COMPUTER ARCHITECTURE
What Is Overflow?
When a correct sum lands outside the representable range, it wraps — silently.
EXAMPLE — 4-bit signed (range −8 to +7)
Add +5 + +4 — both positive, answer should be +9...
+5 =
0
1
1

0
1
+4 =
0
1
0
0
=
1
0
0
1
= −7 ?!
Two positives gave a NEGATIVE. +9 doesn't fit in 4-bit signed (max +7), so it wrapped past the top of the wheel into negative territory.
The hardware didn't crash — it just gave a wrong answer.
THE DANGER
Overflow is SILENT.
No error, no exception by default — just a wrong number that flows on into the next computation.
That's what makes it dangerous. The CPU flags it (next slide) — but only if your code checks the flag.
L04 · MODERN COMPUTER ARCHITECTURE
How Hardware Detects Overflow
One elegant rule: compare the carry INTO and OUT of the sign bit.
THE RULE (signed)
Overflow = Cin(MSB) ⊕ Cout(MSB)
If the carry INTO the sign-bit column differs from the carry OUT of it, overflow occurred.
Just ONE extra XOR gate on your adder detects every signed overflow.
Cin=1, Cout=0 → 1 ⊕ 0 = 1 → OVERFLOW
THE INTUITIVE VERSION
(+) + (+) = (−)
Two positives can't sum to a negative → OVERFLOW
(−) + (−) = (+)
Two negatives can't sum to a positive → OVERFLOW
(+) + (−)
Opposite signs can NEVER overflow → always safe
L04 · MODERN COMPUTER ARCHITECTURE
Signed vs Unsigned Overflow
Same bits, different overflow rules — the CPU computes both flags every time.
SIGNED overflow
Detected by: Cin ⊕ Cout at the sign bit.
Means: the true result fell outside −2^{n−1} ... +2^{n−1}−1.
Flag name (x86): OF (Overflow Flag).
Your code reads OF if it cares about signed overflow.
UNSIGNED overflow
Detected by: the carry-out of the MSB alone.
Means: the true result exceeded 2ⁿ−1 (didn't fit).
Flag name (x86): CF (Carry Flag).
Your code reads CF if the numbers are unsigned.
Key insight: the ALU doesn't know if YOU meant signed or unsigned. It computes both flags; your instructions decide which matters.
L04 · MODERN COMPUTER ARCHITECTURE
Overflow: Four Cases
4-bit signed. Watch which combinations overflow and which don't.
+5 + (+4)
0101 + 0100
= 1001 = −7
X
OVERFLOW
(+)+(+)=(−)
−6 + (−5)
1010 + 1011
= 0101 = +5
X
OVERFLOW
(−)+(−)=(+)
+5 + (−3)
0101 + 1101
= 0010 = +2
✓
OK
opposite signs
+3 + (+2)
0011 + 0010
= 0101 = +5
✓
OK
fits in range
The pattern is unmistakable: overflow happens ONLY when both inputs share a sign and the result flips it.
L04 · MODERN COMPUTER ARCHITECTURE
When Overflow Broke the Real World
These aren't textbook hypotheticals — they're famous, expensive failures.
Ariane 5 (1996)
A 64-bit float converted to 16-bit signed int overflowed. The rocket self-destructed 37s after launch. Cost: ~\$370 million.
Gangnam Style (2014)
YouTube's view counter was a 32-bit signed int, max ~2.1 billion. The video blew past it — forcing a switch to 64-bit.
Boeing 787 (2015)
A counter overflowing after 248 days of continuous power could shut down all generators. Fix: reboot every 248 days.
Pac-Man level 256
An 8-bit level counter overflowed at 256, corrupting the level into the infamous unplayable 'kill screen'.
L04 · MODERN COMPUTER ARCHITECTURE
Common Mistakes to Avoid
The slips that cost marks — and cause real bugs.
X

Zero-padding a negative
Widening −5 as 0000_1011 gives +11. Always SIGN-extend,
not zero-pad.
X
Forgetting the +1
Two's complement negate is invert AND add 1. One's complement (invert only) is a different, obsolete scheme.
X
Misreading the MSB as −0
10000000 isn't −0 — there's no −0 in two's complement. It's the most negative value.
X
Confusing the two overflows
Signed overflow ≠ carry-out. Carry-out flags UNSIGNED overflow; sign-bit carry mismatch flags SIGNED.
X
Assuming symmetric range
8-bit is −128 to +127, not −127 to +127. One extra negative, no positive twin.
X
Thinking overflow throws an error
By default it's silent — the wrong number just flows onward. You must check the flag.
L04 · MODERN COMPUTER ARCHITECTURE
Key Takeaways
Five sentences that underpin every integer a computer touches.
01
Bits carry no inherent meaning — a number system is a convention you apply (base 2 for machines, base 16 for readable binary).
02
One hex digit maps to exactly 4 bits; two hex digits = one byte — that's the entire reason hex exists.
03
Two's complement (invert + 1) is the winner: one zero, and one adder handles both addition and subtraction.
04
In two's complement the MSB carries a NEGATIVE weight, and widening a value means sign-extending, not zero-padding.
05
Overflow is silent: it happens when two same-signed numbers produce the wrong sign, detected by Cin ⊕ Cout at the sign bit.
L04 · MODERN COMPUTER ARCHITECTURE
Practice & Homework
Convert, encode, and hunt for overflow.
01
Base Triathlon
Convert 0xB7 to binary and to signed decimal (8-bit). Then −45 to 8-bit two's complement.
02
Two's Complement Lab
Encode +18 and −18 in 8 bits. Verify that negating −18 gives back +18.
03
Overflow Drill
Add several pairs of 4-bit signed numbers; mark which overflow and justify with the sign rule.
EXIT TICKET
1. Convert −45 to 8-bit two's complement.
2. Compute 0110 + 1100 in 4-bit two's complement — did overflow occur?
3. Convert 0xB7 to binary and to a signed 8-bit decimal value.
L04 · MODERN COMPUTER ARCHITECTURE
Rapid-Fire Recap
Three questions — shout the answers.
Q1
The 8-bit two's complement of +18 is...
11101110
Q2
Signed overflow occurs when the result...
has the wrong sign
Q3
0xFF as an 8-bit SIGNED integer is...
−1
All three instant? You're ready for the labs and L5 (Sequential Circuits — latches & flip-flops).
END OF L04
One adder, every integer.
NEXT — L05
From Combinational to Sequential — latches, flip-flops, and the clock. How circuits gain MEMORY and start to remember what they computed.
Thanks
for watching!
=====

DOCUMENT

=====

LAB: 0101

Learns to Remember.
From Combinational to Sequential
How a circuit gains memory — and how a clock keeps that memory sane.
LAB 05 · MODERN COMPUTER ARCHITECTURE
What You'll Build Today
Three storage elements, in order of increasing discipline. By the end, you'll have the building block of every register on a CPU.
01
SR Latch
Two cross-coupled NORs. Holds one bit.
You'll meet its forbidden state — the warning sign of all latches.
02
D Latch

Tame the SR latch with an enable.
While E=1 the output follows D. While E=0 it remembers. Transparent and glitch-prone.
03
D Flip-Flop
Same idea, but it samples only on the clock's rising edge. The workhorse of every synchronous CPU.
Today is where Verilog stops being just combinational equations and starts looking like state.
LAB 05 • MODERN COMPUTER ARCHITECTURE
Recap • Where We Are
Labs 1–4 built circuits whose outputs are pure functions of their inputs. Today the output depends on history too.
01
Combinational so far
Adders, MUXes, decoders — outputs change the instant inputs change. No state. No clock.
02
But CPUs have state
Registers hold partial sums between cycles. The PC remembers what to fetch next. Memory remembers everything.
03
Feedback is the answer
Loop an output back into an input through a gate. With the right topology, the circuit holds onto a bit.
04
Then add discipline
Pure feedback is messy — async, glitch-sensitive. The clock turns chaos into a sequence of well-defined moments.
Every circuit from today onward will have memory. Get used to thinking in time, not just in logic.
LAB 05 • MODERN COMPUTER ARCHITECTURE
Why We Need Memory
Pure logic forgets the instant inputs change. To compute anything iterative, the circuit has to hold a value between steps.
f(x)
what combinational gates compute
0 ns
how long they remember
1 bit
the minimum unit of state we need
∞
complexity we can build once we have it
THE INSIGHT
Try writing a counter with only AND/OR/XOR. You can't. There's no way to ask 'what was Q a moment ago?' without something that remembers. That something is built from feedback.
L05 • MODERN COMPUTER ARCHITECTURE
Combinational vs Sequential
The single most important distinction in digital design.
COMBINATIONAL
Output = f(inputs now)
No memory — purely reactive.
Same inputs always give same output.
Examples: adders, MUXes, decoders.
The instant inputs change, output follows.
SEQUENTIAL
Output = f(inputs, past state)
Has memory — remembers history.
Same inputs can give different outputs.
Examples: registers, counters, FSMs.
A clock decides WHEN state updates.
The difference is one thing: a feedback loop. Add it, and a circuit can remember.
LAB 05 • MODERN COMPUTER ARCHITECTURE
The Idea • Feedback Creates State
Wire an output back into an input. If the topology supports two stable configurations, the circuit can be in either — and stay there.
WHAT YOU SEE
Two NOR gates feeding each other's inputs. Two stable configurations: (Q=1, Q'=0) and (Q=0, Q'=1). Either holds itself.
Each gate's output feeds the OTHER gate's input — that's the cross-coupling that holds state.
LAB 05 • MODERN COMPUTER ARCHITECTURE
SR Latch • Schematic & Truth Table
Two NOR gates, four input combinations, four very different behaviours.
S
R
Q (next)
Behaviour
0
0
Q (hold)
remember
1
0
1
set
0
1
0
reset
1
1
??
forbidden
Read row 4: with S=R=1, both NORs try to drive their outputs to 0 — but then both feedbacks are 0, which would force both back to 1. The latch oscillates.
LAB 05 • MODERN COMPUTER ARCHITECTURE
SR Latch • The Equations
Two assignments, each referring to the other's output. That mutual reference is the storage.
Q (the bit we read)

Q = ~(R | Q')
Q stays 1 unless R is asserted. If R=1, Q is pulled to 0 regardless of Q'.
Q' (the inverted bit)
Q' = ~(S | Q)
Q' stays 1 unless S is asserted. If S=1, Q' is pulled to 0, which lets Q rise to 1.
WHY THIS WORKS
When S=R=0 (no command), each equation simplifies to Q = ~Q', Q' = ~Q. That's a consistent pair — and it's stable. Two such pairs exist; the latch sits in whichever it last entered.
LAB 05 • MODERN COMPUTER ARCHITECTURE
SR Latch • Verilog
Two assigns, each referencing the other's wire. Verilog handles the simultaneous evaluation.
sr_latch.v
// NOR-based SR latch

input wire s,
input wire r,
output wire q,
output wire qbar
);
// Cross-coupled NORs:
// q depends on qbar, qbar depends on q.
assign q = ~(r | qbar);
assign qbar = ~(s | q);
endmodule
WHY THIS COMPILES
• Each assign is continuous — it re-evaluates whenever its RHS changes.
• When you change s or r, one assign reacts, then the other, then it settles.
• With S=R=1, both keep ringing forever — that's the simulator's way of telling you about the forbidden state.
• Real hardware doesn't oscillate cleanly; it lands at whichever NOR is slightly faster.
LAB 05 • MODERN COMPUTER ARCHITECTURE
The Forbidden State • S = R = 1
What the truth table calls 'forbidden' isn't an error in Verilog — it's the moment the latch stops being a latch.
1
Both fire
S=R=1 forces both NOR outputs to 0. So Q=0 and Q'=0 — they're equal, breaking the invariant Q ≠ Q'.
2
Release S and R
Now both inputs return to 0. The NORs
feed back the previous 0's — each tries to flip to 1. They race.
3
Whoever is faster wins
The final state depends on tiny propagation differences between the two gates. Non-deterministic — which is unusable in a CPU.
THE LESSON
Every latch design that exposes both S and R must guarantee they're never both 1. The D latch solves this by construction — R is always ~S.
L05 • MODERN COMPUTER ARCHITECTURE
SR Latch, NAND Version
Same idea, built from NAND gates — but the inputs are active-LOW.
NOR vs NAND LATCH
NOR latch: active-HIGH (1 = act).
NAND latch: active-LOW (0 = act).
Forbidden state flips too: for the NAND latch, S' = R' = 0 is the illegal one.
NAND latches are common in practice because NAND is the cheap, universal gate (recall L2).
Note the primes: S' and R' are active-LOW — a 0 triggers the action, not a 1.
LAB 05 • MODERN COMPUTER ARCHITECTURE
SR Latch • Testbench
Walk through all four cases. The forbidden case will show oscillation in iverilog — read the log carefully.
sr_tb.v

reg s, r; wire q, qbar;
sr_latch dut(.s(s), .r(r), .q(q), .qbar(qbar));
initial begin
\$dumpfile("sr.vcd"); \$dumpvars(0, sr_tb);
s=0; r=0; #5; // initial — unknown
s=1; r=0; #5; // SET -> q=1
s=0; r=0; #5; // HOLD -> q stays 1
s=0; r=1; #5; // RESET -> q=0
s=0; r=0; #5; // HOLD -> q stays 0
s=1; r=1; #5; // FORBIDDEN
s=0; r=0; #5; // metastable!
\$finish;
end
endmodule
WHAT TO LOOK FOR
• Steps 1–5: q follows the rules — set, hold, reset, hold.
• Step 6: q and qbar both go to 0 — illegal state.
• Step 7: after release, iverilog may keep toggling — that's the oscillation.
• If you see X (unknown) on q after step 7, the simulator gave up. That's the right behaviour.
LAB 05 • MODERN COMPUTER ARCHITECTURE
The SR Latch Is Hard to Use
Even when you avoid the forbidden state, two things make raw SR latches dangerous in a real design.
01
Two control lines is one too many
Every consumer of this latch has to remember not to assert S and R together. That's a rule humans break.
02
Any input glitch propagates

A 1 ns spike on S during HOLD will set the latch. There's no way to ignore noise without external gating.
03
No notion of 'now'
The latch reacts to inputs the instant they change. Synchronous design needs a clock to decide when things happen.
The D latch fixes problems 1 and 2. The flip-flop fixes problem 3.
L05 • MODERN COMPUTER ARCHITECTURE
Adding an Enable: The Gated Latch
Now the latch only listens when we say so — the first step toward a clock.
The Enable (E) controls the latch: when enabled, S sets Q = 1 and R resets Q = 0; when disabled, the previous state is retained.
WHY THIS MATTERS
When E = 1 (Enable): The latch responds to S and R — S = 1 sets Q = 1, while R = 1 resets Q = 0.
When E = 0 (Disable): The latch holds its previous state, so Q remains unchanged.
LAB 05 • MODERN COMPUTER ARCHITECTURE
D Latch • One Input, One Enable
Take the SR latch. Tie R = ~S internally. Add an enable that gates everything. Now S and R can never both be 1.
S = D · E
R = ~D · E
01
When E = 0
Both S and R are 0. The SR latch is in HOLD. D can wiggle all it wants — output doesn't change.
02
When E = 1, D = 1
S=1, R=0. SR latch is SET.
Q = 1.
03
When E = 1, D = 0
S=0, R=1. SR latch is RESET. Q = 0.
While E is high, Q follows D — we call this 'transparent'. While E is low, Q holds.
LAB 05 • MODERN COMPUTER ARCHITECTURE
D Latch • Transparency
While E is high, Q tracks D step-for-step. While E is low, whatever D was at the falling edge of E is what Q holds.
D
E
Q
D
E
Q
← transparent →
← holding →
← transparent →
During the green windows, Q is a copy of D. During the amber window, Q remembers the last D from before E fell.
LAB 05 • MODERN COMPUTER ARCHITECTURE
D Latch • Verilog
Two assigns to compute S_int and R_int from D and E. Then reuse the SR latch as a submodule.
d_latch.v
// Transparent D latch built from an SR latch

input wire d,
input wire e,
output wire q,
output wire qbar
);
wire s_int, r_int;
assign s_int = d & e;
assign r_int = ~d & e;
sr_latch core(s(s_int), .r(r_int),
.q(q), .qbar(qbar));
endmodule
NOTES
• S and R are mutually exclusive by construction.
• When E=0, both s_int and r_int are 0 — the SR latch holds.
• Reusing sr_latch keeps the design hierarchical — like the FA from 2 HAs in Lab 3.
• Compile with both .v files: iverilog -o d.vvp sr_latch.v d_latch.v d_tb.v
Thanks
for watching!
=====

DOCUMENTS
=====

LAB 2
Logic Minimisation & Universal Gates
From truth tables to minimal circuits --- practice with the solution directly beside the problem.
35 Questions + Detailed Worked Solutions
Computer Architecture & Digital Logic • GATE Preparation
PRINT-READY LATEX EDITION
LAB 2 • MODERN COMPUTER ARCHITECTURE

SECTION A
Basic • Q1–Q7
STRATEGY — Canonical Forms
• SOP: 1 → uncomplemented, 0 → complemented
• POS: 1 → complemented, 0 → uncomplemented

• Minterm number = binary value of the input combination (MSB left)

• Maxterm number = same binary product of all maxterms where $F = 0$

Q1

Basic

2 marks - Evaluation

Evaluate $F = A \cdot B + A \cdot B' + A' \cdot B$ for $A=1, B=0$.

What is F?

WORKED SOLUTION

STEP 1 — Substitute

$A=1, B=0 \rightarrow B'=1$

STEP 2 — Terms

$A \cdot B=0, A \cdot B'=1, A' \cdot B=0 \rightarrow F=1$

FINAL ANSWER $\rightarrow F = 1$

Q2

Basic

3 marks - Truth Table $\rightarrow$ SOP

Write the canonical SOP for $F(A,B,C)$ from the truth table:

A

B

C

F

0

0

0

0

0

0

1

1

0

0

1

1

0

0

1

0

1

1

0

1

0

1

1

0

0

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

WORKED SOLUTION

SOP: $A'B'C + A'BC' + AB'C' + ABC$

Missing $\rightarrow$ maxterms 0,3,5,6

POS: $(A+B+C)(A+B'+C')(A'+B+C')(A'+B'+C)$

FINAL ANSWER $\rightarrow$ SOP = $\Sigma m(1,2,4,7)$

POS = $\Pi M(0,3,5,6)$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

SECTION B

Basic $\rightarrow$ Intermediate • Q6-Q13

STRATEGY — Algebraic Minimisation

• Apply identity, null, complement, idempotent, absorption, distributive laws

• Factor or create cancelling terms use consensus when useful

• Verify by substitution or truth-table check

• Count gates / literals for area–power–speed impact

Q8

Basic

2 marks - Law ID

Which law justifies $X + X \cdot Y = X$?

A. Distributive B. Absorption C. Idempotent D. DeMorgan

WORKED SOLUTION

$X + XY = X(1+Y) = X \cdot 1 = X$ (Absorption)

FINAL ANSWER $\rightarrow$ B. Absorption

Q9

Basic

2 marks - Law Application

Simplify $A + A'B$. Name the law used.

WORKED SOLUTION

$A + A'B = (A+A')(A+B) = 1 \cdot (A+B) = A+B$ (Distributive / cover)

FINAL ANSWER $\rightarrow A + B$

Q10

Intermediate

3 marks - Algebraic Min

Minimise $F = XY + X'Z + YZ$. Is the YZ term necessary?

WORKED SOLUTION

By consensus theorem, YZ is the consensus of XY and $X'Z \rightarrow$ redundant.

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

FINAL ANSWER $\rightarrow F = XY + X'Z$ (YZ can be dropped)

Q11

Intermediate

3 marks - Equivalence

Prove or disprove: $(A+B)(A'+C) = AC + A'B$.

WORKED SOLUTION

Expand LHS: $AA' + AC + A'B + BC = AC + A'B + BC$.

Consensus removes BC $\rightarrow AC + A'B$.

Equality holds after dropping consensus.

FINAL ANSWER $\rightarrow$ True (after consensus) $\blacksquare F = AC + A'B$

Q12

Intermediate

3 marks - Redundant Terms

Identify and remove all redundant terms from $F = A'B'C + A'BC + AB'C + ABC + A'B$.

WORKED SOLUTION

$A'B'C + A'BC = A'C$ $AB'C + ABC = AC$ then $A'C + AC = C$ remaining $A'B$. Result: $F = C + A'B$

FINAL ANSWER $\rightarrow F = C + A'B$

Q13

Intermediate

3 marks - Dual / Complement

Find the complement of $F = (A+B)(A'+C)$ and simplify.

WORKED SOLUTION

$F' = (A+B)' + (A'+C)' = A'B' + AC'$ (DeMorgan)

FINAL ANSWER $\rightarrow F' = A'B' + AC'$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

SECTION C

Intermediate • Q14-Q20

K-MAP STRATEGY

• Fill correctly (Gray-code order)

• Largest groups only (powers of 2: 1,2,4,8...)

• Rectangular & adjacent (incl. wrap-around)

• Every 1 covered overlapping allowed

• Variables that change inside a group are eliminated

Q14

Intermediate

3 marks - Gray Code

Why must K-map order be 00-01-11-10 instead of natural binary 00-01-10-11?

WORKED SOLUTION

Adjacent cells must differ by exactly one bit for the combining theorem. Natural order places 10 next to 01 (two-bit difference). Gray code guarantees single-bit adjacency including wrap-around.

FINAL ANSWER $\rightarrow$ Gray code ensures single-bit adjacency required by combining theorem.

Q15

Intermediate

3 marks - 2-var K-map

Draw/fill the 2-variable K-map for

$F(A,B) = \Sigma m(0,2)$ and give the minimal SOP.

WORKED SOLUTION

B=0

B=1

A=0

1

0

A=1

1

0

Group of 2 (vertical) $\rightarrow B'$

FINAL ANSWER $\rightarrow F = B'$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

Q16

Intermediate

4 marks - 3-var K-map

Fill 3-var K-map for $F(A,B,C) = \Sigma m(0,2,3,5)$ and obtain minimal SOP.

WORKED SOLUTION

00

01

11

10

0

1

0

1

1

0

0

1

0

Groups: $A'C'$ (m_0+m_2 wrap), $A'B$ (m_2+m_3), $AB'C$ (m_5)

FINAL ANSWER $\rightarrow F = A'B + A'C' + AB'C$

Q17

Intermediate

3 marks - Legal Groups

Which groupings are legal on a 3-var K-map? (Select all)

A. Four corners B. 2×2 block C. Three adjacent 1s D. Two cells differing in two variables

WORKED SOLUTION

A legal (size 4 wrap). B legal. C illegal (size $\neq$ power of 2). D illegal (not adjacent).

FINAL ANSWER $\rightarrow$ A and B

Q18

Intermediate

3 marks - Variable Cancellation

On a 3-var K-map a group of four 1s covers cells m_0, m_2, m_4, m_6 .

What is the product term?

WORKED SOLUTION

Cells: 000, 010, 100, 110 $\rightarrow C$ is always 0 $\blacksquare$ A and B change $\rightarrow$ term = C'

FINAL ANSWER $\rightarrow C'$

Q19

Intermediate

3 marks - Group of 8

A 3-variable function has all eight minterms =

1. What is the minimal expression?

WORKED SOLUTION

The whole map is one group of 8 $\rightarrow F = 1$.

FINAL ANSWER $\rightarrow F = 1$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

Q20

Intermediate

4 marks - Fill Empty Map

Given $F = A'C + AB$, place the 1s on a 3-var

K-map and list the minterms.

WORKED SOLUTION

$A'C$ covers m_1 (001) and m_3 (011). AB covers m_6 (110) and m_7 (111). Minterms: 1,3,6,7

FINAL ANSWER $\rightarrow \Sigma m(1,3,6,7)$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

SECTION D

Intermediate $\rightarrow$ Advanced • Q21-Q26

ADVANCED K-MAP STRATEGY

• Wrap-around and four-corner groups are valid

• Don't-cares (X) treated as 1 only when they enlarge a group

• Prime implicant = largest group that cannot be grown further

• Essential PI covers at least one 1 not covered by any other PI

• Minimal cover = all EPIs + smallest set of remaining PIs

Q21

Advanced

5 marks - 4-var + Corners

$F(A,B,C,D) = \Sigma m(0,2,5,7,8,10,13,15)$. Obtain

0
0
1Four corners $\rightarrow B'D \blacksquare$ quad m5,7,13,15 $\rightarrow BD$ FINAL ANSWER $\rightarrow F = B'D + BD$ Trap: Missing the four-corner group forces a non-minimal expression.
Q22

Advanced

4 marks - Wrap-around

On a 4-var K-map the cells m0, m2, m8, m10 are 1. What single product term covers all four?

WORKED SOLUTION

These are the four corners. Variables B and D are 0 throughout $\rightarrow B'D'$ FINAL ANSWER $\rightarrow B'D'$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

Q23

Advanced

6 marks - PI / EPI

For $F(A,B,C) = \sum m(0,1,2,5,6,7)$: List all PIs, identify EPIs, give one minimal cover.

WORKED SOLUTION

PIs: $A'B'$, $A'C'$, $B'C$, BC' , AC , AB One minimal cover: $A'B' + BC' + AC$ FINAL ANSWER $\rightarrow F = A'B' + BC' + AC$

Q24

Advanced

4 marks - Essential Count

A function has five prime implicants of which three are essential. What is the minimum number

of product terms in any minimal SOP?

WORKED SOLUTION

All three EPIs must appear. If they already cover every 1, the minimum is exactly 3.

FINAL ANSWER $\rightarrow$ At least 3 (exactly 3 if EPIs cover all 1s)

Q25

Advanced

4 marks - Overlapping Groups

True or False: Overlapping groups on a K-map are always illegal.

WORKED SOLUTION

False. Overlapping is legal and frequently necessary for largest groups and a minimal expression.

FINAL ANSWER $\rightarrow$ False

Q26

Advanced

5 marks - Minimal Cover Selection

F has PIs: $P1=A'B$, $P2=BC'$, $P3=AC$, $P4=A'C'$. The 1s covered uniquely are: m0 only by P4, m7

only by P3. Select a minimal cover.

WORKED SOLUTION

EPIs are P4 and P3. Remaining 1s covered by choosing among P1/P2. One minimal cover:

 $A'C' + AC + BC'$ (or $+ A'B$).FINAL ANSWER $\rightarrow$ Example: $A'C' + AC + BC'$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

SECTION E

Intermediate • Q27=Q30

DeMORGAN / BUBBLE STRATEGY

• Outside-in: $(XY)' = X' + Y'$, $(X + Y)' = X'Y'$

• Never stop after one application if nested complements remain

• Bubble on output $\Rightarrow$ bubbles on all inputs of the dual gate

• Two bubbles in series cancel

Q27

Intermediate

3 marks - Nested DeMorgan

Simplify $F = ((A+B)' \cdot C)' + (A \cdot B)'$. Show every step.

WORKED SOLUTION

 $(A+B)' = A'B'$, $(AB)' = A' + B'$ $F = (A'B' \cdot C)' + (A' + B') = (A+B+C)' + (A' + B') = 1$ FINAL ANSWER $\rightarrow F = 1$ Trap: Stopping after first DeMorgan leaves nested complements.
Q28

Intermediate

3 marks - Bubble Push

Convert $(A \cdot B)' + C$ into simplest form with primary inputs.

WORKED SOLUTION

 $(AB)' = A' + B' \rightarrow F = A' + B' + C$ FINAL ANSWER $\rightarrow F = A' + B' + C$

Q29

Intermediate

3 marks - Dual Form

Apply DeMorgan to convert $(A \cdot B \cdot C)'$ into an OR with inverted inputs.

WORKED SOLUTION

 $(ABC)' = A' + B' + C'$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

FINAL ANSWER $\rightarrow A' + B' + C'$

Q30

Intermediate

4 marks - Nested + Simplify

Simplify $F = ((A' + B)' + (A \cdot C'))'$.

WORKED SOLUTION

Inner: $(A' + B)' = AB'$, $(AC)' = A' + C'$ $F = (AB' + A' + C')' = (AB')' \cdot (A')' \cdot (C')' =$ $(A' + B) \cdot A \cdot C = ABC$ FINAL ANSWER $\rightarrow F = ABC$

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

SECTION F

Advanced • Q31=Q33

UNIVERSAL GATE STRATEGY

• Simplify the function first

• NOT = NAND/NOR with tied inputs

• AND = NAND + inverter (or DeMorgan form)

• OR = inverted inputs into NAND

• Count 2-input gates $\blacksquare$ multi-level may reduce total

• Verify by expanding back

Q31

Advanced

3 marks - NAND Primitives

Show 2-input NAND realisations of NOT, AND and OR. State gate counts.

WORKED SOLUTION

NOT: A tied $\rightarrow 1$ NANDAND: NAND then NAND-inverter $\rightarrow 2$ NANDsOR: invert both inputs then NAND $\rightarrow 3$ NANDsFINAL ANSWER $\rightarrow$ NOT=1, AND=2, OR=3

Q32

Advanced

3 marks - NOR Primitives

Show 2-input NOR realisations of NOT, OR and AND. State gate counts.

WORKED SOLUTION

NOT: A tied $\rightarrow 1$ NOROR: NOR then NOR-inverter $\rightarrow 2$ NORsAND: invert inputs then NOR $\rightarrow 3$ NORsFINAL ANSWER $\rightarrow$ NOT=1, OR=2, AND=3

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

Q33

Advanced

5 marks - NAND-only F

Implement $F = AB + C$ using only 2-input NAND gates. Minimise gate count.

WORKED SOLUTION

 $F = ((AB)' \cdot C')' \rightarrow$ three NANDs: one for AB, one for C' , one final.FINAL ANSWER $\rightarrow 3$ NAND gates

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

LAB 2 • MODERN COMPUTER ARCHITECTURE

SECTION G

Advanced • Q34=Q35

Q34

GATE-Style

2 marks - MCQ

Minimum number of 2-input NAND gates to implement

 $F = (A+B)(C+D)$ is

A. 3 B. 4 C. 5 D. 6

WORKED SOLUTION

Classic multi-level NAND count for this function is 6.

FINAL ANSWER $\rightarrow$ D. 6

Q35

GATE-Style / MSQ

3 marks - Concepts

Which statements are true?

A. Number of PIs always equals number of EPIs

B. Minimal SOP must contain every EPI

C. Don't-care minterms may appear in a PI

D. Two minimal covers can differ in number of product terms (classic definition)

WORKED SOLUTION

A False $\cdot$ B True $\cdot$ C True $\cdot$ D generally False under classic term-count minimalityFINAL ANSWER $\rightarrow$ B and C

Sources & Further Practice

• GATE Overflow – Digital Logic / K-Maps archives

• EXAMSIDE GATE ECE / CSE previous-year collections

• NPTEL Digital Circuits (IIT Madras, IIT Bombay)

• Morris Mano – Digital Design $\blacksquare$ Roth – Fundamentals of Logic Design

Logic Minimisation & Universal Gates • 35 Questions + Worked Solutions

End of Lab 2

Fewer gates. Same truth table. Better design.

35 QUESTIONS • WORKED SOLUTIONS • PRINT READY

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

=====

L05 • MODERN COMPUTER ARCHITECTURE

Level-Triggered vs Edge-Triggered

The difference between 'listen the whole time' and 'listen for one instant'.

LEVEL-TRIGGERED (latch)

Active during an entire clock LEVEL (high or low).

Transparent — output tracks input the whole window.

Simpler, fewer gates.

Risky: data can slip through unintentionally.

EDGE-TRIGGERED (flip-flop)

Active only at the clock EDGE (rising or falling).

Samples input at one instant, then locks.

More gates, but predictable.

The safe choice for pipelines & registers.

 $\uparrow$ rising edgeAn edge is a MOMENT (0 $\rightarrow$ 1 transition), not a duration. That's why edge-triggering

samples clearly.

LAB 05 • MODERN COMPUTER ARCHITECTURE

Transparency Is Sometimes a Problem

Anything that wiggles on D while E is high lands on Q. Glitches included. That's why CPUs don't use latches for the critical path.

D LATCH • TRANSPARENT

D FLIP-FLOP • EDGE TRIGGERED

Same noisy D. Same clock. Latch propagates everything. Flip-flop ignores everything except the value at the rising edge.

L05 • MODERN COMPUTER ARCHITECTURE

The D Flip-Flop

Samples D on the rising clock edge, then holds it for the whole cycle.

The small triangle on the clock input is the universal symbol for

'edge-triggered'.

THE RULE

At each rising edge of CLK:

 $Q \leftarrow D$ (sampled once).

Between edges: Q is frozen, no matter what D does.

So D can wiggle all it wants — only its value AT the

edge is captured. This solves the transparency

problem completely.

L05 • MODERN COMPUTER ARCHITECTURE

How Edge-Triggering Works: Master-Slave

Two latches in series on opposite clock phases — data can only advance one step per cycle.

Master captures D while CLK is high; slave passes it on the falling edge. Result: one clean

update per cycle.

THE HAND-OFF

The two latches are NEVER transparent

at the same time.

When the master listens, the slave is

locked — and vice versa.

So data can't race straight through. It's a

two-stage airlock for one bit.

LAB 05 • MODERN COMPUTER ARCHITECTURE

From Latch to Flip-Flop • The Clock

An edge-triggered $\blacksquare$ ip-flop is a latch that opens for an instant on the clock edge — too briefly for glitches to pass.

LAB 05 • MODERN COMPUTER ARCHITECTURE

D Flip-Flop • Verilog

Procedural this time. The `always @(posedge clk)` block tells the simulator: 'evaluate this only at the rising edge of clk'.

d_ff.v

// Positive-edge-triggered D flip-flop

```
input wire clk,
input wire d,
output reg q // reg, not wire
);
always @(posedge clk) begin
q <= d; // nonblocking
end
endmodule
```

THREE THINGS NEW

• output reg — q is now a procedural variable. wire can't be assigned inside always.

• always @(posedge clk) — the block runs once

each time clk goes 0 $\rightarrow$ 1. Never any other time.

• <= (nonblocking) — the update happens at the

end of the time step. Next slide explains why this matters.

LAB 05 • MODERN COMPUTER ARCHITECTURE

D Flip-Flop • Testbench

Drive a clock. Drive D with deliberate noise between edges. Verify Q only changes on rising edges.

ff_tb.v

```
reg clk, d; wire q;
d_ff dut(.clk(clk), .d(d), .q(q));
initial clk = 0;
always #5 clk = ~clk; // period = 10 ns
initial begin
$dumpfile("ff.vcd"); $dumpvars(0, ff_tb);
d = 0; #3; // before 1st edge
d = 1; #1; // glitch up
d = 0; #1; // glitch down - edge at t=5
d = 1; #6; // settle to 1
d = 1; #10; // hold across 2 edges
d = 0; #10;
$finish;
end
endmodule
```

EXPECTED Q

• Q stays 0 until t=5.

• At t=5 (rising edge), D was 0 $\rightarrow$ Q stays 0.• At t=15, D = 1 $\rightarrow$ Q goes to 1.

• All intermediate glitches on D are ignored.

LAB 05 • MODERN COMPUTER ARCHITECTURE

D Flip-Flop • What the Waveform Shows

CLK ticks. D wiggles. Q updates only at the up-arrows. Everything else gets thrown away.

Notice Q ignores all the glitches on D between edges. It samples only at the four red arrows.

LAB 05 • MODERN COMPUTER ARCHITECTURE

Side by Side • D Latch vs D Flip-Flop

Same circuit symbol, very different behaviour. Memorize this table — it's the most-asked digital-design interview question.

D Latch

D Flip-Flop

Control signal

Enable (level)
Clock (edge)
Output during 'active'
Q follows D continuously
Q samples once, then holds
Transparent?
Yes — D wiggles, Q wiggles
No — Q is opaque between edges
Glitches on D propagate?
Yes
No
Verilog body

```
assign (or always @(*))  
always @(posedge clk)  
Output type
```

wire
reg
Used in modern CPUs?
Rarely — special cases only
Everywhere — pipelines, registers
L05 • MODERN COMPUTER ARCHITECTURE
Setup Time & Hold Time
Data must be STABLE in a window around the clock edge — before and after.
THE ANALOGY
Taking a photo of a moving object.
The subject must hold still just before
AND just after the shutter clicks, or the
picture blurs.
Same for a flip-flop: change D inside the
setup/hold window and the captured bit is a
blur — undermined.
LAB 05 • MODERN COMPUTER ARCHITECTURE
Setup & Hold Time • The Fine Print
The flip-flop only samples reliably if D is stable for a small window around the rising edge. Violate that window and Q goes metastable.

```
t_setup  
how long D must be  
stable BEFORE the edge
```

t_hold
how long D must be
stable AFTER the edge

```
t_cq  
delay from the edge  
to Q being valid
```

WHAT GOES WRONG
If D changes inside the [edge – t_setup, edge + t_hold] window, the
flip-flop captures neither 0 nor 1 cleanly. Q
enters a metastable state — somewhere between rails — and may
stay there for many gate-delays before snapping
to one or the other. This is why CPU pipelines have synchronizers at
every clock-domain crossing.
LAB 05 • MODERN COMPUTER ARCHITECTURE
When to Use a Latch
Flip-flops won the war, but latches still show up in three specific
places. Knowing why matters more than memorizing them.

01
SRAM cells
Every bit of cache is two cross-coupled
inverters — essentially an SR latch.
There are billions of these on a modern
die.

02
Time-borrowing
In pipelines too tight for one clock
period, designers insert a latch
between two flop stages. The latch lets
a slow operation 'borrow' time from
the next cycle.

03
Asynchronous resets
Reset logic is often latched, not flopped
— you want to be sure the reset signal
sticks even if the clock isn't running
yet.
Everywhere else — pipeline registers, control state, the PC, ALU
result registers — it's flip-flops.
LAB 05 • MODERN COMPUTER ARCHITECTURE
Common Errors • And the Fix
Five traps that catch most students. Glance now and save yourself
half an hour of waveform-staring.

X
Declared q as wire inside the flip-flop
Procedural assignments need reg. Verilog will refuse to
compile and the error
message points at the wrong line. Change wire to reg.
X
Used '=' instead of '<=' in posedge block
Compiles fine, often even simulates fine. But cascaded stages get
the wrong values
when clk-skew matters. Use <= in every posedge
block, every time.

X
Forgot the begin/end on multiline blocks
Verilog only ties the first statement to the @(posedge). Subsequent
lines run as
combinational logic. Wrap multi-line bodies in begin ... end.
X
Drove S and R high simultaneously in SR test
iverilog will report 'X' on q after release. That's not a bug — it's the
forbidden state.
Re-read slide 9 and gate your inputs.
X
Built a flip-flop and called it a latch
If your D propagates only at posedge clk, you have a flip-flop. A real
latch uses
always @(*) or continuous assigns with an enable
wire.
LAB 05 • MODERN COMPUTER ARCHITECTURE
Why This Matters • What's Next
Every register on the CPU you'll build later is a row of D flip-flops.
Every pipeline stage is a flop. Today is the foundation of all of that.
TODAY
One bit, one flop

You built the elementary unit of digital memory. From here,
everything scales.
TOMORROW
Registers, banks
Stack 32 flip-flops side by side, share a clock — you have a 32-bit
register. Build 32 of those, you have the register file.
REAL CHIPS
Pipelines and pacing
Every stage boundary in a CPU pipeline is a row of flops. Their
setup/hold determines the max clock frequency.
BEYOND
FSMs and control
Combine flops with combinational logic and you have a
finite-state
machine. That's how every controller in a CPU works.
You just learned, in 90 minutes, the storage cell that ships in every
CPU on Earth.
LAB 05 • MODERN COMPUTER ARCHITECTURE
Rapid-Fire Recap
Three questions — shout the answers.
Q1
S = R = 1 on an SR latch is called...
forbidden
Q2
A D latch is transparent when E equals...
1
Q3
A D flip-flop samples D on the clock's...
rising edge
All three instant? You've got the foundation — Lab 06 builds registers
and register files from exactly this cell.
LAB 05 • MODERN COMPUTER ARCHITECTURE
LAB 05 • MODERN COMPUTER ARCHITECTURE
✓
One latch. One flip-flop.
One bit that remembers.
Next: registers, register files, and the first real piece of a datapath.
END OF LAB 05 • LATCHES AND FLIP-FLOPS
Thanks

```
for  
watching!
```

```
=====
```

LAB 0106
Complement & Overflow
Can you Try 1 & 2 bit Comparator and
Multiplier by yourself ?
Explore Carry Look Ahead Adder and try to
figure out something which is worth noticing !
LAB 04 • MODERN COMPUTER ARCHITECTURE
Two's Complement • The Lay of the Land
Same 4 bits. Two readings. Whether a result is 'right' depends
entirely on which reading you intended.
U
UNSIGNED • 0 to 15
All bits are 'just bits'. 1111 = 15. Carry-out =
wraparound. 15 + 1
→ 0 + Cout. That's it. Cout IS the overflow
flag.
S
SIGNED (TWO'S COMPLEMENT) • -8 to +7
MSB is the sign bit. 1111 = -1, not 15. Cout is
meaningless here —
we need a different flag.
BITS
0000
0001
...
0111
1000
1001
1111
UNSIGNED
0
1
...
7
8
9
15
SIGNED
0
1
...
7
-8
-7
-1
LAB 04 • MODERN COMPUTER ARCHITECTURE
Unsigned Overflow • CF = Cout
If you treat your 4 bits as 0–15, the carry-out tells you the result didn't
fit. End of story.
CF (carry flag) = Cout
0001 = 1
+0001 = 1
0010 = 2
CF = 0
Result fits in 4 bits. No overflow.
1111 = 15
+0001 = 1
0000 = 0
CF = 1
15+1 = 16, but only 4 bits – the top bit fell
off.
1000 = 8
+1000 = 8
0000 = 0
CF = 1
8+8 = 16 = 10000. Bit 4 became Cout.

LAB 04 • MODERN COMPUTER ARCHITECTURE
Signed Overflow • OF = C_{in} ⊕ C_{out}
The XOR of the carry INTO the MSB and the carry OUT of the MSB.
That's the rule, and it just works.
OF (overflow flag) = C_{in-to-MSB} ⊕ C_{out-of-MSB}
THE INTUITION
Signed overflow happens when two positives add to a 'negative', or
two negatives add to a 'positive' — the sign bit lies about the
answer.
The MSB column is exactly where that lie happens. If the carry going
IN disagrees with the carry going OUT, the sign bit got flipped by
arithmetic — not by the real result. That's overflow.
WHERE TO TAP - In your CLA, c3 is the carry into
the MSB and c4 (= cout) is the carry out. So OF
= c3 ⊕ c4.
LAB 04 • MODERN COMPUTER ARCHITECTURE
Why That XOR Works
Two signs in, one sign out. Walk the MSB column. Overflow is
exactly the disagreement of its two carries.
A
B
Result sign
C_{in} in
C_{out} out
OF
Verdict
0 (+)
0 (+)
0 ✓ matches
0
0
no overflow
0 (+)
0 (+)
1 ✗ flipped
1
1
0
1
OVERFLOW
1 (-)
1 (-)
0 ✗ flipped
0
1
1
OVERFLOW
1 (-)
1 (-)
1 ✓ matches
1
1
0
no overflow
READ MIDDLE TWO ROWS • Same-sign inputs
producing the opposite sign ⇔ the MSB column's
two carries disagree.
LAB 04 • MODERN COMPUTER ARCHITECTURE
Overflow Flags • Two More Lines
Promote c3 and c4 to module outputs. One AND, one XOR, and
you're done.
cla_adder.v • with flags

```
input wire [3:0] a, b,  
input wire cin,  
output wire [3:0] sum,  
output wire cout, cf, of  
);  
// ... g, p, c1..c4, sum exactly as before ...  
// The two new flags:  
assign cf = c4; // unsigned overflow  
assign of = c3 ^ c4; // signed overflow  
assign cout = c4;  
endmodule
```

TWO FLAGS, ONE ADDER
cf is meaningful only if you read your inputs as
unsigned.
of is meaningful only if you read them as signed.
The adder emits both flags — your program (or
CPU flag register) picks. Exactly how x86 and ARM
work.
LAB 04 • MODERN COMPUTER ARCHITECTURE
Four Test Cases • Stressing Both Flags
Each case tests a different signed/unsigned scenario. Predict every
flag before you simulate.

01
0001 + 0001
CF=0 OF=0
unsigned 1+1=2 • signed +1+1=2 • quiet case
02
0111 + 0001
CF=0 OF=1
unsigned 7+1=8 (fine) • signed +7+1=-8 ✗
03
1111 + 0001
CF=1 OF=0
unsigned 15+1=16 ✗ wrapped • signed -1+1=0
(fine)
04
1000 + 1000
CF=1 OF=1
unsigned 8+8=16 ✗ • signed -8+-8=0 ✗ • both
broke
Drop these four into your testbench. Print a, b, sum, cf, of for each —
the pattern above is the spec.
LAB 04 • MODERN COMPUTER ARCHITECTURE
Walking Through Test Cases • Part 1
Trace the MSB column by hand. If sim disagrees, your flag logic is
wrong — not the arithmetic.
case 02
0111 + 0001 • signed overflow
0111 = +7
+ 0001 = +1

1000 = -8 (should be +8)
MSB: C_{in}=1 C_{out}=0
OF = 1 xor 0 = 1
CF = Cout = 0
case 03
1111 + 0001 • unsigned wrap

```
1111 = 15 (or -1)
+ 0001 = +1
-----
0000 = 0
MSB: Cin=1 Cout=1
OF = 1 xor 1 = 0
CF = Cout = 1
LAB 04 • MODERN COMPUTER ARCHITECTURE
Walking Through Test Cases • Part 2
The double-failure case, plus a 'should-be-clean' case to show what
a quiet adder looks like.
case 04
1000 + 1000 • both flags
1000 = 8 (or -8)
+ 1000 = 8 (or -8)
-----
0000 = 0
MSB: Cin=0 Cout=1
OF = 0 xor 1 = 1
CF = Cout = 1
sanity check
0011 + 0010 • the quiet case
0011 = +3
+ 0010 = +2
-----
0101 = +5
MSB: Cin=0 Cout=0
OF = 0 xor 0 = 0
CF = Cout = 0
LAB 04 • MODERN COMPUTER ARCHITECTURE
END OF LAB 04
One adder performs all.
One ground truth.
Two flags for what 'overflow'
really means.
Thanks
for
watching!
=====
=====
```

DOCUMENT 11

=====

=====

Lab: 0001.

Implementation.

The journey from a single transistor to the GPUs that train modern AI — starting with 1s and 0s.

Where It All Begins

Why Hardware Description Languages Exist?

The mental-model shift everything else in this lecture depends on.

20 Billion Switches, One Idea

1947: The Switch That Replaced the Vacuum Tube

The Integrated Circuit: From Wires to Wafers

1971: The First CPU on a Single Chip

Moore's Law: The Prediction That Ran the Industry

How Small Is 3nm, Really?

From A Few Gates to Billions

Why One Island Makes Almost Every Advanced Chip

Apple Silicon: One Chip, Every Concept in This Course

1847 + 1937: The Math That Was Waiting for the Transistor

How a chip is actually manufactured?

From Your First assign Statement to a Physical Chip: The Full Pipeline

Prerequisites

This lecture assumes nothing about prior coding or HDL exposure.

L1

Boolean algebra recap

L1's four core gates and truth tables

give the circuits we'll use as running examples — helpful, not required.

0

Zero coding background

No prior programming or HDL

experience is assumed anywhere in this lecture. Every construct is

introduced from first principles.

Everything here becomes muscle memory in Lab 1 — Verilog Setup & Basic Gate Implementation.

What You'll Build Today

By the end of this lab, you'll have all seven core gates running in simulation.

NOT

AND

1 if either input is 1

OR

Inverts the input

NAND

0 if both inputs are 1

PLUS

a single self-checking testbench, a generated VCD file, and a GTKWave screenshot of all four gates switching together.

NOR

1 if both inputs are 0

XOR

1 if either input is 1

XNOR

1 if both inputs are same

1 if both inputs are 1

Software Brain vs. Hardware Brain

One CPU executing instructions in order, versus thousands of gates switching at once.

Software: sequential

A program counter steps through instructions one at a time.

"Turn on lamp 1, then lamp 2, then lamp 3" is a loop — each line waits for the last to finish.

```
for (i=0;i<4;i++) lamp[i]=1;
```

Hardware: concurrent

Four real lamps wired to four real switches all respond the

instant power arrives — simultaneously, not in a queue.

Verilog describes this concurrency directly.

```
assign lamp = switch; // *4, at once
```

Every bug this mismatch causes in Lab 1 traces back to reading Verilog like a to-do list instead of a wiring diagram.

Simulation vs. Synthesis

An HDL file is compiled two completely different ways, for two completely different purposes.

■

Simulation

Runs your Verilog on a normal CPU, modeling circuit behaviour

over simulated time. Produces console output and waveforms.

Nothing physical is built — this is everything Lab 1 does.

■

Synthesis

A synthesis tool (Synopsys, Cadence) reads the same style of Verilog and generates a real gate-level netlist — the actual

circuit that gets fabricated on silicon.

Same language, two audiences: a simulator that pretends to be hardware, and a synthesizer that builds it for real.

Verilog, VHDL, SystemVerilog

Three HDLs exist in industry today. Here's why this course uses Verilog.

Verilog (1984)

C-like syntax, compact, the most widely taught HDL for digital design courses. IEEE 1364. What this course uses.

VHDL (1987)

Ada-derived, strongly typed, verbose. Dominant in defense, aerospace, and some European industry.

SystemVerilog (2005)

A superset of Verilog adding verification and object-oriented features. IEEE 1800 — where Verilog leads.

This course uses Verilog: compact syntax, the fastest on-ramp to writing your first working circuit.

A Brief History

Forty years old, still the industry's default teaching language for RTL design.

1984

Gateway Design Automation creates Verilog as a proprietary simulator language.

1989

Cadence acquires Gateway; Verilog spreads across the industry.

1995

IEEE 1364-1995 — Verilog becomes an open standard.

2001

IEEE 1364-2001 — ANSI-style ports, generate blocks, signed arithmetic.

2005

IEEE 1800 — SystemVerilog folds Verilog in as its foundation.

Verilog in the Real Flow

From the RTL you'll write to the silicon that ships.

RTL → Silicon

Every line of Verilog this lecture covers eventually passes through this exact pipeline.

RTL

(your .v files)

→

Logic

Synthesis

→

Gate-Level

Netlist

→

Place &

Route

→

Fabrication

Simulation (this whole lecture, and all of Lab 1) happens entirely at the RTL stage — before anything is synthesized, placed, or fabricated.

■

Tools of the trade

Simulators: Icarus Verilog, Synopsys VCS, Siemens Questa.

Synthesis: Synopsys Design Compiler, Cadence Genus. Waveform viewers:

GTKWave. Lab 1 uses the free, open-source column of this list.

Program Structure

The container every Verilog design lives inside — and two ways to write its header.

module / endmodule & Ports delimited by exactly this pair of keywords.

ANATOMY OF A MODULE

```
input wire a, // a port – a connection point to the
outside world
input wire b,
output wire y
);
assign y = a & b; // the body – what the module does
endmodule
```

module example {
 output logic y;
 // ...
endmodule

endmodule is the implementation.

endmodule is the implementation.

The small syntax rules that make everything else readable.

//

Comments

// runs to end of line. /* ... */ spans multiple lines. Verilog is otherwise

silent about documentation — a comment header naming each module's purpose is a house convention, not a language rule.

■

Whitespace

Free-format — spaces, tabs, and newlines are interchangeable and don't affect meaning. Indentation is purely for humans, same as in C

Aa

Case sensitivity

Verilog is case-sensitive: and_gate and AND_GATE are different identifiers.

Keywords (module, input, assign...) are

always lowercase.

Data Types

Four possible values, two fundamentally different kinds of signal.

Nets vs. Variables

The theory behind Lab 1's #1 rule: wire for assign, reg for always.

Nets — wire

A net represents a physical wire. It has no value of its own — it continuously reflects whatever drives it (a gate, a module port, an assign statement). Declared with wire (or the rarer

wor/wand/tri).

```
wire y; assign y = a & b;
```

Variables — reg, integer, real, time

A variable holds a value between procedural assignments. reg

is the workhorse; integer, real, and time exist for testbench convenience. Despite the name, reg does not always mean a

hardware flip-flop.

```
reg y; always @(*) y = a & b;
```

Vectors & Buses

Bundling bits into a multi-bit signal — and picking individual bits back out.

DECLARING AND INDEXING A VECTOR

```
wire [7:0] data; // an 8-bit vector, bit 7 = MSB, bit 0 = LSB
```

```
data[0] // bit-select — a single bit
data[7:4] // part-select — the top nibble
```

```
wire [3:0] nib = data[3:0]; // assigning a slice to a narrower wire
```

Convention: [MSB:LSB] — this course always declares vectors most-significant-bit first, e.g. [7:0], never [0:7].

Arithmetic & Relational Operators

The same symbols you already know from software — with one hardware caveat.

Arithmetic

+ - * % — addition, subtraction, multiplication, division, modulo. Division and modulo are simulation-friendly but expensive (or unsynthesizable) in real hardware — used sparingly outside testbenches.

```
sum = a + b; rem = a % b;
```

Relational

```
< > <= >= — compare magnitude, always return a single bit: 1 (true), 0 (false), or x if either operand contains an x.
wire gt = (a > b);
```

Logical & Bitwise Operators

Two families that look similar and behave completely differently. Logical & || !

Treat their entire operand as one true/false value (any nonzero = true) and always return a single bit. Used for conditions — if, while.

```
if (a && !b) ...
```

Bitwise & | ^ ~

Operate bit-by-bit across a vector, returning a vector the same width as the operands. This is what and_gate's assign y

```
= a & b; actually uses.
wire [3:0] y = a & b; // per-bit AND
```

The Assign Statement

"Continuous" means exactly that — this line re-evaluates the instant its inputs change.

CONTINUOUS ASSIGNMENT

```
wire y;
assign y = a & b; // re-fires on every change to a or b
```

■

No clock, no trigger, no waiting

This is what makes assign the right tool for combinational logic — a real AND gate has no concept of "running once." It just always reflects

its inputs, and assign models that directly.

Exercise 1 · Build an AND Gate

One module. Two inputs. One output. The simplest possible Verilog.

```
and_gate.v
// 2-input AND gate
```

```
input wire a,
input wire b,
output wire y
);
assign y = a & b;
endmodule
```

NOTES

assign

is continuous — y updates the moment a or b changes.

&

is bitwise AND. For 1-bit signals, that's just AND.

wire

ports default to wire — you can omit it. Shown for clarity.

Exercise 1 · Write the Testbench

Drive every input combination. Watch the output. Record the waveform.

```
and_tb.v
```

```
reg a, b;
wire y;
and_gate dut (.a(a), .b(b), .y(y));
initial begin
$dumpfile("and.vcd");
$dumpvars(0, and_tb);
a=0; b=0; #10;
a=0; b=1; #10;
a=1; b=0; #10;
a=1; b=1; #10;
$finish;
end
endmodule
```

KEY POINTS

Testbench has no ports — it's the top of the simulation.

'reg' for the inputs because they're driven inside a procedural block.

*#10 = wait 10 time units before next change.

\$dumpvars' records every signal under

and_tb into the VCD.
Exercise 2 · Add OR, NOT, XOR
Same pattern as AND. Just three more modules. Three operators to remember.
or_gate.v

```
input a,
input b,
output y
);
assign y = a | b;
endmodule
not_gate.v
```

```
input a,
output y
);
assign y = ~a;
endmodule
xor_gate.v
```

```
input a,
input b,
output y
);
assign y = a ^ b;
endmodule
OPERATORS · & AND | OR ~ NOT ^ XOR
```

Common Errors · And the Fix
Four mistakes that will eat 30 minutes if you don't recognize them.
syntax error near `endmodule`
Missing semicolon at the end of an `assign` line. Verilog won't tell you the right line — look one above.
yourgate.v: undefined identifier `and\_gate`
You forgot to compile and_gate.v alongside the testbench. Pass BOTH files to verilog.
VCD opens but is empty in GTKWave
You forgot `\$dumpfile` and `\$dumpvars` in the testbench. Without them, nothing is recorded.
Output stuck at X (red)
An input was never assigned. In Verilog, unassigned wires default to X (unknown), not 0.
Take These Home
Three short stretch problems. Solutions discussed at the start of Lab 2.

01
NAND and NOR from primitives
Implement a NAND gate using only the AND and NOT modules from this lab. Verify against its truth table.

02
Add an XNOR module
Add `xnor\_gate.v` to your set. Update top.v and the testbench to exercise all five gates together.

END OF LAB 01
Seven gates.
A whole CPU ahead.
Questions?

Thanks
for
watching!
=====

=====

Lecture: 0101

From Combinational to Sequential
How a circuit gains memory — and how a clock keeps that memory sane.

3. Flip-Flops
Sample the data at one precise instant — the clock edge — instead of a whole window. This is the reliable memory a CPU is built from.
L05 · MODERN COMPUTER ARCHITECTURE
Level-Triggered vs Edge-Triggered

The difference between 'listen the whole time' and 'listen for one instant'.

LEVEL-TRIGGERED (latch)
Active during an entire clock LEVEL (high or low).
Transparent — output tracks input the whole window.
Simpler, fewer gates.
Risky: data can slip through unintentionally.
EDGE-TRIGGERED (flip-flop)

Active only at the clock EDGE (rising or falling).
Samples input at one instant, then locks.
More gates, but predictable.
The safe choice for pipelines & registers.
↑ rising edge

An edge is a MOMENT (0→1 transition), not a duration. That's why edge-triggering
samples cleanly.

L05 · MODERN COMPUTER ARCHITECTURE
How Edge-Triggering Works: Master-Slave

Two latches in series on opposite clock phases — data can only advance one step per cycle.
Master captures D while CLK is high; slave passes it on the falling edge. Result: one clean update per cycle.

THE HAND-OFF
The two latches are NEVER transparent
at the same time.

When the master listens, the slave is
locked — and vice versa.

So data can't race straight through. It's a
two-stage airlock for one bit.

L05 · MODERN COMPUTER ARCHITECTURE
Latch vs Flip-Flop

Same job — store a bit — but timing behaviour is completely different.

LATCH
FLIP-FLOP
Triggered by
Clock LEVEL (high/low)
Clock EDGE (rising/falling)

Transparent?
Yes — while enabled
No — samples one instant
Gate count
Fewer (cheaper)
More (~2× a latch)
Data races?
Possible — risky
Blocked — safe
Used for
Simple buffering
Registers, PC, pipelines

Rule of thumb: if a clock edge triggers it, it's a flip-flop. If a level does, it's a latch.

4. Timing
Flip-flops only work if data is stable around the clock edge. Break that rule and the bit becomes undefined.

L05 · MODERN COMPUTER ARCHITECTURE
Setup Time & Hold Time

Data must be STABLE in a window around the clock edge — before and after.

THE ANALOGY
Taking a photo of a moving object.
The subject must hold still just before
AND just after the shutter clicks, or the
picture blurs.

Same for a flip-flop: change D inside the
setup/hold window and the captured bit is a blur — undefined.

L05 · MODERN COMPUTER ARCHITECTURE
Timing Violations

Break setup or hold, and the flip-flop's output is genuinely unpredictable.

SETUP VIOLATION
D changes too LATE — still moving when the edge
arrives.

The new value didn't have time to settle before sampling.
HOLD VIOLATION

D changes too SOON — moves right after the edge.
The old value wasn't held long enough to be captured
cleanly.

THE CONSEQUENCE
The output may settle to 0, settle to 1, or hover in between for a while — a condition called METASTABILITY. This is the deepest hazard in synchronous design, and it's next.

L05 · MODERN COMPUTER ARCHITECTURE
Metastability

A flip-flop caught between 0 and 1 — balanced on a knife's edge.
THE BALL-ON-A-HILL PICTURE

balanced!
0
1
A valid bit is a ball resting in a valley (0 or 1). A timing violation places it on the peak — where it lingers, then falls to a side unpredictably, and after an unknown delay.

WHY IT'S DANGEROUS
The output is neither 0 nor 1 for an unpredictable time.

Downstream logic may read it as 0, another part as 1 —
and the whole system's state diverges.

It's most common when crossing between two unrelated
clocks (asynchronous inputs — buttons, other chips).

L05 · MODERN COMPUTER ARCHITECTURE
Living With Metastability

You can't eliminate it — but you can make it astronomically unlikely.

Can't be banned
For truly asynchronous inputs,
metastability is physically unavoidable — no circuit prevents it 100%.

Synchronizer chains
Feed the signal through 2-3 flip-flops
in series. Each gives the previous one
time to settle before the next samples.

MTBF, not 'never'
Design targets a Mean Time Between
Failures of thousands of years —
practically never, but never truly zero.

The pro's mindset: you don't defeat metastability, you push its probability so low the chip will be obsolete before it ever happens.

L05 · MODERN COMPUTER ARCHITECTURE
Timing in the Real World

Setup, hold, and the clock aren't theory — they set the speed limit of every chip.

Clock speed = the spec
'3.5 GHz' means the clock period is ~0.29 ns. All logic
between two flip-flops must finish inside that window.

→
Critical path sets the max
The slowest logic path between flip-flops decides the
fastest safe clock. Shorten it → clock faster.

Static Timing Analysis (STA)
EDA tools (Synopsys PrimeTime) check every path
for setup/hold violations before a chip is ever built.

Overclocking = risking it
Pushing the clock past its rated speed shrinks timing
margins — eventually setup violations corrupt data.

L05 · MODERN COMPUTER ARCHITECTURE
Reading a D Flip-Flop Waveform

Given CLK and D, trace Q — the classic exam and interview question.

L05 · MODERN COMPUTER ARCHITECTURE
Common Mistakes to Avoid

The slips that cost marks — and cause real timing bugs.

X
Latch vs flip-flop mix-up

A latch is level-triggered (transparent); a flip-flop is edge-triggered. The clock-input triangle marks a flip-flop.

X
Allowing SR = 1,1
The forbidden state breaks Q ≠ Q'. Real designs use a D
latch to make it impossible.

X
Ignoring the transparency window

A D latch copies D the whole time EN is high — not one clean value. Use a flip-flop for pipelines.

X
Forgetting hold time
Setup gets the attention, but violating HOLD (D changes
too soon after the edge) is just as fatal.

X
Thinking metastability is a 'bug'
It's a physical certainty for async inputs — you manage
its probability, you don't fix it.

X
Zero-margin clocking
Running the clock right at the critical-path limit leaves no
safety margin — temperature or voltage drift breaks it.

L05 · MODERN COMPUTER ARCHITECTURE
Key Takeaways

Five sentences — the birth of memory in a machine.

01
Feedback turns a stateless circuit into memory: an output routed back to its input can sustain a value.

02
An SR latch sets, resets, and holds a bit — but S = R = 1 is forbidden; the D latch engineers that danger away.

03
A latch is level-triggered (transparent while enabled); a flip-flop is edge-triggered (samples one instant).

04
The D flip-flop — built as master-slave — captures D on the clock edge and holds it, solving the transparency problem.

05
Data must be stable within the setup/hold window around the edge; violate it and you risk metastability, managed via synchronizer chains.

L05 · MODERN COMPUTER ARCHITECTURE
Practice & Homework

Trace, build, and reason about timing.

01
Latch Truth Tables
Complete the state tables for a NOR
SR latch and a NAND SR latch. Mark
the forbidden state in each.

02
Waveform Trace
Given a CLK and a D waveform, draw
Q for both a D latch AND a D flip-flop.

03
Explain the difference.

Timing Reasoning
A path has 0.28 ns delay; setup is 0.05 ns. What's the fastest safe clock frequency? Show your work.

EXIT TICKET
1. Why is S = R = 1 forbidden in an SR latch?
What does the D latch do about it?

2. In one sentence, the difference between a latch and a flip-flop.

3. What is metastability, and why can't it be fully eliminated for asynchronous inputs?

L05 · MODERN COMPUTER ARCHITECTURE
Rapid-Fire Recap

Three questions — shout the answers.

Q1
A latch is triggered by a clock...
LEVEL

Q2
A flip-flop is triggered by a clock...
EDGE

Q3
Data unstable at the edge risks...
metastability

All three instant? You're ready for the labs and L6 (Registers, Counters & Finite State Machines).

END OF L05
A loop, and
a heartbeat.

NEXT — L06
Registers, Counters & Finite State Machines — chain flip-flops together and a circuit can count, remember words, and follow a plan.

=====

=====

Lecture: 0101

From Combinational to Sequential
How a circuit gains memory — and how a clock keeps that memory sane.

L05 · MODERN COMPUTER ARCHITECTURE
Everything So Far Had No Memory

L1–L4 built powerful circuits — but they forget everything instantly.

&
Gates &
minimisation

AND/OR/NOT, K-maps,
universal NAND (L1–L2).

■

MUX, decoder,
adder
Reusable blocks that build
a datapath (L3).
±
Number
representation
Two's complement,
arithmetic, overflow (L4).
Q
All 'stateless'
Output = f(inputs NOW).
Change inputs → output
forgets the past.
A CPU needs to REMEMBER — the program counter, registers, state. Today: how circuits learn to hold a value.
L05 • MODERN COMPUTER ARCHITECTURE
Today's Roadmap
Feedback creates memory; a clock makes memory trustworthy.
1
Feedback =
Memory
Loop a circuit's output
back to its input, and it can hold a bit.
2
Latches
SR and D latches — the
first storage elements, and their dangers.
3
Flip-Flops
Edge-triggered storage —
the reliable building block of every register.
4
Timing
Setup, hold, and
metastability — the rules that keep it all working.
The big shift: from 'output depends on inputs now' to 'output depends on inputs AND history'. That's the birth of state.
L05 • MODERN COMPUTER ARCHITECTURE
Combinational vs Sequential
The single most important distinction in digital design.
COMBINATIONAL
Output = f(inputs now)
No memory — purely reactive.
Same inputs always give same output.
Examples: adders, MUXes, decoders.
The instant inputs change, output follows.
SEQUENTIAL
Output = f(inputs, past state)
Has memory — remembers history.
Same inputs can give different outputs.
Examples: registers, counters, FSMs.
A clock decides WHEN state updates.
The difference is one thing: a feedback loop. Add it, and a circuit can remember.
1. Feedback = Memory
Route a circuit's output back into its own input, and something remarkable
happens: it remembers.
L05 • MODERN COMPUTER ARCHITECTURE
The Idea of Feedback
Without a loop, signals flow one way and vanish. With a loop, they persist.
NO FEEDBACK (combinational)
LOGIC
in
out
Signal flows left → right, once. Remove the input and the output is gone.
WITH FEEDBACK (sequential)
LOGIC
output feeds back
THE INSIGHT
A feedback loop lets the output 'hold itself up'.
The circuit's current output becomes part of its own
next input — so it can sustain a value even after the original input is gone.
That self-sustaining value IS a stored bit. Memory, from nothing but a loop.
L05 • MODERN COMPUTER ARCHITECTURE
The Bistable Element
Two inverters in a loop = the simplest 1-bit memory. It has two stable states.
If Q = 1, the loop keeps Q = 1. If Q = 0, it keeps Q = 0. Both are stable — hence 'bi-stable'.
TWO STABLE STATES
This holds a bit forever (while powered) — but
there's a catch:
there's no way to CHANGE it. It's sealed.
To make it useful, we need controlled inputs to set and reset the stored value. That's exactly what a
latch adds.
2. Latches
Add controllable inputs to the feedback loop — now you can set, reset, and hold a bit on demand.
L05 • MODERN COMPUTER ARCHITECTURE
The SR Latch (NOR-based)
Two cross-coupled NOR gates: Set makes Q=1, Reset makes Q=0.
HOW IT BEHAVES
S = 1 → Set: Q becomes 1
R = 1 → Reset: Q becomes 0
S = R = 0 → Hold: Q keeps its value
The 'hold' state is the memory — the latch
remembers whatever it was last set to.
Each gate's output feeds the OTHER gate's input — that's the cross-coupling

that holds state.
L05 • MODERN COMPUTER ARCHITECTURE
SR Latch: The Four States
Every combination of S and R, and what the latch does.
S
R
Action
Result
0
0
0
Hold
Q unchanged — remembers last value
1
0
Set
Q = 1
0
1
Reset
Q = 0
1
1
Forbidden
Q and Q' both 0 — invalid!
Three useful states — Set, Reset, Hold — plus one you must never allow. That forbidden state is next.
L05 • MODERN COMPUTER ARCHITECTURE
The Forbidden State
S = R = 1 breaks the latch's fundamental promise: Q and Q' are supposed to be opposites.
WHAT GOES WRONG
With S = R = 1, BOTH NOR gates output 0.
So Q = 0 AND Q' = 0 — but Q' is supposed to be NOT Q. The invariant is broken.
Worse: if S and R drop to 0 at the same instant, the latch enters a RACE — its final state is unpredictable, decided by tiny gate-delay differences.
WHY YOU CARE
This is a real design hazard.
It's why raw SR latches are rarely used directly — designers add logic to make S = R = 1 impossible.
The D latch (two slides away) solves this elegantly: it makes S and R always opposite, so the forbidden state simply can't occur.
L05 • MODERN COMPUTER ARCHITECTURE
SR Latch, NAND Version
Same idea, built from NAND gates — but the inputs are active-LOW.
Note the primes: 'S' and 'R' are active-LOW — a 0 triggers the action, not a 1.
NOR vs NAND LATCH
NOR latch: active-HIGH (1 = act).
NAND latch: active-LOW (0 = act).
Forbidden state flips too: for the NAND latch, S' = R' = 0 is the illegal one.
NAND latches are common in practice because NAND
is the cheap, universal gate (recall L2).
L05 • MODERN COMPUTER ARCHITECTURE
The D Latch
One data input, no forbidden state. The forbidden combination is engineered away.
D drives S directly, and NOT-D drives R — so S and R are ALWAYS opposite.
Forbidden state: impossible.
BEHAVIOUR
EN = 1 → Q follows D. (transparent — Q = D)
EN = 0 → Q holds its last value. (opaque — ignores D)
One clean data input, one enable, zero forbidden states. This is genuinely useful memory.
LAB 05 • MODERN COMPUTER ARCHITECTURE
D Latch • Transparency
While E is high, Q tracks D step-for-step. While E is low, whatever D was at the falling edge of E is what Q holds.
D
E
Q
Q
D
D
E
Q
← transparent →
← holding →
← transparent →
During the green windows, Q is a copy of D. During the amber window, Q remembers the last D from before E fell.
L05 • MODERN COMPUTER ARCHITECTURE
The Clock: The System Heartbeat
One periodic signal synchronises every flip-flop in the chip.
WHY ONE CLOCK
Every flip-flop updates on the same edge.
That shared instant is what keeps a whole chip's state consistent — everyone steps together.
The clock is the drummer the entire orchestra follows. Lose sync, lose everything.
=====

QUIZ 11

Lecture: 05.10
Universal Gates
How engineers take 16 gates and leave only 2 — the art of doing less.
L02 • MODERN COMPUTER ARCHITECTURE
Where We Left Off — Recap of L1
Four gates, a handful of laws. Today we turn them into minimal circuits.

&
AND • OR • NOT • XOR
The four core gates and
their truth tables — the vocabulary of all digital logic.
01
Boolean values 1 / 0
Every signal is a voltage that means TRUE or FALSE
— nothing in between.
=
Identity • Null • Idempotent
A+0=A, A·1=A, A+1=1, A+A=A — the algebra that
simplifies expressions.
!
NAND / NOR
previewed
We hinted these are 'universal'. Today we
prove it and build with them.
Prereq check: if truth tables for AND/OR/NOT/XOR feel automatic, you're ready.
L02 • MODERN COMPUTER ARCHITECTURE
Today's Roadmap
Three stops, one destination: fewer gates, same truth table.
1
Truth Tables → Expressions
SOP & POS: two mirror-image
recipes for turning any truth table into algebra.
→
2
Karnaugh Maps
Minimisation becomes a visual
grouping game — no messy algebra required.
→
3
Universal Gates
DeMorgan's theorems prove one gate — NAND — can build every
circuit that exists.
L02 • MODERN COMPUTER ARCHITECTURE
Why Shrink a Circuit?
Every extra gate is silicon area, power burned, and picoseconds lost.
16 → 2
A typical 4-variable function
can collapse from 16 gates to as few as 2 — same truth table, 8x less silicon.
■
AREA
Fewer gates → smaller die → more chips per wafer →
cheaper.
■
POWER
Less switching → lower dynamic power → longer battery life.
■
SPEED
Shorter gate chains → faster propagation → higher clock.
■
COST
Less silicon + higher yield =
more profit at fab scale.
VLSI Design Flow (Logic Synthesis)
L02 • MODERN COMPUTER ARCHITECTURE
●
This is our Goal to understand !!
●
Apple's M3 packs 98 billion transistors on TSMC's
second-generation 3nm process (N3E) — at that density, a 1% gate-count reduction, multiplied across billions of gates, is real die area and real dollars.
●
This is exactly why chip
companies run logic synthesis tools (Synopsys Design Compiler, Cadence Genus) before fabrication.
L02 • MODERN COMPUTER ARCHITECTURE
From Truth Table to Expression
You can't minimise a table. First turn it into algebra — two canonical ways.
A
B
C
F
0
0
0
0
0
0
0
1
1
0
1
0
0
0
1
1
1
1
0
1
0

0
1
0
1
1
1
1
0
0
1
1
1
1
1

Highlighted rows are where $F = 1$

SOP — Sum of Products

OR together the rows where $F = 1$. Each such row → one **AND** term (a **minterm**).

$F = A'B'C + A'BC + AB'C + ABC$

POS — Product of Sums

AND together the rows where $F = 0$. Each such row → one **OR** term (a **maxterm**).

$F = (A+B+C)(A+B'+C)(A'+B+C)(A'+B'+C)$

Same function, two equivalent descriptions — pick whichever gives fewer terms.

L02 · MODERN COMPUTER ARCHITECTURE

Sum of Products (SOP)

For every row where $F = 1$, write a minterm. Then **OR** them together.

1

Find every row where $F = 1$

These are the only rows that contribute to an SOP expression.

2

Write one minterm per row

A single AND term true for exactly that one input combination.

3

Variable = 1 → X · Variable = 0 → X'

Keep it if it's 1; invert it if it's 0.

4

OR all minterms together

The finished canonical SOP expression.

WORKED EXAMPLE

Row 5: $A=1, B=0, C=1, F=1$

$A = 1$
→ A
 $B = 0$
→ B'
 $C = 1$
→ C
minterm $m_5 = A \cdot B' \cdot C$

L02 · MODERN COMPUTER ARCHITECTURE

Product of Sums (POS)

The mirror image — for every 0-row write a maxterm, then **AND** them.

1

Find every row where $F = 0$

The 0-rows are the ones that build a POS expression.

2

Write one maxterm per row

A single OR term that is 0 for exactly that one combination.

3

Variable = 1 → X' · Variable = 0 → X

Inverted relative to SOP — this is THE trap. Flip it.

4

AND all maxterms together

The finished canonical POS expression.

WORKED EXAMPLE

Row 2: $A=0, B=1, C=0, F=0$

$A = 0$
→ A
 $B = 1$
→ B'
 $C = 0$
→ C
maxterm $M_2 = (A + B' + C)$

L02 · MODERN COMPUTER ARCHITECTURE

Minterm & Maxterm Shorthand

Textbooks and exams write canonical forms in one compact line — read it instantly.

Σ NOTATION

Sigma means **SOP** — **OR** together the listed minterm numbers.

$F = \Sigma m(1, 3, 5, 7)$

Π NOTATION

Pi means **POS** — **AND** together the listed maxterm numbers.

$F = \Pi M(0, 2, 4, 6)$

WHY IT MATTERS

- The number in $\Sigma m(\dots)$ is the decimal value of that row's input bits.
- Row $A=1, B=0, C=1$ → binary 101 → 5, so it appears as m_5 .

Σ and Π over the SAME function are
complementary: every index is in exactly one.

L02 · MODERN COMPUTER ARCHITECTURE

Boolean Algebra: Core Laws

The algebraic toolkit K-maps are about to make visual — no grid required.

LAW FAMILY

KEY IDENTITIES

WHY IT MATTERS

Identity & Null
 $A+0=A \cdot A \cdot 1=A$
 $A+1=1 \cdot A \cdot 0=0$

A single wire or a constant wins — no gate needed.

Idempotent & Complement
 $A+A=A \cdot A \cdot A=A$
 $A+A'=1 \cdot A \cdot A'=0$

No duplicate gates; a signal and its inverse cancel.

Absorption

$A + A \cdot B = A$
 $A \cdot (A + B) = A$
 The bigger term swallows the smaller one whole.
Distributive
 $A(B + C) = AB + AC$
 $A + BC = (A + B)(A + C)$
Yes — OR distributes over AND too, unlike ordinary algebra.
 Every K-map grouping you draw next is one of these laws applied visually instead of algebraically.
Karnaugh Maps
 Minimisation as pattern recognition — group adjacent 1s and watch variables cancel.
L02 · MODERN COMPUTER ARCHITECTURE
Enter Karnaugh Maps a' b' edit
 A truth table re-organised so neighbours differ by exactly ONE bit.
A1B
 0
 1
 0
 1
 0
 1
 0
 1
 0
Group the B=1 column → F = B
A cancels: both A=0 and A=1 give 1.
THE KEY INSIGHT
Each cell = one minterm.
 The grid is just the truth table, rearranged.
 Adjacent 1-cells differ by one variable.
 So grouping them makes that variable cancel.
Grouping = simplification.
Bigger groups → more variables cancel → shorter expression.
 It's pattern recognition, not algebra.
 Your eye does the work the Boolean laws would.
L02 · MODERN COMPUTER ARCHITECTURE
3-Variable K-Map
 8 minterms in Gray-code order — every horizontal step flips exactly one bit.
A1BC
 00
 01
 11
 10
 0
 1
 0
 1
 1
 1
 0
 0
 0
 1
 1
 1
 0
Group of 4 → F = C
 4 minterms collapse into a single variable.
GRAY-CODE ORDERING
Columns run 00 → 01 → 11 → 10, not binary count. This guarantees
 every step flips just one variable, keeping adjacency meaningful.
LEGAL GROUP SIZES
1 cell → full 3-variable minterm
 2 cells
 → **2-variable term**
 4 cells
 → **1-variable term**
 8 cells
 → **constant 1**
L02 · MODERN COMPUTER ARCHITECTURE
4-Variable K-Map
16 cells, both axes Gray-coded. Bigger groups = bigger savings.
AB1CD
 00
 01
 11
 10
 00
 01
 11
 10
 1
 0
 1
 0
 1
 1
 1
 1
 0
 1
 1
 0
 0
 0
 0
 0
 0
 0
GROUP ANATOMY
OCTET
 8 cells
 3 variables cancel
QUAD
 4 cells
 2 variables cancel
PAIR
 2 cells
 1 variable cancels
SINGLE
 1 cell
 nothing cancels
 Each doubling of group size cancels exactly one more variable.
F = A'B + BD + A'D'
L02 · MODERN COMPUTER ARCHITECTURE
The Wrap-Around Trick
 A K-map is a torus (donut) — opposite edges and all four corners are neighbours.
AB1CD
 00

[illegible]

Two rules that let you swap freely between AND-world and OR-world.

RULE 1
~(A · B) = ~A + ~B

NOT (A AND B) = (NOT A) OR (NOT B)
RULE 2
~(A + B) = ~A · ~B

NOT (A OR B) = (NOT A) AND (NOT B)
The one-line memory hook: “Break the bar, flip the sign.”

L02 · MODERN COMPUTER ARCHITECTURE
Bubble Pushing
DeMorgan as a physical motion — slide a bubble through a gate and it ■ips type.

BEFORE
AND
A
B
~(A·B)

AND gate with an output bubble = NAND.
→
AFTER
OR
A
B
~A + ~B

OR gate with input bubbles — same function!
Bubble pushing IS DeMorgan — just drawn instead of written. Same theorem, no new rule.

L02 · MODERN COMPUTER ARCHITECTURE
DeMorgan in Practice
Apply it repeatedly, one bar at a time, from the outside in.

GOAL
Simplify: ~(A · B + C)
STEP 1
Break the outer NOT over the OR: ~(A·B) · ~C
STEP 2
Break the remaining bar on ~(A·B): (~A + ~B) · ~C
DONE
(~A + ~B) · ~C

Scan for EVERY remaining bar after each step — students stop after one and miss the nested NOT.

L02 · MODERN COMPUTER ARCHITECTURE
Quick Check
Show of hands — no wrong answers, only data points.
Q. ~(A + B) is equivalent to...
A
~A + ~B
B
~A · ~B
✓ DeMorgan's Rule 2

C
A · B
D
A + B

NOT of an OR becomes an AND of NOTs — the theorem we prove on the next slide.

L02 · MODERN COMPUTER ARCHITECTURE
NAND Is Everything
One gate type, wired to itself the right way, builds every Boolean function.

ONLY
NAND
is needed to build every gate — and therefore every CPU.

THE PROOF, IN THREE STEPS
NOT
= NAND(A, A)
Tie both inputs together.
AND
= NAND then NOT
Invert the NAND's output.
OR
= NAND of inverted inputs
DeMorgan, realised in silicon.

L02 · MODERN COMPUTER ARCHITECTURE
Build #1 — NOT from NAND
Tie both inputs of a NAND together. Done.

NAND
A
~A
A
~A
0
1
1
0
RESULT
NOT(A) =
NAND(A, A)
With both inputs equal, NAND is 1 when A=0 and 0 when A=1 — precisely the NOT truth table. One gate, zero extra parts.

L02 · MODERN COMPUTER ARCHITECTURE
Build #2 — AND from NAND
A NAND, followed by another NAND wired as a NOT. Two gates.

NAND
A
B
NOT
A·B
~(A·B)
RESULT
AND(A, B) =
NAND(NAND(A,B),
NAND(A,B))
The ■rst NAND gives ~(A·B); the second — wired as a NOT — inverts it back to

A·B. This is why a 'real' AND cell costs
extra transistors.

L02 · MODERN COMPUTER ARCHITECTURE
Build #3 — OR from NAND
Invert each input, then NAND the results. DeMorgan in silicon.

NOT
NOT
A
B
NAND
A+B
~A
~B
RESULT
OR(A, B) =
NAND(NAND(A,A),
NAND(B,B))
NAND(~A, ~B) = ~(~A·~B) = A+B by
DeMorgan. This is Slide 20's Rule 1
turned into a working circuit — algebra becoming hardware.

L02 · MODERN COMPUTER ARCHITECTURE
NOR Is Universal Too
Every NAND trick has a NOR mirror — swap AND ↔ OR and the inversion positions.

BUILD
NAND-ONLY
NOR-ONLY
NOT
NAND(A, A)
NOR(A, A)
AND
NAND(NAND(A,B), NAND(A,B))
NOR(NOR(A,A), NOR(B,B))
OR
NAND(NAND(A,A), NAND(B,B))
NOR(NOR(A,B), NOR(A,B))

Both are universal — but NAND maps more naturally to CMOS transistor layouts. That's why it wins (next slide).
Bonus: NAND ■ash vs NOR ■ash are named after which gate topology their memory cells resemble.

L02 · MODERN COMPUTER ARCHITECTURE
Why Chip Fabs Pick NAND
At the transistor level, a CMOS NAND needs fewer transistors than an AND.
CMOS NAND
4
TRANSISTORS
2 NMOS + 2 PMOS — a direct, natural mapping.
CMOS AND
6
TRANSISTORS
NAND + an inverter (adds 2 more transistors). Multiply by billions of gates per chip. Apple's M4 packs 28 billion transistors on TSMC's 3nm — that 2-transistor delta becomes a city block of silicon.

L02 · MODERN COMPUTER ARCHITECTURE
Common Mistakes to Avoid
The exact slips that cost marks in exams and interviews.

X
Flipping the SOP/POS inversion rule
SOP: 1→X, 0→X'. POS is the opposite. Say it out loud each time.

X
Reading K-map columns as binary
They're Gray code (00,01,11,10). Binary order breaks single-bit adjacency.

X
Grouping diagonally
Diagonal cells differ by two bits — never adjacent. Only rectangles of size 2■.

X
Forgetting wrap-around & corners
Always check edges and all four corners before ■nalising groups.

X
Stopping DeMorgan after one bar
Scan for every remaining NOT bar — nested ones need another pass.

X
Thinking NAND is 'more powerful'
It's self-sufficient, not stronger. AND+OR+NOT together are equally universal.

L02 · MODERN COMPUTER ARCHITECTURE
Key Takeaways
Five sentences. Pin them up — everything else in this course builds from here.

01
Any truth table → canonical SOP (from the 1-rows) or POS (from the 0-rows).
02
K-maps turn minimisation into a visual pattern-■inding game with Gray-code adjacency.

03
Bigger legal groups cancel more variables; the grid wraps like a torus, and don't-cares are free wildcards.

04
DeMorgan: break the bar, ■ip the sign — the bridge between AND-world and OR-world.

05
NAND (or NOR) alone can implement every Boolean function — and fewer CMOS transistors is why fabs choose it.

L02 · MODERN COMPUTER ARCHITECTURE
Practice & Homework
Three activities plus exit problems — pair up on the whiteboard.

01
Truth Table → SOP
3-input table with 1s at rows {1,3,5,7}.
Write the canonical SOP, then
minimise on a K-map.

02
K-Map Race

In pairs, minimise three 4-variable K-maps under a timer. Compare who found the largest groups.

03
NAND-Only Challenge
Implement AND, OR, NOT — then XOR
— using only 2-input NAND gates.
Verify with a truth table.

EXIT TICKET
1. Minimise F(A,B,C,D) = Σm(0,2,5,7,8,10,13,15) using a K-map.
2. Apply DeMorgan to ~(A·B + C).
3. Build a 2-input OR gate using only NAND gates — sketch and verify.

END OF L02
One gate,
every circuit.
NEXT — L03
Combinational Building Blocks: MUX, DEMUX, Decoders & Adders — the actual guts
of a CPU's ALU, built from everything you minimised today.

Thanks
for watching!

=====

DOCUMENT 15

=====

Lab: 0110
From Flip-Flops to Sequencing Circuits

SESSION MAP
LAB 06 · 2
Lab Roadmap
Two theory blocks, following the same order and depth as the Neso Academy Digital Electronics playlist — then six build exercises.

A
Registers & Shift Registers
Intro to registers → data formats → SISO → SIPO → PISO → PIPO → bidirectional → universal shift register.

B
Counters
Asynchronous (ripple) up/down → synchronous design method → 2/3/4-bit up/down → Ring Counter → Johnson Counter .

C
Verilog Exercises
6 build questions: two registers, two shift registers, two synchronous counters — problem + solution each.

SECTION A
Registers & Shift Registers
Storage elements built from flip-flops — how data enters, moves, and leaves.

1
What is a Register?
A group of flip-flops sharing one clock.

2
Data Formats
Serial vs. parallel in, serial vs. parallel out.

3
4 Basic Modes
SISO, SIPO, PISO, PIPO.

4
Advanced Modes
Bidirectional & Universal shift registers.

A1 · INTRODUCTION TO REGISTERS
LAB 06 · 4
What Is a Register?
A register is a group of flip-flops, each storing one bit, all sharing a common clock.

◆
An n-bit register = n flip-flops in parallel, clocked together, so an n-bit
word updates atomically

◆
Registers are the fastest storage a digital system has — no addressing
or bus arbitration needed, just a clock edge

◆
Two control lines are common on a practical register: LOAD (accept new data) and CLEAR (force all bits to 0)

◆
Registers differ from plain flip-flop banks only in how data is organised: as a stored word with defined I/O behaviour

◆
Building block for: CPU register files, pipeline stage latches, the Program Counter, and every shift register on the next slides
Are registers made of flip-flops? Yes — always. Everything from here builds on this one fact.

A2 · DATA FORMATS & CLASSIFICATION
LAB 06 · 5
Data Formats and Classification of Registers
Every register is classified by how data enters and how it is read back out.

◆
Serial format: bits arrive/leave one at a time, on successive clock
edges, over a single wire

◆
Parallel format: all bits arrive/leave together, in one clock edge, over

n wires

Mode

Input

Output

Typical Use

SISO

Serial

Serial

Pure delay line, data buffering

SIPO

Serial

Parallel

Serial-to-parallel conversion
(e.g. UART receive)

PISO

Parallel

Serial

Parallel-to-serial conversion
(e.g. UART transmit)

PIPO

Parallel

Parallel

Ordinary buffer register with
load enable

Bidirectional and Universal registers (slides A7–A8) extend this table with a direction/mode control.

A3 • SHIFT REGISTER — SISO MODE

LAB 06 • 6

SISO: Serial-In, Serial-Out

Each flip-flop's Q feeds the next flip-flop's D. Data enters one bit per clock and exits one bit per clock, n cycles later.

◆

Shift-right convention: bit walks FF0 -> FF1 -> FF2 -> FF3, one hop per rising edge.

FILMSTRIP — shifting in 1,0,1,1 (serial_in shown at left of each frame)

t=0 (reset)

0

0

0

0

t=1: in=1

1

0

0

0

t=2: in=0

0

1

0

0

t=3: in=1

1

0

1

0

t=4: in=1

1

1

0

1

FF3 = serial_out

always @(posedge clk) begin

q <= {q[2:0], serial_in}; // shift right:

q3<-q2<-q1<-q0<-serial_in

end

assign serial_out = q[3]; // read one bit out, one bit at a time

After 4 clocks the input word has fully entered — but you can only read it 1 bit at a time out of serial_out.

A3 • SHIFT REGISTER — SISO MODE

LAB 06 • 6

SISO: Serial-In, Serial-Out

Each flip-flop's Q feeds the next flip-flop's D. Data enters one bit per clock and exits one bit per clock, n cycles later.

A4 • SHIFT REGISTER — SIPO MODE

LAB 06 • 7

SIPO: Serial-In, Parallel-Out

Same shifting chain as SISO — the difference is purely in how you read the output: tap every flip-flop's Q at once.

FILMSTRIP — same 1,0,1,1 input, but every Q is available immediately (parallel taps)

t=0

0

0

0

0

t=1

1

0

0

0

q[3:0]=0001 readable now

t=2

0

1

0

0

q[3:0]=0010

t=3

1

0

1

0

q[3:0]=0101

t=4

1

1

0

1

q[3:0]=1011 (full word)

◆

Used to reassemble a serial stream (UART RX, SPI MISO) back into a parallel word for the rest of the system

◆

No extra hardware vs SISO — it is the SAME circuit; the four Q outputs were always there

◆

This is exactly the sipo_reg circuit you'll build in Exercise Q3

```
input clk, sin,
output reg [3:0] q;
always @(posedge clk)
q <= {q[2:0], sin};
endmodule // q IS the parallel out
A4 • SHIFT REGISTER — SIPO MODE
LAB 06 • 7
```

SIPO: Serial-In, Parallel-Out

Same shifting chain as SISO — the difference is purely in how you read the output: tap every flip-flop's Q at once.

A5 • SHIFT REGISTER — PISO MODE

LAB 06 • 8

PISO: Parallel-In, Serial-Out

Load an entire word in one clock edge, then shift it out one bit per clock — needs a 2:1 mux in front of every D input.

FILMSTRIP — load 1011 in one edge, then shift out right-to-left

LOAD=1: t0

1

0

1

1

parallel word captured

LOAD=0: t1

0

1

0

1

```
sout=1 (old q0)
LOAD=0: t2
0
0
0
1
sout=1
LOAD=0: t3
0
0
0
0
1
sout=0
LOAD=0: t4
0
0
0
0
0
sout=1 -> word out
```

◆

Each flip-flop's D input is a MUX: LOAD=1 selects the parallel data bit, LOAD=0 selects the neighbour's Q (shift)

◆

This is the mirror image of PIPO's load-enable idea, combined with SISO's shifting idea

◆

Classic use: convert a wide parallel bus into a narrow serial line (e.g. driving an SPI shift-out chip)

```
always @(posedge clk) begin
if (load) q <= d; // parallel load
else q <= {1'b0, q[3:1]}; // shift right
end
assign sout = q[0];
A5 • SHIFT REGISTER — PISO MODE
LAB 06 • 8
PISO: Parallel-In, Serial-Out
```

Load an entire word in one clock edge, then shift it out one bit per clock — needs a 2:1 mux in front of every D input.

A6 • SHIFT REGISTER — PIPO MODE

LAB 06 • 9

PIPO: Parallel-In, Parallel-Out

No shifting at all — every bit loads and reads together. This is simply a load-enable register (Slide A1) wearing the 'shift register' family name.

◆

On a clock edge with LOAD=1, all n bits update simultaneously from d[n-1:0]

◆

With LOAD=0, the register holds — no shifting, no serial ports at all

◆

It's grouped with the shift-register family because it shares the same D-flip-flop-row hardware, just without the neighbour-to-neighbour wiring

◆

This is the exact behaviour tested in Exercise Q1 and Q2 (8-bit register, with and without synchronous reset)

```
always @(posedge clk)
if (load)
q <= d;
// else: hold
Compare to A3 (SISO): same register skeleton, different D-input wiring — neighbour's Q vs. an external bus.
A6 • SHIFT REGISTER — PIPO MODE
LAB 06 • 9
```

PIPO: Parallel-In, Parallel-Out

No shifting at all — every bit loads and reads together. This is simply a load-enable register (Slide A1) wearing the 'shift register' family name.

◆

On a clock edge with LOAD=1, all n bits update simultaneously from d[n-1:0]

◆

With LOAD=0, the register holds — no shifting, no serial ports at all

◆

It's grouped with the shift-register family because it shares the same D-flip-flop-row hardware, just without the neighbour-to-neighbour wiring

◆

This is the exact behaviour tested in Exercise Q1 and Q2 (8-bit register, with and without synchronous reset)

```
always @(posedge clk)
if (load)
q <= d;
// else: hold
Compare to A3 (SISO): same register skeleton, different D-input wiring — neighbour's Q vs. an external bus.
A6 • SHIFT REGISTER — PIPO MODE
LAB 06 • 9
```

PIPO: Parallel-In, Parallel-Out

No shifting at all — every bit loads and reads together. This is simply a load-enable register (Slide A1) wearing the 'shift register' family name.

SECTION B

Counters

A counter is a register that always overwrites itself with the next count value — no external data input needed.

- 1 **Asynchronous**
- Ripple counters — up, down,**
up/down.
- 2 **Synchronous**
- Excitation-table design method;
2/3/4-bit up/down.
- 3
- Ring Counter**
- One '1' circulates through n flip-flops.

- 4
- Johnson Counter**
- Complemented feedback — 2n
- unique states.
- B1 • ASYNCHRONOUS COUNTERS**
- LAB 06 • 13**
- Asynchronous (Ripple) Up Counter — 2-Bit**
- Each flip-flop toggles (T=1) using the PREVIOUS flip-flop's Q as its own clock — the count ripples through, stage by stage.
- ◆
- Only FF0 sees the real clock; FF1 is clocked by FF0's Q, FF2 by FF1's Q, and so on
- ◆
- Every stage is wired T=1, so it simply toggles every time it is clocked
- ◆
- This reproduces the same divide-by-2 chain from Lecture 6 — reading it as bits IS counting
- ◆
- Simple to build (no extra logic gates) but has ripple delay — real count is only valid after all stages settle
- FILMSTRIP — 2-bit ripple up counter (Q1 Q0), 4 clock edges**
- start
- 0
- 0
- clk 1
- 0
- 1
- clk 2
- 1
- 0
- 0
- clk 3
- 1
- 1
- Q0 toggles every clock; Q1 toggles only when Q0 falls 1->0 — exactly like a divide-by-2 chain.
- B2 • ASYNCHRONOUS COUNTERS (CONT.)**
- LAB 06 • 14**
- 3-Bit & 4-Bit Ripple Counters, and Their Speed Limit**
- The 2-bit pattern extends directly: add one more T flip-flop, clocked by the previous stage's Q, per extra bit.
- ◆
- 3-bit: FF0 -> FF1 -> FF2, counts 000 through 111, then wraps
- ◆
- 4-bit: one more stage, counts 0000 through 1111 (MOD-16)
- ◆
- In general, n ripple stages count MOD-2^n and divide the input clock by 2^n
- After edge
- Q3 Q2 Q1 Q0**
- Value**
- before
- 0 1 1 1
- 7 (correct)
- +1 tpd
- 0 1 1 0
- 6 (wrong, transient)
- +2 tpd
- 0 1 0 0
- 4 (wrong, transient)
- +4 tpd
- 1 0 0 0
- 8 (correct at last)
- THE SPEED LIMIT**
- t_settle = n × t_pd**
- f_max = 1 / (n × t_pd)**
- Any circuit reading the count while it ripples — a comparator, a memory
- address bus — briefly sees the wrong value. At t_pd = 10 ns, a 4-bit ripple**
- counter tops out near 25 MHz; a modern CPU clock is ~3 GHz. This is
- exactly why synchronous counters (next slides) exist.
- B2 • ASYNCHRONOUS COUNTERS (CONT.)**
- LAB 06 • 14**
- 3-Bit & 4-Bit Ripple Counters, and Their Speed Limit**
- The 2-bit pattern extends directly: add one more T flip-flop, clocked by the previous stage's Q, per extra bit.
- B4 • DESIGN OF SYNCHRONOUS COUNTERS**
- LAB 06 • 16**
- How to Design a Synchronous Counter**
- Every flip-flop shares the real clock; combinational logic in front decides each flip-flop's next input.
- 1
- Write the state (count) sequence you want, e.g. 00 -> 01 -> 10 -> 11 -> 00
- 2
- Build a present-state / next-state table from that sequence**
- 3
- For each flip-flop, use its excitation table (Lecture 6) to find what input makes that Q -> Q' transition
- 4
- K-map each flip-flop's input as a function of the present-state bits**
- 5
- Wire the resulting gates to each flip-flop's input; all flip-flops still share one clock
- This is the same 6-step FSM design flow from Lecture 6 — a counter is just an FSM whose states happen to be binary numbers.
- B5 • 2-BIT SYNCHRONOUS UP COUNTER**
- LAB 06 • 17**
- Worked Example: 2-Bit Synchronous Up Counter**
- Q1 Q0**
- Next Q1 Q0**
- T1**
- T0**
- 0 0
- 0 1
- 0
- 1

```

01
10
1
1
10
11
0
1
11
00
1

```

♦ **T0 = 1** always -> Q0 toggles every clock (same as the ripple LSB)

♦ **T1 = Q0** -> Q1 only toggles when Q0 was 1 the previous edge

♦ Both flip-flops now share the SAME real clock — no ripple delay

```

input clk, rst,
output reg [1:0] q
);
wire T0 = 1'b1;
wire T1 = q[0];
always @(posedge clk) begin
  if (rst) q <= 2'b00;
  else begin
    q[0] <= q[0] ^ T0;
    q[1] <= q[1] ^ T1;
  end
end
endmodule

```

B6 • 3-BIT SYNCHRONOUS UP COUNTER

LAB 06 • 18

3-Bit Synchronous Up Counter — the General Rule

Extending the 2-bit result reveals the pattern used for any width. **THE RULE**

T0 = 1

T1 = Q0

T2 = Q0 • Q1

A bit toggles exactly when every bit below it is 1 — identical to the ripple-counter rule from Lecture 6, just computed combinatorially instead of chained through real hardware clocking.

```

input clk, rst,
output reg [2:0] q
);
always @(posedge clk) begin
  if (rst) q <= 3'b000;
  else q <= q + 3'b001;
  // synthesiser derives
  // T0/T1/T2 automatically
  // from the '+1' -- this
  // is why real designs
  // write q+1, not manual
  // T-equations.
end
endmodule

```

B7 • SYNCHRONOUS UP/DOWN COUNTER

LAB 06 • 19

3-Bit & 4-Bit Synchronous Up/Down Counter

Add one control input; the next-state adder now adds +1 or -1 depending on it — everything else is unchanged.

♦ **up_down = 1** -> q <= q + 1 (counts up, wraps MAX -> 0)

♦ **up_down = 0** -> q <= q - 1 (counts down, wraps 0 -> MAX)

♦ Still fully synchronous: one clock, one clean transition per edge

♦ This is exactly the difference between Exercise Q5 (up) and Q6 (down)

later in this deck

```

input clk, rst, up_down,
output reg [3:0] q
);
always @(posedge clk) begin
  if (rst) q <= 4'b0000;
  else if (up_down)
    q <= q + 4'b0001;
  else
    q <= q - 4'b0001;
end
endmodule

```

B8 • RING COUNTER

LAB 06 • 20

Ring Counter

A shift register (SISO-style) with its output fed back to its own input — a single 1 circulates forever.

♦ **n flip-flops, connected exactly like a shift register, but Q(last) feeds back to D(first)**

♦ Preset exactly one flip-flop to 1 and clear the rest — that single 1 then walks around the ring, one position per clock

♦ n flip-flops give exactly n unique output states (one-hot) — no decoding logic needed to know which state you're in

♦ Used for simple round-robin sequencing: LED chasers, one-hot state

machines, simple timing generators

FILMSTRIP — 4-bit ring counter, one 1 circulating

```

t0
1
0
0
0
0
t1
0
1
0
0
0

```

```

t2
0
0
1
0
t3
0
1
0
0
1

```

```

always @(posedge clk)
  if (rst) q <= 4'b0001; // preset one 1
  else q <= {q[2:0], q[3]}; // feedback

```

B8 • RING COUNTER

LAB 06 • 20

Ring Counter

A shift register (SISO-style) with its output fed back to its own input — a single 1 circulates forever.

B10 • COUNTER FAMILY COMPARISON

LAB 06 • 22

Ripple vs Synchronous vs Ring vs Johnson

Counter

Clocking

States (n stages)

Extra Logic

Best For

Ripple (async)

Chained

2^n

None

Cheap, low-speed

dividers

Synchronous

Shared

2^n

Next-state gates/adder

Any speed-critical count

(PC, timers)

Ring

Shared

n

None (just feedback)

One-hot sequencing, LED

chasers

Johnson

Shared

2n

None (complemented

feedback)

More states than Ring,

still glitch-free

All four are the same skeleton — a row of flip-flops with a shared clock — differing only in what feeds each D input.

SECTION C

Verilog Build Exercises

Six designs, drawn directly from the register / shift-register / counter theory just covered.

1-2

Registers

Load-enable, then add synchronous reset.

3-4

Shift Registers

SIPO, then PISO.

5-6

Counters

4-bit synchronous up, then down.

EXERCISE 1 OF 6

LAB 06 • 24

Q1

8-bit Register with Load Enable

PROBLEM

♦ Write reg8_load: inputs clk, load, d[7:0]; output q[7:0].

♦

♦ On a rising edge: if load=1, capture d into q.

♦

♦ If load=0, q holds its previous value (not X, not 0).

♦

♦ No reset pin required for this exercise.

SOLUTION

```

input clk,
input ld,
input [7:0] d,
output reg [7:0] q
);
always @(posedge clk) begin
  if (ld)
    q <= d;
end
endmodule

```

EXERCISE 2 OF 6

LAB 06 • 25

Q2

8-bit Register with Synchronous Reset

PROBLEM

♦ Extend Q1: add a synchronous rst input.

♦

♦ On a rising edge, if rst=1, q clears to 8'b0 — regardless of load.

♦

♦ Priority order: rst beats load beats hold.

♦

♦ rst only takes effect on a clock edge (NOT asynchronous).

SOLUTION

```

input clk,
input rst,
input ld,
input [7:0] d,
output reg [7:0] q

```

```

);
always @(posedge clk) begin
  if (rst)
    q <= 8'b00000000;
  else if (ld)
    q <= d;
end
endmodule

```

EXERCISE 3 OF 6

LAB 06 • 26

Q3

SIPO Shift Register

PROBLEM

♦ Write sipo_reg: a 4-bit Serial-In, Parallel-Out shift

register.

♦

♦ Inputs: clk, sin (1 bit). Output: q[3:0] (parallel).

♦

♦ Each rising edge: shift q left by one, load sin into bit 0.

♦

♦ After 4 clocks, q[3:0] holds the last 4 bits received.

SOLUTION

```

input clk,
input rst,
input d_in,
output reg [3:0] q
);
always @(posedge clk) begin
  if (rst)
    q <= 4'b0000;
  else begin
    q[3] <= q[2];
    q[2] <= q[1];
    q[1] <= q[0];
    q[0] <= d_in;
  end
end
endmodule

```

EXERCISE 4 OF 6

LAB 06 • 27

Q4

PISO Shift Register

PROBLEM

♦ Write piso_reg: a 4-bit Parallel-In, Serial-Out shift

register.

♦

♦ Inputs: clk, load, d[3:0]. Output: sout (1 bit).

♦

♦ If load=1 on a clock edge, capture d (parallel load).

♦

♦ If load=0, shift right each edge; output the LSB on sout.

SOLUTION

```

input clk,
input rst,
input sh_ld,
input [3:0] d,
output reg [3:0] q,
output d_out
);
assign d_out = q[0];
always @(posedge clk) begin
  if (rst)
    q <= 4'b0000;
  else if (!sh_ld)
    q <= d;
  else begin
    q[0] <= q[1];
    q[1] <= q[2];
    q[2] <= q[3];
    q[3] <= 1'b0;
  end
end
endmodule

```

EXERCISE 5 OF 6

LAB 06 • 28

Q5

4-bit Synchronous Up Counter

PROBLEM

♦ Write up_counter4: a 4-bit synchronous up counter.

♦

♦ Inputs: clk, rst, en. Output: q[3:0].

♦

♦ On a rising edge: if rst=1, q clears to 0000.

♦

♦ Else if en=1, q increments by 1; else, q holds.

♦

♦ Must wrap 1111 -> 0000 with no special-case code.

SOLUTION

```

input clk, rst, en,
output reg [3:0] q
);
always @(posedge clk) begin
  if (rst)
    q <= 4'b0000;
  else if (en)
    q <= q + 4'b0001; // wraps
  // for free:
  // 1111+1
  // =0000
end
endmodule

```

EXERCISE 6 OF 6

LAB 06 • 29

Q6

4-bit Synchronous Down Counter

PROBLEM

♦

♦ Write down_counter4: a 4-bit synchronous down

counter.

♦

♦ Inputs: clk, rst, en. Output: q[3:0].

♦

♦ On a rising edge: if rst=1, q loads to 1111 (its

MAX, not

0).

♦

three wrong numbers appear on the pins.
TIME AFTER EDGE
Q3
Q2
Q1
Q0
VALUE ON PINS
before
0
1
1
1
1
7 — correct
1 × t_{pd}
0
1
1
0
0
6 — wrong
2 × t_{pd}
0
1
0
0
0
4 — wrong
3 × t_{pd}
0
0
0
0
0
0 — wrong
4 × t_{pd}
1
0
0
0
0
8 — correct at last
Anything reading those pins during the ripple — a comparator, a decoder, a memory address bus — sees 6, then 4, then 0.
THE SPEED LIMIT
The count is only valid after ALL n stages have settled:
t_{settle} = n × t_{pd}
t_{max} = 1 / (n × t_{pd})
PUT NUMBERS ON IT
At t_{pd} = 10 ns, a 4-bit ripple counter tops out near 25 MHz — and a 16-bit one near 6 MHz. A modern CPU clock is 3 GHz.
THE WAY OUT
Give every flip-flop the same clock, and compute in advance which ones should flip.
L06 • MODERN COMPUTER ARCHITECTURE
Synchronous Counter
All flip-flops share ONE clock — they all update at the exact same instant.
How It Works
Combinational logic (the & gates) computes each FF's next bit; the shared clock updates them all together.
Key Advantage
✓ No ripple delay — the whole count is correct the instant after the edge. This is what real designs use.
L06 • MODERN COMPUTER ARCHITECTURE
Ripple vs Synchronous
Same count sequence, very different timing behaviour.
RIPPLE (async)
SYNCHRONOUS
Clocking
Each FF clocks the next
All FFs share one clock
Speed
Slow — carry ripples through
Fast — all update at once
Glitches
Yes — transient wrong counts
No — clean output
Complexity
Simple, fewer gates
Needs next-state logic
Real use
Toy / low-speed dividers
Everything performance-critical
Trade-off in one line: ripple is cheaper to build; synchronous is the only one safe at real clock speeds.
L06 • MODERN COMPUTER ARCHITECTURE
Mod-N & Divide-by Counters
Reset early to count any range — or use a counter to slow a clock down.
MOD-N: STOP EARLY
A 3-bit counter naturally counts mod-8 (0–7). To count mod-6 (0–5), detect the value 6 and reset to 0.
0
→
1
→
2
→
3
→
4
→
5
reset at 6
Used for: clock dividers, digital clocks (mod-60 for seconds!), any custom cycle length.
DIVIDE-BY: SLOW THE CLOCK
Each flip-flop in a counter toggles at HALF the rate of the one before it.
So bit 0 = clock ÷ 2, bit 1 = clock ÷ 4, bit 2 = clock ÷ 8...
A counter is a frequency divider for free. This is how a 32 kHz crystal becomes a 1 Hz tick in a watch.

L06 • MODERN COMPUTER ARCHITECTURE
Counters in the Wild
Once you see them, they're everywhere.
PC
Program Counter
Holds the address of the next instruction; increments each cycle, or jumps on a branch. The heart of instruction flow.
Timers & delays
Count clock ticks to measure time — every microcontroller timer, watchdog, and PWM generator is a counter.
Frequency division
Turn a fast crystal oscillator into slower usable clocks — watches, UART baud rates, display refresh.
Memory addressing
Sequentially step through addresses in DMA transfers, refresh cycles, and scan-out logic.
3. Finite State Machines
The universal model of sequential logic: a set of states, rules to move between them, and outputs. This is how a CPU 'thinks'.
L06 • MODERN COMPUTER ARCHITECTURE
What Is a Finite State Machine?
A circuit that is always in exactly ONE of a finite set of states, and moves between them on rules.
THREE INGREDIENTS
1
States
A finite list of situations the machine can be in (e.g. IDLE, RUNNING, DONE).
2
Transitions
Rules for moving between states, based on current state + inputs.
3
Outputs
What the machine produces in each state (or during each transition).
EVERYDAY EXAMPLE
A traffic light is an FSM.
States: GREEN, YELLOW, RED.
Transitions: timed (green→yellow→red→green).
Outputs: which lamp is lit.
A vending machine, an elevator, a game character's AI, a network protocol — all FSMs. This model is everywhere.
L06 • MODERN COMPUTER ARCHITECTURE
Two Flavours: Moore vs Mealy
The difference is simply WHERE the output comes from.
Rule of thumb: Moore = 'output in the circle', Mealy = 'output on the arrow'. Both compute the same things.
MOORE MACHINE
MEALY MACHINE
Output depends on the STATE only
The output is written inside each state circle
It changes only when the state changes
Simpler to reason about, and glitch-free
May need more states to do the same job
Output appears one cycle after the input
Output depends on STATE + INPUT
The output is written on each transition arrow
It can react to an input within the same cycle
Often needs fewer states
Output can glitch when the input glitches
Output appears in the same cycle as the input
L06 • MODERN COMPUTER ARCHITECTURE
The Anatomy of Any FSM
Every sequential circuit is this same three-part structure.
L06 • MODERN COMPUTER ARCHITECTURE
The State Diagram
The picture that captures an FSM: circles are states, arrows are transitions.
HOW TO READ IT
Each circle = one state.
Each arrow = a transition, labelled with the input/condition that triggers it.
Follow the arrows and you can trace exactly what the machine does for any input sequence.
This diagram IS the design — everything else is just translating it to gates.
L06 • MODERN COMPUTER ARCHITECTURE
The State Diagram
The picture that captures an FSM: circles are states, arrows are transitions.
HOW TO READ IT
Each circle = one state.
Each arrow = a transition, labelled with the input/condition that triggers it.
Follow the arrows and you can trace exactly what the machine does for any input sequence.
This diagram IS the design — everything else is just translating it to gates.
GREEN
YELLOW
RED
timer
timer
timer
TRAFFIC LIGHT — A THREE-STATE MOORE MACHINE
L06 • MODERN COMPUTER ARCHITECTURE
The State Table
The same FSM, written as a table — the bridge from diagram to hardware.
Current State
Input
Next State
Output

GREEN
expired
YELLOW
Go
YELLOW
expired
RED
Slow
RED
expired
GREEN
Stop
any
not expired (same)
(hold)
Why a table? It lists every (state, input) → (next state, output). Encode the states as bits, and each column becomes a truth table you minimise with K-maps (L2) into gates.
L06 • MODERN COMPUTER ARCHITECTURE
FSM Design Flow
Six steps from a word problem to a working circuit — always the same recipe.
1
Understand
List the states and what triggers each transition.
2
State diagram
Draw circles and arrows
— the visual spec.
3
State table
Tabulate every
state/input → next-state/output.
4
Encode states
Assign a bit pattern to each state (00, 01, 10...).
5
Derive logic
K-map each next-state & output bit into gates.
6
Build
Flip-flops for the state
register + that logic.
L06 • MODERN COMPUTER ARCHITECTURE
FSMs Run the World
Once you know the pattern, you'll see state machines in everything you build.
The CPU control unit
Fetch → Decode → Execute → Writeback is literally an FSM. It sequences every instruction (Module 4).
Communication protocols
TCP connection states, USB handshakes, traffic-light controllers — all specified as FSMs.
Software & games
UI flows, game AI ('patrol → chase → attack'), regex engines, parsers — FSMs in code, not gates.
Embedded control
Washing machines, elevators, vending machines, engine controllers — physical FSMs everywhere.
L06 • MODERN COMPUTER ARCHITECTURE
The Big Picture: Datapath + Control
Everything in this course so far now snaps together into a computer. The control unit is an FSM (today). The datapath is registers + ALU + MUXes (L3–L6). Together they're a CPU. This is exactly where Module 3 & 4 pick up — you now have every building block a processor needs.
CONTROL (FSM)
today
DATAPATH
registers · ALU · MUXes · adders
L3 – L6
control signals
status flags
L06 • MODERN COMPUTER ARCHITECTURE
Common Mistakes to Avoid
The slips that cost marks — and cause real design bugs.
X
Register without load enable
A plain register overwrites every clock. Real ones need a load/hold control, or they can't keep a value.
X
Ripple counter at high speed
The carry ripple makes intermediate counts briefly wrong. Use synchronous counters for anything fast.
X
Wrong select/state-bit count
N states need log₂N flip-flops (binary). Miscounting breaks the whole encoding.
X
Moore/Mealy confusion
Moore output = state only (in the circle). Mealy = state + input (on the arrow). Know which you're drawing.
X
Forgetting the reset state
Every FSM needs a defined power-on state, or it starts in a random, possibly illegal state.
X
Unreachable / dead states
After encoding, unused bit patterns can trap the FSM. Handle them (usually route to reset).
L06 • MODERN COMPUTER ARCHITECTURE
Key Takeaways

Five sentences — from a stored bit to a thinking machine.

01
A register is **n** **lip**-**ops** sharing a clock — the CPU's fastest storage; add a load-enable MUX so it holds until told to update.

02
A shift register moves bits along a chain each clock — enabling serial↔parallel conversion and free multiply/divide-by-2.

03
A counter advances by 1 each clock; synchronous (shared clock) beats ripple (chained) for speed and glitch-free counts.

04
A **nite** state machine — states, transitions, outputs — is the universal model; registers and counters are special cases of it.

05
FSM design is a mixed recipe: diagram → table → encode states → K-map the logic → build with a state register + gates.

L06 • MODERN COMPUTER ARCHITECTURE

Practice & Homework

Design, trace, and build state machines.

01
Shift & Count
Trace a 4-bit shift register loading 1011 over 4 clocks. Then draw a 3-bit synchronous counter's Q outputs.

02
FSM from Words
Design a '110' sequence detector:
state diagram, state table, and binary encoding.

03
Divide the Clock
How many lip-ops divide a 1 MHz clock down to 1 kHz? Show the divide-by math.

EXIT TICKET

1. How many lip-ops does a mod-12 counter need, and why?

2. In one line: the difference between a Moore and a Mealy machine.

3. Draw the state diagram for a circuit that detects the input sequence '11'.

L06 • MODERN COMPUTER ARCHITECTURE

Rapid-Fire Recap

Three questions — shout the answers.

Q1
A register is built from...
flip-flops

Q2
Synchronous counters share one...
clock

Q3
A Moore output depends only on...
the state

All three instant? You've completed the digital-logic foundation — Module 3 builds the datapath on top of it.

END OF L06 • MODULE 2 COMPLETE

States, and a machine that runs.

NEXT — MODULE 3

The Datapath & Instruction Set — assemble registers, ALU, and control into a processor that actually executes machine instructions.

Thanks

for watching!

=====

=====

DOCUMENT 10

=====

=====

Lecture: \$111

State machines

How a CPU reads two operands and writes one result every cycle — and how FSMs control it all.

The Storage at the Heart of a CPU

1. Finite State Machines

The universal model of sequential logic: a set of states, rules to move between them, and outputs. This is how a CPU 'thinks'.

L06 • MODERN COMPUTER ARCHITECTURE

What Is a Finite State Machine?

A circuit that is always in exactly ONE of a nite set of states, and moves between them on rules.

THREE INGREDIENTS

1
States
A nite list of situations the machine can be in (e.g. IDLE, RUNNING, DONE).

2
Transitions
Rules for moving between states, based on current state + inputs.

3
Outputs
What the machine produces in each state (or during each transition).

EVERYDAY EXAMPLE

A traffic light is an FSM.
States: GREEN, YELLOW, RED.
Transitions: timed (green→yellow→red→green).
Outputs: which lamp is lit.

A vending machine, an elevator, a game character's AI, a network protocol — all FSMs. This model is everywhere.

L06 • MODERN COMPUTER ARCHITECTURE

Two Flavours: Moore vs Mealy

The difference is simply WHERE the output comes from.

Rule of thumb: Moore = 'output in the circle', Mealy = 'output on the arrow'. Both compute the same things.

MOORE MACHINE

MEALY MACHINE

Output depends on the STATE only

The output is written inside each state circle

It changes only when the state changes

Simpler to reason about, and glitch-free

May need more states to do the same job

Output appears one cycle after the input

Output depends on STATE + INPUT

The output is written on each transition arrow

It can react to an input within the same cycle

Often needs fewer states

Output can glitch when the input glitches

Output appears in the same cycle as the input

L06 • MODERN COMPUTER ARCHITECTURE

The Anatomy of Any FSM

Every sequential circuit is this same three-part structure.

L06 • MODERN COMPUTER ARCHITECTURE

The State Diagram

The picture that captures an FSM: circles are states, arrows are transitions.

HOW TO READ IT

Each circle = one state.

Each arrow = a transition, labelled with the input/condition that triggers it.

Follow the arrows and you can trace exactly what the machine does for any input sequence.

This diagram IS the design — everything else is just translating it to gates.

L06 • MODERN COMPUTER ARCHITECTURE

The State Diagram

The picture that captures an FSM: circles are states, arrows are transitions.

HOW TO READ IT

Each circle = one state.

Each arrow = a transition, labelled with the input/condition that triggers it.

Follow the arrows and you can trace exactly what the machine does for any input sequence.

This diagram IS the design — everything else is just translating it to gates.

GREEN

YELLOW

RED

timer

timer

timer

TRAFFIC LIGHT — A THREE-STATE MOORE MACHINE

L06 • MODERN COMPUTER ARCHITECTURE

The State Table

The same FSM, written as a table — the bridge from diagram to hardware.

Current State

Input

Next State

Output

GREEN

expired

YELLOW

Go

YELLOW

expired

RED

Slow

RED

expired

GREEN

Stop

any

not expired

(Same)

(hold)

Why a table? It lists every (state, input) → (next state, output). Encode the states as bits, and each column becomes a truth table you minimise with K-maps (L2) into gates.

L06 • MODERN COMPUTER ARCHITECTURE

FSM Design Flow

Six steps from a word problem to a working circuit — always the same recipe.

1
Understand
List the states and what triggers each transition.

2
State diagram
Draw circles and arrows
— the visual spec.

3
State table
Tabulate every state/input → next-state/output.

4
Encode states
Assign a bit pattern to each state (00, 01, 10...).

5
Derive logic
K-map each next-state & output bit into gates.

6
Build
Flip-ops for the state register + that logic.

L06 • MODERN COMPUTER ARCHITECTURE

FSMs Run the World

Once you know the pattern, you'll see state machines in everything you build.

■

The CPU control unit

Fetch → Decode → Execute → Writeback is literally

an FSM. It sequences every instruction (Module 4).

■

Communication protocols

TCP connection states, USB handshakes, traffic-light controllers — all specified as FSMs.

◆

Software & games

UI flows, game AI ('patrol → chase → attack'), regex engines, parsers — FSMs in code, not gates.

■

Embedded control

Washing machines, elevators, vending machines, engine controllers — physical FSMs everywhere.

L06 • MODERN COMPUTER ARCHITECTURE

The Big Picture: Datapath + Control

Everything in this course so far now snaps together into a computer. The control unit is an FSM (today). The datapath is registers + ALU + MUXes (L3–L6). Together they're a CPU.

This is exactly where Module 3 & 4 pick up — you now have every building block a processor needs.

CONTROL (FSM)

today

DATAPATH

registers • ALU • MUXes • adders

L3 – L6

control signals

status flags

L07 • MODERN COMPUTER ARCHITECTURE

Where We Left Off

L5–L6 gave us lip-ops, registers, and counters. Now we organise them.

■

Flip-flops

Edge-triggered 1-bit

memory, safe under a clock (L5).

■

Registers

n lip-ops sharing a clock

= an n-bit word store (L6).

■

Counters & FSMs

Structures that count and follow state-based rules (L6).

»

Today: organise them

Pack 32 registers into a file; formalise FSM control.

A CPU has dozens of registers accessed constantly. How do you read two and write one, all in one cycle?

L07 • MODERN COMPUTER ARCHITECTURE

Today's Roadmap

The CPU's fastest storage, and the machines that control it.

1
Register Files
32 registers as one block: 1 write port, 2 read ports — the RISC standard.

→

2
Finite State Machines
The formal model of controlled sequential behaviour.

→

3
Moore vs Mealy
Two ways to build an FSM — and a '1011' detector in both.
Why it matters: the register file is the structure a CPU touches on nearly every instruction — and an FSM is what tells it when.

L07 • MODERN COMPUTER ARCHITECTURE

From Registers to a Register File

A CPU needs many registers, accessed together — so we pack them into one addressable block.

THE NEED

A single instruction like add R1, R2, R3 must:

- **READ** two source registers (R2, R3)
- **WRITE** one destination register (R1)
- ...all in ONE clock cycle.

A register file is a bank of registers with address-based access — like a tiny, ultra-fast memory that lives inside the CPU.

THE STANDARD

MIPS & RISC-V: 32 registers, each 32 bits wide.

Ports: 2 read + 1 write, every cycle. That's exactly enough for a two-operand, one-result instruction.

This exact structure feeds the ALU you'll wire up in Module 4 (L13–L14).

2. Register Files

A bank of registers with addressed access: a decoder picks where to write, MUXes pick what to read.

L07 • MODERN COMPUTER ARCHITECTURE

Anatomy of a Register File

Three ports in, addresses select which registers each port touches.

THREE PORTS

2 READ ports (blue):
each takes an address, outputs that register's value.

1 WRITE port (amber):
address + data + write-enable, updates one register on the clock edge.

Reads are instant (combinational); writes wait for the clock. That asymmetry is key — next slides.

L07 • MODERN COMPUTER ARCHITECTURE

The Write Port: Decoder + Enable

An address decoder activates exactly one register's clock-enable.

HOW A WRITE WORKS

1. The write address goes into a 5-to-32 decoder.
2. The decoder raises exactly ONE register's enable line (one-hot).
3. If WE = 1, that one register captures wdata on the clock edge.
All other registers ignore the bus — their enables are 0.
L07 • MODERN COMPUTER ARCHITECTURE
The Read Ports: One Big MUX Each
Each read address drives a 32:1 MUX that selects one register onto the output.
WHY A MUX
Every register's output is wired to BOTH read MUXes at once.
The read address is the select code — it picks which of the 32 register outputs flows to that port.
Two independent MUXes = two independent reads, simultaneously. Recall the MUX from L3.
L07 • MODERN COMPUTER ARCHITECTURE
Two Reads + One Write, Every Cycle
The full picture: one instruction's worth of register access in a single diagram.
L07 • MODERN COMPUTER ARCHITECTURE
Reads Are Instant, Writes Wait for the Clock
This asymmetry is the single most important fact about a register file.
READ = COMBINATIONAL
No clock needed
Put an address on a read port...
...the data appears after just gate delay.
Both read ports work this way, at once.
So the ALU gets its operands immediately.
WRITE = SYNCHRONOUS
Happens on the clock edge
Address + data + WE are set up early...
...the register updates only at the edge.
This prevents races and half-written values.
The new value is readable next cycle.
Why it works: read the old values combinationaly at the start of a cycle, compute, then write the result at the edge — read-before-write, safely, in one cycle.
L07 • MODERN COMPUTER ARCHITECTURE
The Register File in a Real CPU
This isn't theory — it's the exact structure inside RISC-V and MIPS chips.
RISC-V: 32 registers
x0–x31, each 32 or 64 bits. x0 is hardwired to zero — reads always give 0, writes are ignored.
Always 2 read + 1 write
Matches R-type instructions (add rd, rs1, rs2): read two sources, write one destination per cycle.
Fastest storage tier
Register file → L1 cache → L2 → RAM. Registers are a fraction of a nanosecond away from the ALU.
Feeds the datapath
In Module 4, the read ports drive the ALU and the write port catches its result — the core loop.
L07 • MODERN COMPUTER ARCHITECTURE
Common Mistakes to Avoid
The slips that cost marks — and cause real bugs.
X
Clocking the read port
Register-file reads are combinational. Clocking them adds a needless cycle of latency.
X
Forgetting write-enable
Without WE, a register is overwritten every cycle. The decoder picks WHERE; WE picks WHETHER.
X
Resetting a detector on match
After '1011', the trailing '1' may start a new match. Go to the right prenex state, not S0.
X
Mixing Moore & Mealy notation
Moore: output in the circle (S1/1). Mealy: output on the arrow (in/out). Don't blend them.
X
Miscounting detector states
Moore needs a state per output value → often one more than the Mealy version. Count carefully.
X
Tri-state with two drivers
On a tri-state read bus, enabling two buffers at once causes bus contention — a hardware fault.
L07 • MODERN COMPUTER ARCHITECTURE
Key Takeaways
Five sentences — the CPU's storage and its controller.
01
A register file packs many registers (RISC: 32 × 32-bit) into one block with 2 read ports and 1 write port.
02
The write port uses a decoder + write-enable to update exactly one register on the clock edge (synchronous).
03
The read ports are MUX-based and combinational — two operands appear instantly, no clock needed.
04
An FSM = states + inputs + transitions + outputs, built as next-state logic → state register → output logic.
05
Moore output = f(state) (in the circle); Mealy output = f(state, input) (on the arrow) – Mealy often uses fewer states.
L07 • MODERN COMPUTER ARCHITECTURE
Practice & Homework
Design register files and state machines.
01

8×16 Register File
Design a register file with 8 registers of 16 bits. Show the write-port decoder logic (3→8).
02
Mealy '10' Detector
Draw the state diagram for a Mealy FSM that outputs 1 whenever it sees the pattern '10'.
03
Mealy → Moore
Convert a given 3-state Mealy FSM into an equivalent Moore FSM. How many states now?
EXIT TICKET
1. In a register file, why is the read combinational but the write synchronous?
2. How many states does a Moore '1011' detector need, and why one more than Mealy?
3. Trace the Mealy detector on input '101011' — when does the output pulse?
END OF L07
Storage, and the machine to run it.
NEXT — L08
Memory & the Memory Hierarchy — SRAM vs DRAM, how RAM is organised, and why your CPU waits on memory more than anything else.
DOCUMENT 10
Lecture: 1000
Architectures
From a 6-transistor cell to the memory wall — the trade-offs that shape every computer.
Why Computers Have So Many Kinds of Memory
4. Von Neumann
One memory holds both instructions and data, reached by a single bus. The stored-program idea that started it all (1945).
L09 • MODERN COMPUTER ARCHITECTURE
The Stored-Program Concept
The radical 1945 idea: keep the program IN memory, as numbers, right alongside the data.
BEFORE vs AFTER
Before: computers were 'wired' for one task — reprogramming meant rewiring by hand.
After (von Neumann): the program is stored in the same memory as data, encoded as numbers.
To run a different program, you just load different numbers. Software as we know it was born.
ONE MEMORY, TWO ROLES
0x00
lw r1, 0x40
instr
0x04
add r3, r1, r2
instr
0x08
sw r3, 0x44
instr
0x40
0x0000002A
data
0x44
0x00000000
data
Same memory, same address space — instructions and data live side by side.
L09 • MODERN COMPUTER ARCHITECTURE
Von Neumann Architecture
One memory, one bus — everything shares a single path between CPU and memory.
CPU
Control
ALU
Registers
MEMORY
instructions
+
data
SINGLE SHARED BUS
(address + data + control)
Instructions and data take turns on the ONE bus — the defining feature, and the built-in limit.
L09 • MODERN COMPUTER ARCHITECTURE
The Key Insight: Programs ARE Data
Because code lives in memory as numbers, it can be treated like any other data.
λ
Compilers & assemblers
A program that writes another program — it outputs instructions as data into memory.
Loading & the OS
The OS loads a program by copying its instruction-bytes into memory, then jumps to them.
Just-in-time compilation
JITs (JVM, browsers) generate machine code at runtime and then execute it — data becomes code.
Self-modifying code
Programs can, in principle, rewrite their own instructions — powerful, and a security minefield.
This blurring of code and data is von Neumann's superpower — and

the root of whole classes of security exploits.
5. The Von Neumann Bottleneck
One bus, two jobs: instructions and data can't both cross at once. This single limit shapes modern architecture.
L09 • MODERN COMPUTER ARCHITECTURE
The Von Neumann Bottleneck
Every instruction fetch and every data access share ONE bus — so they must take turns.
CPU
MEMORY
(code + data)
ONE BUS
fetch instruction OR load/store data
— never both in the same cycle
The CPU can compute far faster than the single bus can feed it — so the bus becomes the limit.
This traffic jam on the memory bus is 'the Von Neumann bottleneck' — the term coined by John Backus in 1977.
L09 • MODERN COMPUTER ARCHITECTURE
Why It Limits Performance
A fast CPU stalls whenever the bus is busy — computation waits on communication.
Bus contention
Instruction fetch and data access compete for the same wires. One must wait for the other.
Idle compute units
While the ALU waits for the bus, it does nothing. Raw compute power sits unused.
Caps achievable IPC
Instructions-per-cycle is limited not by the ALU but by how fast the bus delivers work.
Worsens with CPU speed
Faster CPUs need data faster — but the bus doesn't scale as quickly. The gap grows.
The bottleneck is why caches, wider buses, and Harvard-style splits exist — all attempts to widen this pinch.
L09 • MODERN COMPUTER ARCHITECTURE
Why It Gets Worse Over Time
CPU speed raced ahead of memory bandwidth for decades — the bottleneck tightened.
THE RESPONSE
Architects fought back with:
• Caches — keep hot code & data on-chip (Module 5).
• Wider & faster buses — move more bits per cycle.
• Harvard-style splits — separate paths for code and data.
The rest of today — and much of this course — is about that last idea.
L09 • MODERN COMPUTER ARCHITECTURE
Quick Check
Show of hands — no notes.
Q. The Von Neumann bottleneck exists because...
A
The ALU is too slow
One bus is shared
by code & data
✓ they take turns on one path
C
Memory is too small
D
There are too many registers
The single shared bus means an instruction fetch and a data access can't happen simultaneously — that's the bottleneck.
B
3. Harvard Architecture
Separate memories and buses for instructions and data — so the CPU can fetch and load at the same time.
L09 • MODERN COMPUTER ARCHITECTURE
Harvard Architecture
Two memories, two buses — instructions and data never compete for the same path.
INSTRUCTION MEMORY
DATA MEMORY
CPU
Control
ALU
I-BUS
D-BUS
Instruction fetch (I-bus) and data access (D-bus) happen in the SAME cycle — no contention.
L09 • MODERN COMPUTER ARCHITECTURE
Two Buses, Twice the Throughput
The whole point: parallel access. What took two cycles now takes one.
VON NEUMANN
one bus, take turns
Cycle 1: fetch instruction.
Cycle 2: access data.
Two cycles for fetch + data.
Bus is the bottleneck.
HARVARD
two buses, in parallel
Cycle 1: fetch instruction AND access data.
Both buses active at once.
One cycle for fetch + data.
Up to 2× memory throughput.
The cost: two memories and two bus systems — more wires, more pins, more silicon. Nothing is free.
L09 • MODERN COMPUTER ARCHITECTURE
One Cycle, Two Transfers
A timeline makes the advantage obvious — Harvard overlaps what Von Neumann serialises.

VON NEUMANN
FETCH instruction
ACCESS data
cycle 1
cycle 2
→ 2 cycles
HARVARD
FETCH instruction
ACCESS data
cycle 1 (both at once)
→ 1 cycle
Same work, half the time — because the two transfers ride separate buses simultaneously.
L09 · MODERN COMPUTER ARCHITECTURE
Where Harvard Shines
When predictable, high-throughput access matters more than flexibility.
Digital Signal Processors
DSPs stream data through mixed algorithms (filters, FFTs). Parallel code+data fetch keeps the pipeline fed.
Microcontrollers
The ATmega328 (Arduino Uno) is true Harvard — flash for code, SRAM for data, separate buses.
Real-time systems
Deterministic timing matters. No bus contention means predictable, repeatable cycle counts.
Embedded & IoT
Small, mixed programs in mixed + small data in RAM map naturally onto the Harvard split.
L09 · MODERN COMPUTER ARCHITECTURE
Von Neumann vs Harvard
The trade-off in one table: simplicity & flexibility vs throughput & predictability.
VON NEUMANN
HARVARD
Memory
One (unified)
Two (separate I & D)
Buses
One shared
Two independent
Fetch + data
Sequential (2 cycles)
Simultaneous (1 cycle)
Throughput
Bottlenecked
Up to 2x higher
Cost / complexity
Lower
Higher (more wires/pins)
Flexibility
High (code = data)
Lower (separate spaces)
Typical use
General-purpose CPUs
DSPs, microcontrollers
Neither is 'better' — general CPUs pick flexibility, embedded/DSP picks throughput. Modern chips blend both...
6. Modified Harvard
What your laptop actually is: Von Neumann at main memory, Harvard inside the chip. The best of both.
L09 · MODERN COMPUTER ARCHITECTURE
The Real World: Modified Harvard
Modern CPUs are Von Neumann on the outside, Harvard on the inside.
TWO LEVELS, TWO ANSWERS
Outside the chip: one unified DRAM holds code and data — pure Von Neumann. Simple, flexible, cheap.
Inside the chip: the L1 cache is SPLIT into a separate instruction cache (L1-I) and data cache (L1-D) — Harvard-style.
So the CPU fetches code and data in parallel from L1, but everything still lives in one memory underneath.
BEST OF BOTH
Flexibility of Von Neumann:
one address space, code = data, easy to program & load.
Speed of Harvard:
split L1 caches feed the pipeline code + data every cycle, no contention.
This hybrid is why the split L1 exists — a direct preview of

Module 3.

Split L1-I and L1-D Caches
Where the two worlds meet: unified memory below, split caches above.
CPU
CORE
L1-I
(instructions)
L1-D
(data)
L2 / L3
(unified)
DRAM
(unified code+data)
← HARVARD ZONE →
← VON NEUMANN ZONE →
Split at L1 for parallel fetch; unified from L2 down for simplicity. Modified Harvard in one picture.
L09 · MODERN COMPUTER ARCHITECTURE
Why This Hybrid Wins

It captures each architecture's strength exactly where that strength matters most.
Parallel L1 access
Split L1-I/L1-D lets the pipeline fetch an instruction and a data word every cycle — Harvard speed where it counts.
Unified main memory
One DRAM address space keeps loading, linking, and programming simple — Von Neumann flexibility.
Cache coherence handled
Hardware keeps the split caches consistent with unified memory, so software sees one memory.
★
Scales to real workloads
General-purpose speed AND generality — which is why essentially every modern CPU does this.
L08 · MODERN COMPUTER ARCHITECTURE
Key Takeaways
Five sentences — and Module 2 is complete.
01
SRAM stores a bit in a 6-transistor latch: fast, non-destructive, no refresh — but bulky and expensive.
02
DRAM stores a bit as charge on a capacitor (1T1C): tiny and cheap and dense — but slow, destructive, and needs refresh.
03
SRAM builds caches (L1/L2/L3); DRAM builds main memory — each chosen for its trade-off, not because one is 'better'.
04
The memory hierarchy layers small-fast over big-slow, and locality lets a small cache catch most accesses.
05
The memory wall — CPUs outpacing DRAM for decades — is the problem the whole memory hierarchy exists to fight.
L08 · MODERN COMPUTER ARCHITECTURE
Practice & Homework
Draw the cells, place the caches, and reason about the numbers.
01
Cell Sketch
Draw a 6T SRAM cell and a 1T1C DRAM cell. Label transistors, capacitor, bit lines, word line.
02
Refresh Math
A DRAM row refreshes every 64 ms with 8192 rows. On average, how often must one row be refreshed?
03
Plot It
Plot access time vs capacity (log-log) for registers, L1, L2, L3, DRAM, SSD, HDD.
EXIT TICKET
1. List three reasons SRAM is faster than DRAM.
2. Why must DRAM be refreshed but SRAM need not be?
3. Order these by latency: L2, DRAM, registers, L1, SSD.
L08 · MODERN COMPUTER ARCHITECTURE
Rapid-Fire Recap
Three questions — shout the answers.
Q1
An SRAM cell uses how many transistors?
6
Q2
DRAM needs periodic...
refresh
Q3
Caches are built from...
SRAM
All three instant? You've completed Module 2 — Module 3 turns to the instruction set: the CPU's vocabulary.
END OF L08 · MODULE 2 COMPLETE
Fast, big, cheap — pick two.
NEXT — MODULE 3, L09
Computer Organization & the Instruction Set — how a program becomes machine instructions, and the vocabulary a CPU actually understands.

DOCUMENTS

LECTURE: 1010
MOX • DEMO • Decoders • Adders
The small library of parts every CPU is assembled from.
L10 · MODERN COMPUTER ARCHITECTURE
We Have the Machine — Now the Language
L9 organised CPU + memory. But what commands can the CPU actually understand?
The organisation
Von Neumann / Harvard — how CPU, memory & buses connect (L9).
?
The missing piece
What set of commands does the hardware accept? That's the ISA.
The ISA
Instruction Set Architecture — the CPU's vocabulary and the HW/SW contract.
The philosophies
CISC (rich instructions) vs

RISC (simple instructions). Today's battle.
Every program you write ends up as ISA instructions. The ISA's design shapes compilers, chips, and speed.
L10 · MODERN COMPUTER ARCHITECTURE
Today's Roadmap
Two philosophies, their history, and the surprising truth about modern chips.
1
The Two Ideas
CISC: rich, variable instructions. RISC: simple, mixed ones. The core trade-off.
→
2
History & Compilers
VAX → MIPS → ARM → RISC-V, and what each choice costs the compiler.
→
3
Modern Reality
x86 secretly runs RISC-like micro-ops; RISC-V opens the ISA to everyone.
The big reveal: the RISC-vs-CISC 'war' ended in a truce — modern CPUs borrow from both. But the ideas still shape every design.
L09 · MODERN COMPUTER ARCHITECTURE
From Gates to a Complete Machine
A computer is more than an ALU — it needs memory, I/O, and a way to wire them together.
CPU
(compute)
MEMORY
(store)
I/O
(talk to world)
BUS
(connect)
The question of computer organisation: how exactly do these connect — and how many buses? The answer shapes everything: performance, cost, and how you program the machine. Two classic blueprints follow.
4. Von Neumann
One memory holds both instructions and data, reached by a single bus. The stored-program idea that started it all (1945).
L09 · MODERN COMPUTER ARCHITECTURE
The Stored-Program Concept
The radical 1945 idea: keep the program IN memory, as numbers, right alongside the data.
BEFORE vs AFTER
Before: computers were 'wired' for one task — reprogramming meant rewiring by hand.
After (von Neumann): the program is stored in the same memory as data, encoded as numbers.
To run a different program, you just load different numbers. Software as we know it was born.
ONE MEMORY, TWO ROLES
0x00
lw r1, 0x40
instr
0x04
add r3, r1, r2
instr
0x08
sw r3, 0x44
instr
0x40
0x0000002A
data
0x44
0x00000000
data
Same memory, same address space — instructions and data live side by side.
L09 · MODERN COMPUTER ARCHITECTURE
Von Neumann Architecture
One memory, one bus — everything shares a single path between CPU and memory.
CPU
Control
ALU
Registers
MEMORY
instructions + data
SINGLE SHARED BUS
(address + data + control)
Instructions and data take turns on the ONE bus — the defining feature, and the built-in limit.
L09 · MODERN COMPUTER ARCHITECTURE
The Key Insight: Programs ARE Data
Because code lives in memory as numbers, it can be treated like any other data.
A
Compilers & assemblers
A program that writes another program — it outputs instructions as data into memory.
Loading & the OS
The OS loads a program by copying its instruction-bytes into memory, then jumps to them.
Just-in-time compilation
JITs (JVM, browsers) generate machine code at runtime and then execute it — data becomes code.
Self-modifying code
Programs can, in principle, rewrite their own instructions — powerful, and a security minefield.
This blurring of code and data is von Neumann's superpower — and the root of whole classes of security exploits.

5. The Von Neumann

Bottleneck

One bus, two jobs: instructions and data can't both cross at once. This single limit shapes modern architecture. **L09 · MODERN COMPUTER ARCHITECTURE**
The Von Neumann Bottleneck

Every instruction fetch and every data access share ONE bus — so they must take turns.

CPU

MEMORY

(code + data)

ONE BUS

fetch instruction OR load/store data

— never both in the same cycle

■ The CPU can compute far faster than the single bus can feed it — so the bus becomes the limit.

This traffic jam on the memory bus is 'the Von Neumann bottleneck' — the term coined by John Backus in 1977.

L09 · MODERN COMPUTER ARCHITECTURE

Why It Limits Performance

A fast CPU stalls whenever the bus is busy — computation waits on communication.

Bus contention

Instruction fetch and data access compete for the

same wires. One must wait for the other.

Idle compute units

While the ALU waits for the bus, it does nothing.

Raw compute power sits unused.

Caps achievable IPC

Instructions-per-cycle is limited not by the ALU

but by how fast the bus delivers work.

Worsens with CPU speed

Faster CPUs need data faster — but the bus

doesn't scale as quickly. The gap grows.

The bottleneck is why caches, wider buses, and Harvard-style splits exist — all attempts to widen this pinch.

L09 · MODERN COMPUTER ARCHITECTURE

Why It Gets Worse Over Time

CPU speed raced ahead of memory bandwidth for decades — the bottleneck tightened.

THE RESPONSE

Architects fought back with:

- Caches — keep hot code & data on-chip (Module 5).
- Wider & faster buses — move more bits per cycle.
- Harvard-style splits — separate paths for code and data.

The rest of today — and much of this course — is about that last idea.

3. Harvard Architecture

Separate memories and buses for instructions and data — so the CPU can

fetch and load at the same time.

L09 · MODERN COMPUTER ARCHITECTURE

Harvard Architecture

Two memories, two buses — instructions and data never compete for the same path.

INSTRUCTION

MEMORY

DATA

MEMORY

CPU

Control

ALU

I-BUS

D-BUS

Instruction fetch (I-bus) and data access (D-bus) happen in the SAME cycle — no contention.

L09 · MODERN COMPUTER ARCHITECTURE

Two Buses, Twice the Throughput

The whole point: parallel access. What took two cycles now takes one.

VON NEUMANN

one bus, take turns

Cycle 1: fetch instruction.

Cycle 2: access data.

Two cycles for fetch + data.

Bus is the bottleneck.

HARVARD

two buses, in parallel

Cycle 1: fetch instruction AND access data.

Both buses active at once.

One cycle for fetch + data.

Up to 2× memory throughput.

The cost: two memories and two bus systems — more wires, more pins, more silicon. Nothing is free.

L09 · MODERN COMPUTER ARCHITECTURE

One Cycle, Two Transfers

A timeline makes the advantage obvious — Harvard overlaps what Von Neumann serialises.

VON NEUMANN

FETCH instruction

ACCESS data

cycle 1

cycle 2

→ 2 cycles

HARVARD

FETCH instruction

ACCESS data

cycle 1 (both at once)

→ 1 cycle

Same work, half the time — because the two transfers ride separate buses simultaneously.

L09 · MODERN COMPUTER ARCHITECTURE

Where Harvard Shines

When predictable, high-throughput access matters more than flexibility.

Digital Signal Processors

DSPs stream data through fixed algorithms

(filters, FFTs). Parallel code+data fetch keeps the pipeline fed.

Microcontrollers

The ATmega328 (Arduino Uno) is true Harvard —

flash for code, SRAM for data, separate buses.

Real-time systems

Deterministic timing matters. No bus contention

means predictable, repeatable cycle counts.

Embedded & IoT

Small, fixed programs in flash + small data in RAM

map naturally onto the Harvard split.

L09 · MODERN COMPUTER ARCHITECTURE

Von Neumann vs Harvard

The trade-off in one table: simplicity & flexibility vs throughput & predictability.

VON NEUMANN

HARVARD

Memory

One (unified)

Two (separate I & D)

Buses

One shared

Two independent

Fetch + data

Sequential (2 cycles)

Simultaneous (1 cycle)

Throughput

Bottlenecked

Up to 2× higher

Cost / complexity

Lower

Higher (more wires/pins)

Flexibility

High (code = data)

Lower (separate spaces)

Typical use

General-purpose CPUs

DSPs, microcontrollers

Neither is 'better' — general CPUs pick flexibility, embedded/DSP picks throughput. Modern chips blend both...

6. Modified Harvard

What your laptop actually is: Von Neumann at main memory, Harvard inside

the chip. The best of both.

L09 · MODERN COMPUTER ARCHITECTURE

The Real World: Modified Harvard

Modern CPUs are Von Neumann on the outside, Harvard on the inside.

TWO LEVELS, TWO ANSWERS

Outside the chip: one unified DRAM holds code and data

—

pure Von Neumann. Simple, flexible, cheap.

Inside the chip: the L1 cache is SPLIT into a separate instruction cache (L1-I) and data cache (L1-D) —

Harvard-style.

So the CPU fetches code and data in parallel from L1, but everything still lives in one memory underneath.

BEST OF BOTH

Flexibility of Von Neumann:

one address space, code = data, easy to program & load.

Speed of Harvard:

split L1 caches feed the pipeline code + data every cycle, no contention.

This hybrid is why the split L1 exists — a direct preview of

Module 5.

Split L1-I and L1-D Caches

Where the two worlds meet: unified memory below, split caches above.

CPU

CORE

L1-I

(instructions)

L1-D

(data)

L2 / L3

(unified)

DRAM

(unified code+data)

← HARVARD ZONE →

← VON NEUMANN ZONE →

Split at L1 for parallel fetch; unified from L2 down for simplicity. Modified Harvard in one picture.

L09 · MODERN COMPUTER ARCHITECTURE

Why This Hybrid Wins

It captures each architecture's strength exactly where that strength matters most.

Parallel L1 access

Split L1-I/L1-D lets the pipeline fetch an instruction and a data word every cycle — Harvard speed where it counts.

Unified main memory

One DRAM address space keeps loading, linking, and programming simple — Von Neumann flexibility.

Cache coherence handled

Hardware keeps the split caches consistent with unified memory, so software sees one memory.

Scales to real workloads

General-purpose speed AND generality — which is why essentially every modern CPU does this.

L10 · MODERN COMPUTER ARCHITECTURE

The ISA: Hardware/Software Contract

The Instruction Set Architecture is the agreed interface — the boundary the whole industry builds around.

SOFTWARE · compilers, OS, your programs

◆ THE ISA — the instruction set ◆

HARDWARE · the CPU that implements it

everything above

compiles TO the ISA

everything below

EXECUTES the ISA

Why it's powerful: as long as both sides honour the ISA, hardware and software teams can innovate independently. Intel can redesign a chip and your old programs still run. The ISA is the promise that makes that possible.

PART 1 · L10

1

1. Two Philosophies

CISC packs power into each instruction; RISC keeps every instruction simple. One

trades hardware complexity for the other's compiler complexity.

L10 · MODERN COMPUTER ARCHITECTURE

CISC: Complex Instruction Set

Make each instruction do a lot — fewer lines of assembly, richer operations.

THE PHILOSOPHY

One instruction can do a whole task — even

memory-to-memory arithmetic in a single line.

Instructions are variable-length: simple ones are short,

complex ones are long.

Born when memory was scarce and expensive — fewer, denser

instructions saved precious space.

One CISC instruction:

add [mem1], [mem2]

CHARACTER

Hundreds of instructions

Variable length (1–15 bytes on x86)

Instructions can touch memory directly

Complex hardware decoder

Fewer lines of assembly

e.g. x86, VAX, 68000

L10 · MODERN COMPUTER ARCHITECTURE

RISC: Reduced Instruction Set

Keep every instruction simple and uniform — let the compiler combine them.

THE PHILOSOPHY

Each instruction does ONE simple thing, in one cycle where possible.

All instructions are the same fixed length — easy to fetch,

decode, and pipeline.

Only load and store touch memory — everything else works on

registers (load/store architecture).

Same work, RISC style:

lw r1,m1 · lw r2,m2 · add r1,r1,r2 · sw r1,m1

CHARACTER

Few, simple instructions

Fixed length (e.g. 32 bits)

Only load/store touch memory

Simple, fast hardware decoder

More instructions per task

e.g. MIPS, ARM, RISC-V, SPARC

L10 · MODERN COMPUTER ARCHITECTURE

One Task, Two Styles (Activity 1)

Compute C = A + B; D = C * 2 — count the instructions each way.

CISC STYLE

mov ax, [A]

add ax, [B] ; ax = A + B

mov [C], ax

shl ax, 1 ; ax = C * 2

mov [D], ax

~5 instructions · richer ops · direct memory operands

RISC STYLE

lw r1, A

lw r2, B

add r3, r1, r2 # C = A + B

sw r3, C

add r4, r3, r3 # D = C * 2

sw r4, D

~6 instructions · each simple & 1 cycle · load/store only

L10 · MODERN COMPUTER ARCHITECTURE

Instruction Encoding

Fixed-width vs variable-width — the choice that ripples through the whole chip.

RISC — FIXED 32-BIT (every instruction identical width)

opcode

rd

rs1

rs2

funct

opcode

rd

rs1

immediate

Same length every time → trivial to find where the next instruction starts.

CISC — VARIABLE LENGTH (1 to 15 bytes on x86)

op

prefix

op

modRM

prefix

op

modRM

disp

imm

Different lengths → the CPU must decode one instruction before it knows where the next begins.

This single difference is why RISC pipelines so much more easily —

predictable fetch and decode.

L10 • MODERN COMPUTER ARCHITECTURE
The Decoder Tells the Story

Where the complexity lives is the whole difference — hardware vs compiler.

CISC DECODER
complexity in HARDWARE

Must handle variable-length instructions.
Often uses microcode — a mini-program per instruction.
Large, power-hungry, hard to pipeline.
But: shorter programs, denser code.

RISC DECODER
complexity in the COMPILER

Fixed length → decode is almost trivial.
One instruction = one simple action.

Small, fast, easy to pipeline & replicate.
But: the compiler works harder to sequence more instructions.

The trade never disappears — it just moves. CISC asks more of the chip; RISC asks more of the compiler.

L10 • MODERN COMPUTER ARCHITECTURE
1990s–2000s: Two Empires

RISC conquered mobile; CISC held the desktop. Both thrived — for different reasons.

ARM — RISC wins mobile

Simple = low power = long battery life.
Licensable design → everyone builds ARM chips.

Billions of phones, tablets, embedded devices.
Now in Apple Silicon laptops too.

x86 — CISC holds desktop

Backward compatibility: decades of software.
Intel & AMD poured R&D into raw speed.
The whole PC & server ecosystem locked in.
Still dominant in desktops & data centres.
The lesson: 'best ISA' rarely decides the winner — ecosystems, power budgets, and compatibility do.

L10 • MODERN COMPUTER ARCHITECTURE
The ISA Family Tree

A rough map of who came from where — and which philosophy they follow.

CISC
x86 (1978)
x86-64 (2003)
VAX
68000
RISC
IBM 801 → MIPS
SPARC
ARM (1985)
RISC-V (2010)

Two lineages, one lesson: the split defined 40 years of chips — but the next slides show how they've converged.

PART 3 • L10
3

3. Compilers & Code Density

The philosophy choice lands hardest on two things: how compact your code is, and how easily the chip can pipeline it.

L10 • MODERN COMPUTER ARCHITECTURE
Code Density: CISC's Edge

Fewer, richer instructions mean smaller programs — which eases pressure on the instruction cache.

WHY DENSITY MATTERS

CISC programs are often smaller in bytes — one instruction does more work.
Smaller code = fewer bytes to fetch, and less pressure on the

L1 instruction cache (recall L8).
In memory-constrained or cache-sensitive settings, density is a real, measurable advantage.

PROGRAM SIZE (illustrative)

CISC
smaller
RISC
more instrs

But it's not the whole story:
RISC's fixed 32-bit instructions can waste some space, yet uniformity buys huge pipelining gains. Compressed RISC ISAs (ARM Thumb, RISC-V C) close much of the density gap.

L10 • MODERN COMPUTER ARCHITECTURE
Pipelining: RISC's Edge

Uniform instructions flow through a pipeline like clockwork — the reason RISC dominates performance-per-watt.

■ **Fixed length = easy fetch**
The CPU always knows where the next instruction is — no decode needed ■**rst**. Perfect for a pipeline.

■ **Uniform format = simple decode**
Every instruction decodes the same way, in one stage, at one speed.

■ **One cycle each = smooth flow**
Simple ops rarely stall the pipeline, so stages stay full and throughput stays high.

■ **Load/store = predictable memory**
Only two instructions touch memory, so hazards are easier to detect and handle (Module 4).
Pipelining is the heart of Module 4 — and RISC's uniformity is exactly what makes it clean. Remember this slide.

L10 • MODERN COMPUTER ARCHITECTURE
The Trade-Off in One View

Every RISC strength has a matching CISC strength — it's genuinely a balance.

CISC STRENGTHS

- ✓ **Denser code (smaller programs)**
- ✓ **Fewer instructions per task**
- ✓ **Less instruction-fetch bandwidth**

- ✓ **Rich, expressive operations**

RISC STRENGTHS

- ✓ **Simple, fast, cheap decoder**
- ✓ **Easy, efficient pipelining**
- ✓ **Higher clock speeds possible**
- ✓ **Simpler to design & verify**

Because both lists are real, the industry didn't pick a winner — it learned to combine them. That's next.

PART 4 • L10
4

4. The Modern Truth

The war is over. Inside every modern x86 chip beats a RISC-like heart — the two philosophies have merged.

L10 • MODERN COMPUTER ARCHITECTURE
The Secret Inside x86: Micro-Ops

Modern x86 chips are CISC on the outside, RISC on the inside.

CISC instruction (e.g. x86 add mem)
DECODER (splitter)
µop: load
µop: add
µop: store
RISC-like core

The front-end decodes each complex x86 instruction into several simple micro-ops (µops)...

...then the back-end pipelines and renames those µops exactly like a RISC machine. Best of both, in one chip.

L10 • MODERN COMPUTER ARCHITECTURE
The Boundary Has Blurred

Both camps borrowed each other's best ideas until the labels stopped meaning much.

→ **CISC went RISC inside**
x86 decodes to RISC-like µops, then pipelines and reorders them. The CISC is just a compatibility skin.

← **RISC added complexity**
Modern ARM/RISC-V have hundreds of instructions, SIMD, and vector extensions — not so 'reduced' anymore.

■ **Compressed instructions**
ARM Thumb and RISC-V 'C' add 16-bit encodings for density — a very CISC-like concern.

◆ **It's about implementation now**
What matters today is the microarchitecture — caches, pipelines, predictors — more than the ISA label.

L10 • MODERN COMPUTER ARCHITECTURE
So Who Won?

Both — and neither. The pragmatic answer is what every modern high-performance chip does.
The modern consensus: a RISC-like execution core is the way to go for performance — so keep a clean, pipelineable core, and put whatever instruction 'skin' the market needs on the front.

x86 **keeps CISC skin for compatibility, RISC core** inside
ARM
RISC through and through — efficiency king
RISC-V
clean RISC, open to all, modular extensions

PART 5 • L10
5

5. Where It's Heading

Open ISAs and domain-specific accelerators are rewriting the rules — a preview of

Module 6.

RISC-V: the Open ISA

For the first time, a serious instruction set that anyone can use, free — no license, no royalties.

◆ **Open & royalty-free**
Anyone — students, startups, giants — can design a RISC-V chip without paying anyone. Unprecedented.

■ **Modular by design**
A small base ISA plus optional extensions (multiply, float, vector, atomic). Take only what you need.

■ **Exploding adoption**
From university teaching to microcontrollers to data-centre and AI chips — growth is rapid.

★ **Born at Berkeley**
Created in 2010 by the same research lineage that gave us RISC in the first place.
We'll use RISC-V assembly in the coming lectures — its clean design makes it the perfect teaching ISA.

L10 • MODERN COMPUTER ARCHITECTURE
Beyond CPUs: Domain-Specific ISAs

The newest frontier — instruction sets built for one job, especially AI.

■ **GPU ISAs (NVIDIA PTX)**
Thousands of simple cores with their own instruction set, built for massive parallelism.

◆ **TPU / AI accelerators**
Google's TPU has an ISA centred on matrix multiply — the core operation of neural networks.

■ **Why specialise?**
A fixed workload lets you strip away generality and win

huge efficiency — the opposite of general-purpose.

→ **The Module 6 preview**
Multicore, GPUs, and AI accelerators are where architecture is headed. This is the door to it.

L10 • MODERN COMPUTER ARCHITECTURE
Common Mistakes to Avoid

The slips that cost marks — and reveal shaky understanding.

X **'RISC is always faster'**
Not inherently. RISC eases pipelining, but real speed depends on the whole microarchitecture.

X **'CISC is obsolete'**
x86 still dominates desktops & servers. It just runs RISC-like µops inside.

X **'RISC = fewer instructions run'**
RISC has fewer instruction TYPES, but often runs MORE instructions per task.

X **Forgetting load/store**
The load/store rule is RISC's defining feature — only load & store touch memory.

X **'The ISA determines speed'**
The implementation (pipeline, caches, predictors) matters far more than the ISA label.

X **Mixing up length rules**
RISC = fixed length. CISC = variable length. This drives decoder complexity.

L10 • MODERN COMPUTER ARCHITECTURE
Key Takeaways

Five sentences — the two philosophies and their truce.

01 The ISA is the hardware/software contract — the instruction set both compilers and chips agree to.

02 CISC uses rich, variable-length instructions (x86, VAX): dense code, complex decoder, harder to pipeline.

03 RISC uses simple, fixed-length, load/store instructions (MIPS, ARM, RISC-V): easy to pipeline, simpler chips.

04 The trade-off just moves complexity between hardware and compiler — neither philosophy escapes it.

05 Modern x86 decodes CISC into RISC-like micro-ops; RISC-V opens the ISA to all — the two worlds have merged.

L10 • MODERN COMPUTER ARCHITECTURE
Practice & Homework

Write assembly, compare ISAs, and debate.

01 **Vector Add**
Write a 3-element vector add in (a) x86-style CISC and (b) RISC-V style. Count instructions.

02 **Why Pipeline?**
List 3 reasons RISC is friendlier to pipelining. Tie each to a concrete instruction property.

03 **Mini Debate**
Two teams: 'RISC won' vs 'CISC won'. 5 minutes of arguments each — then we debrief.

EXIT TICKET

1. Name the two hallmarks of RISC (hint: instruction length + memory access).
2. What are micro-ops, and why does x86 use them?
3. Why is Apple able to use ARM (RISC) for laptops while x86 still thrives on the desktop?

L10 • MODERN COMPUTER ARCHITECTURE
Rapid-Fire Recap

Three questions — shout the answers.

Q1
RISC instructions are what length?
fixed

Q2
In RISC, only these touch memory:
load / store

Q3
x86 decodes instructions into... micro-ops
All three instant? You understand the ISA landscape — next we get concrete with instruction formats & assembly.

■ **END OF L10**
Simple or rich?
The answer is: both.
NEXT — L11

Instruction Formats & Addressing Modes — how an instruction is actually encoded in bits, and the ways it can name its operands.

Modern Computer Architecture • Anuj Kumar Jha • Questions?